

Transformers

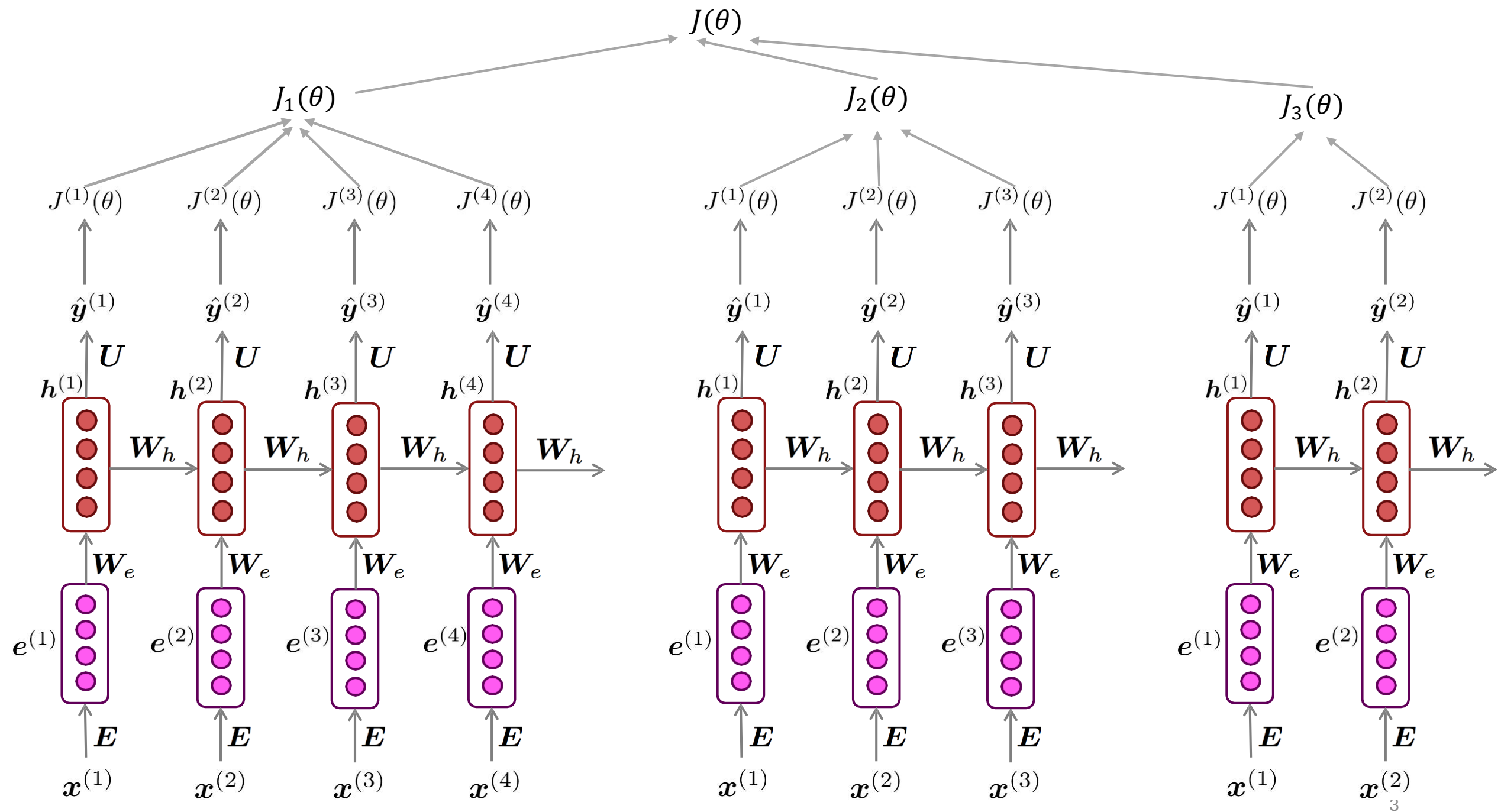
Mausam

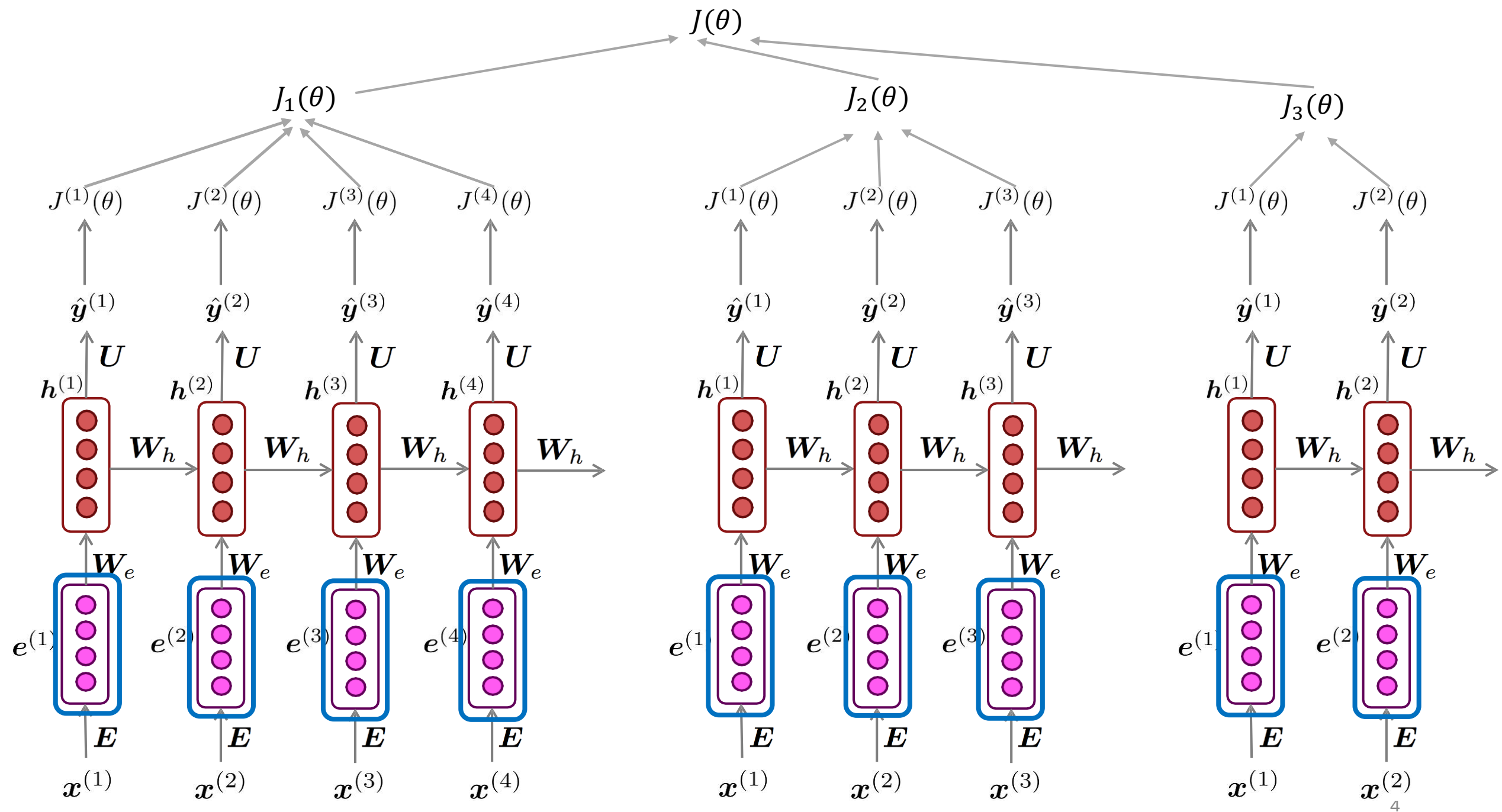
(Some slides by Tanmoy Chakraborty)

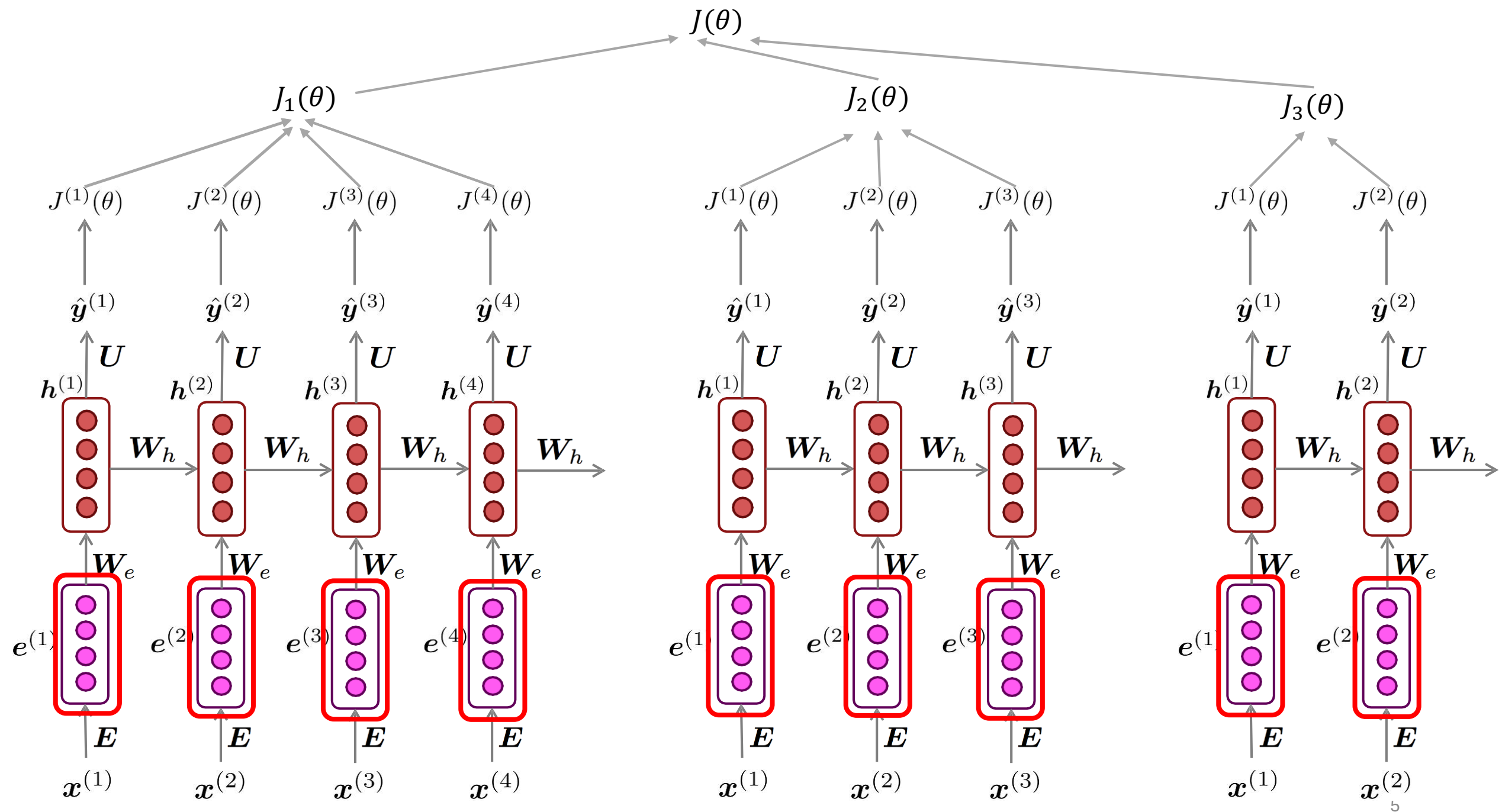


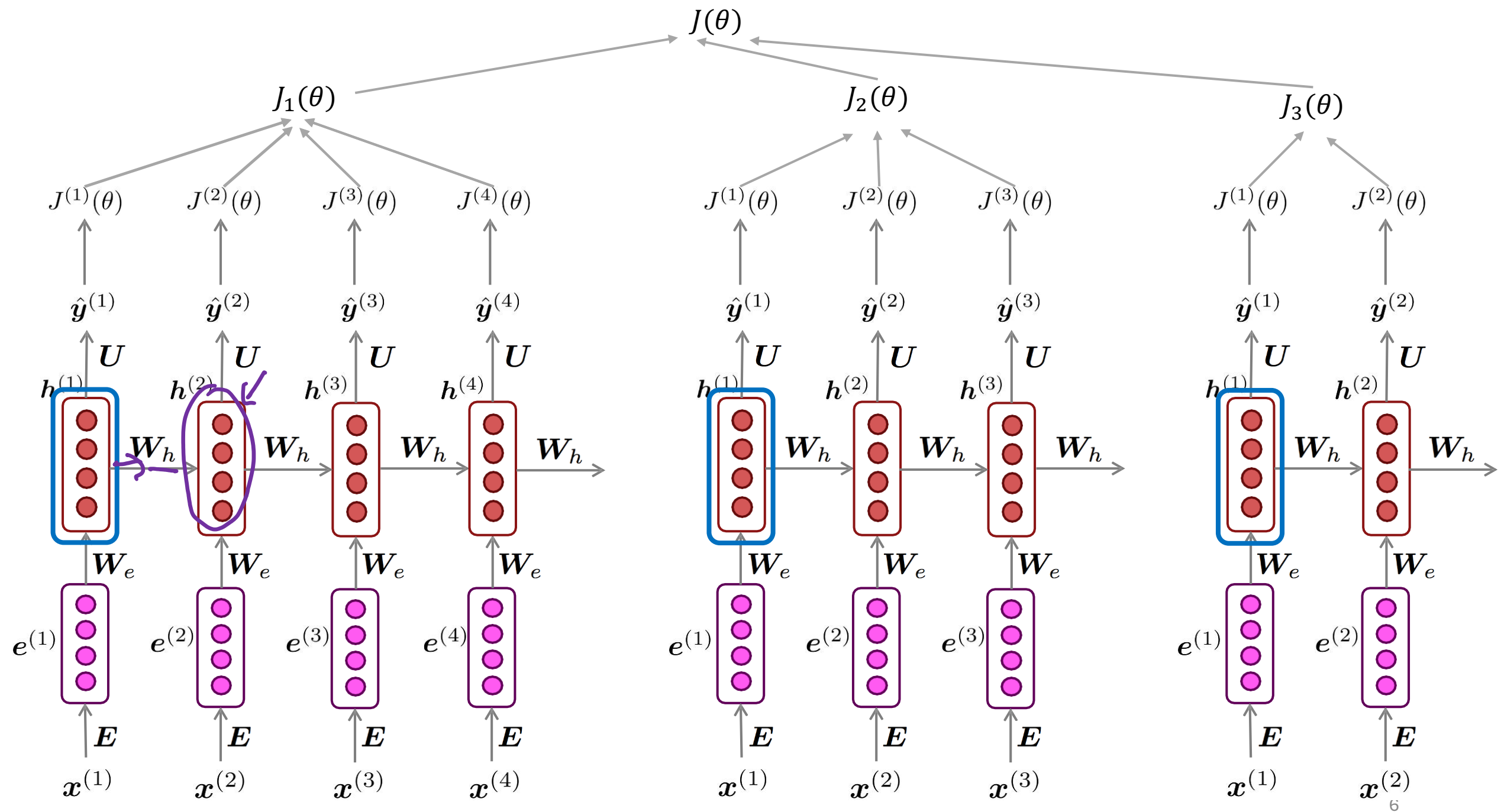
Neural Language Models

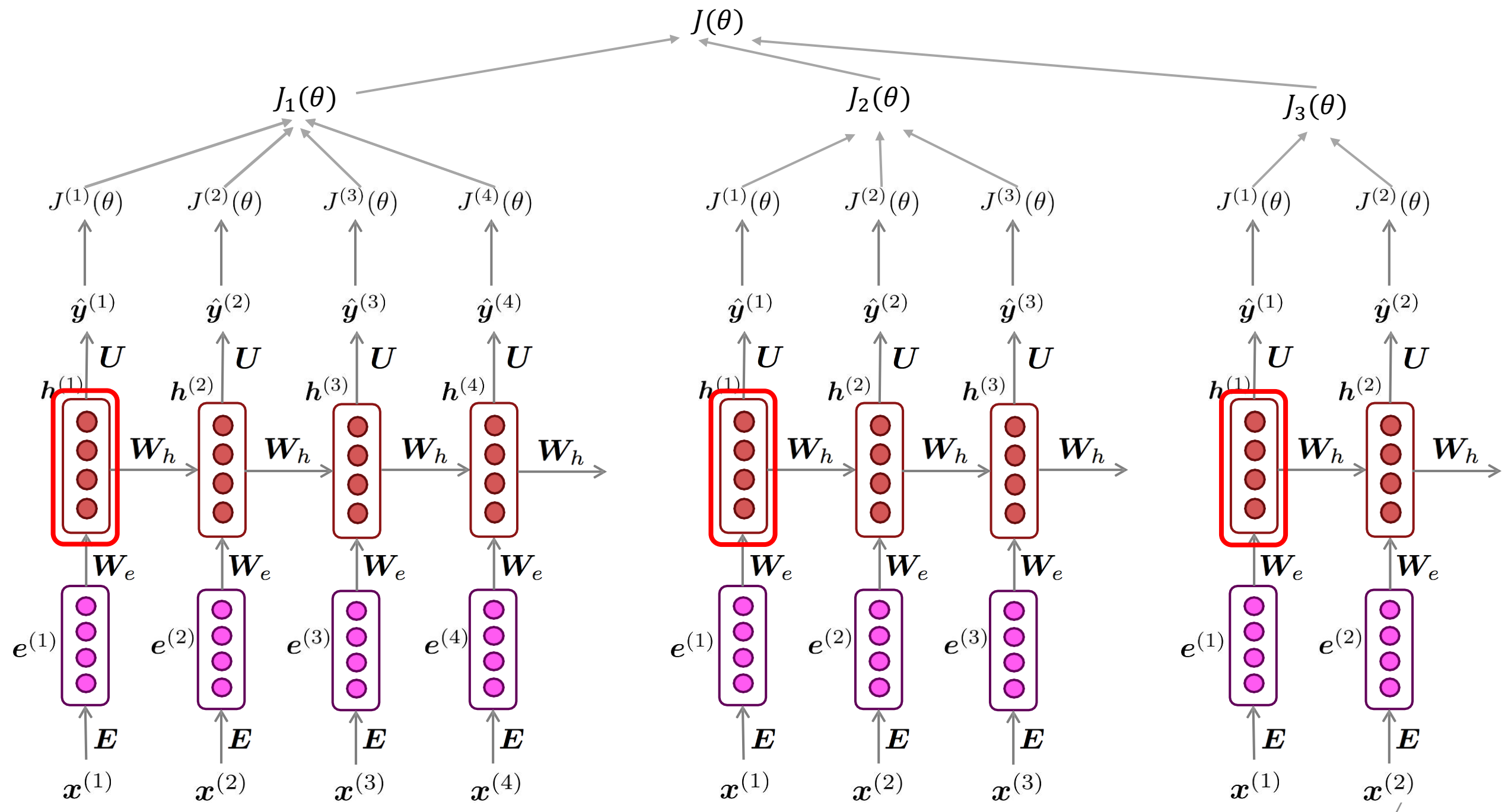
- Where are We?
 - Sparsity is mitigated by the use of dense vector representations
 - Prefix length is no longer constant due to the use RNNs/LSTMs
- Challenges Remaining
 - Hard to learn long-range dependencies: Vanishing Gradients
 - Forward and backward passes are $O(\text{sequence length})$
 - Hard to parallelize

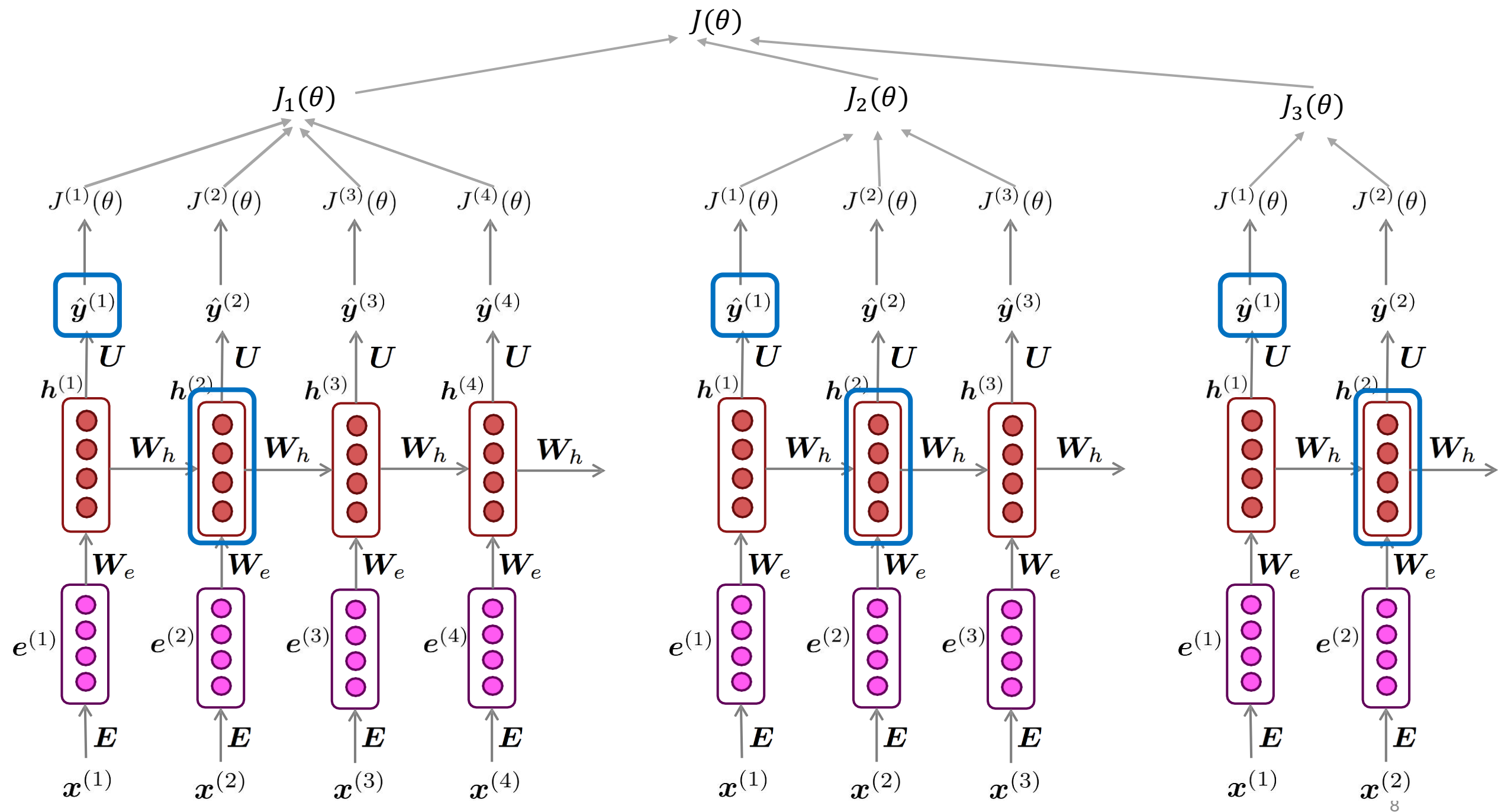


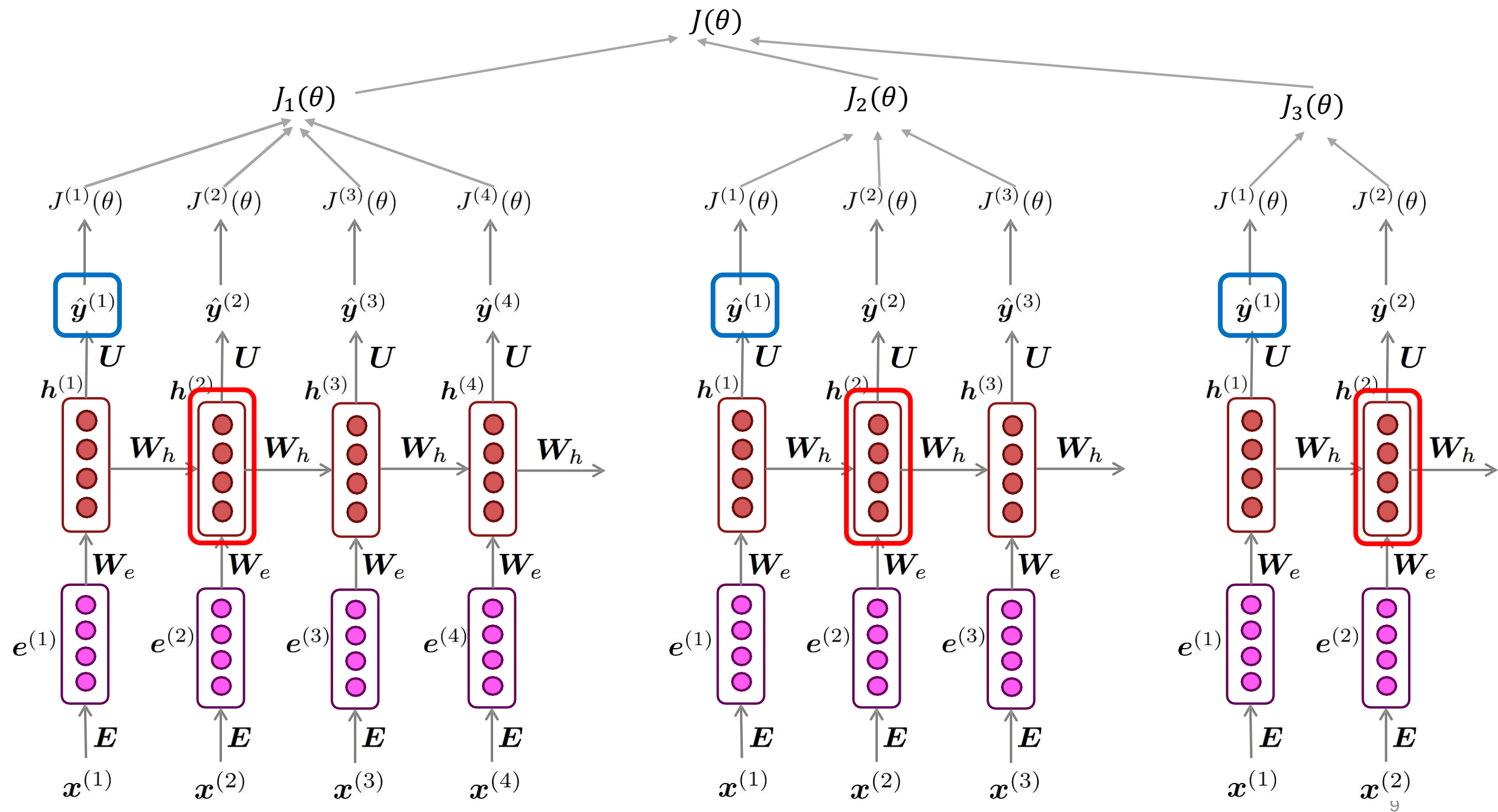


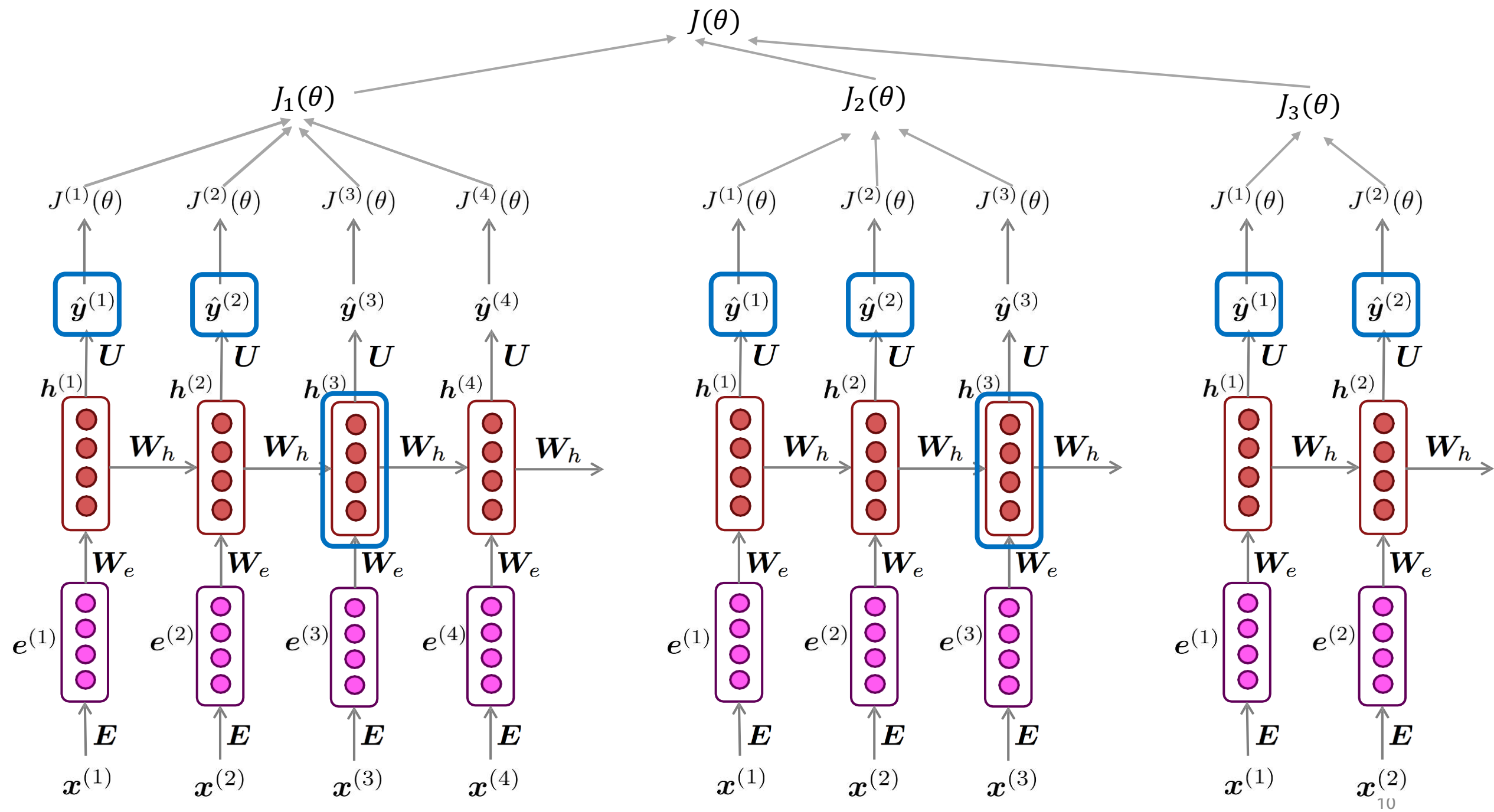


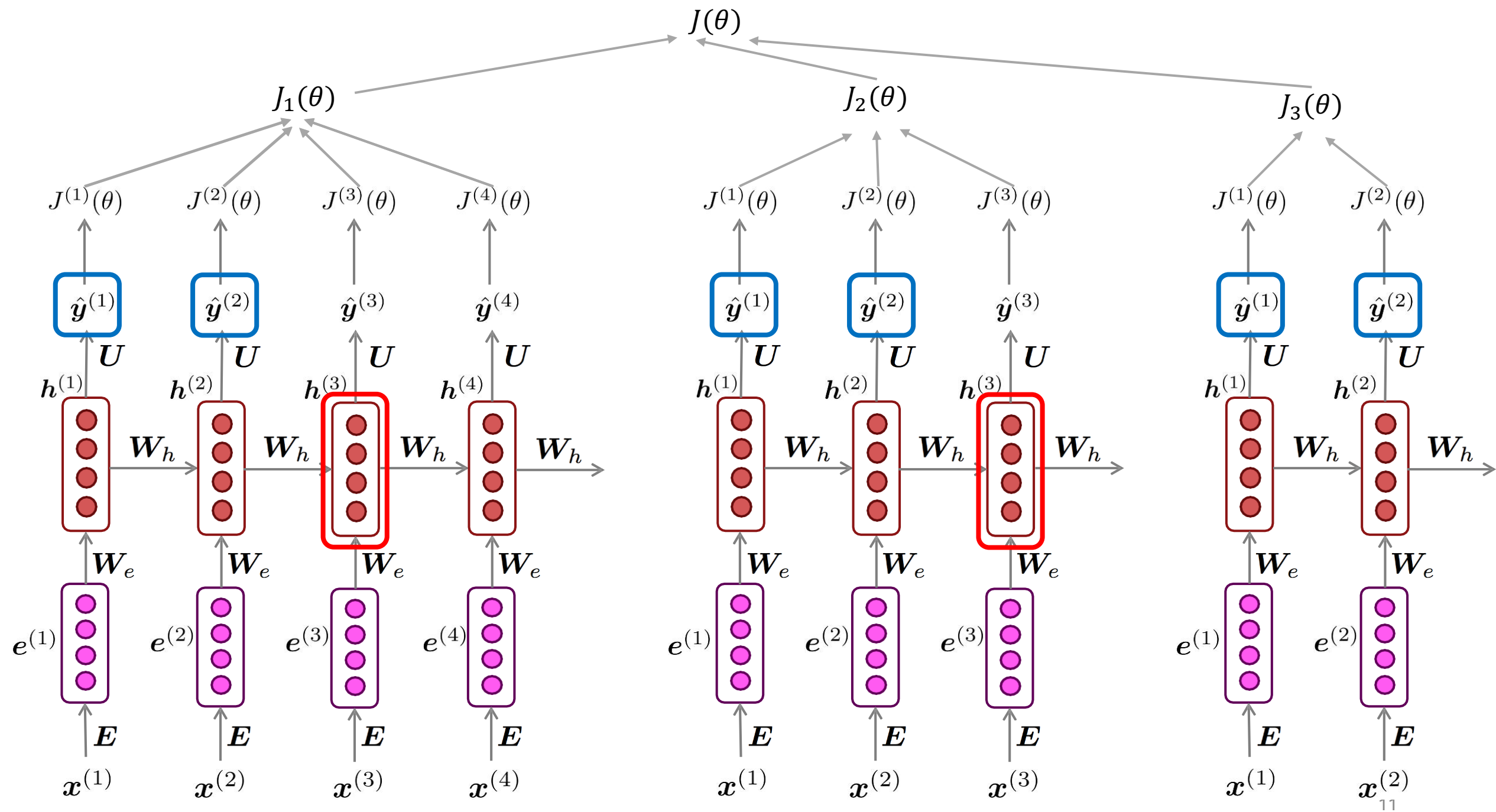


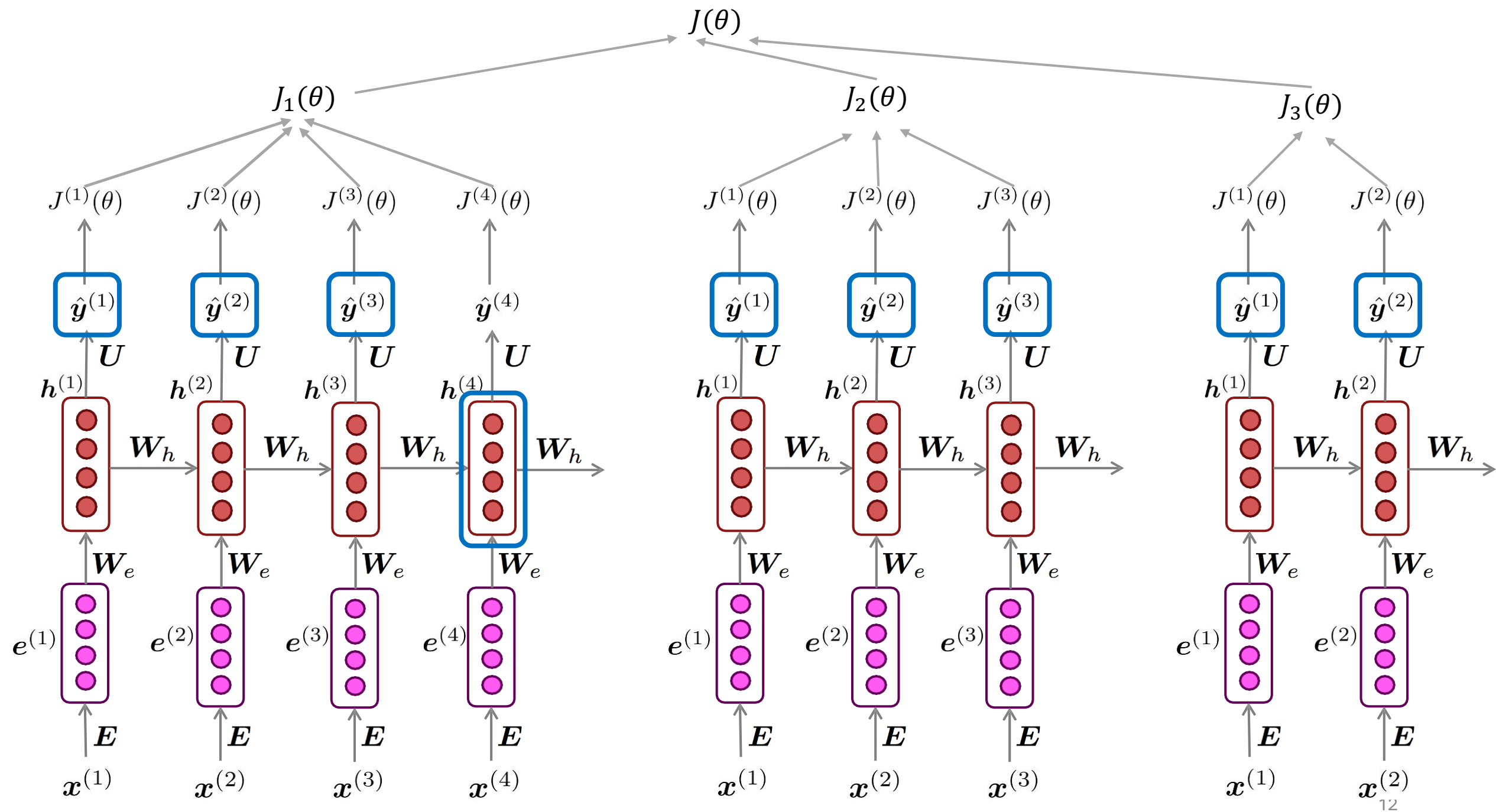


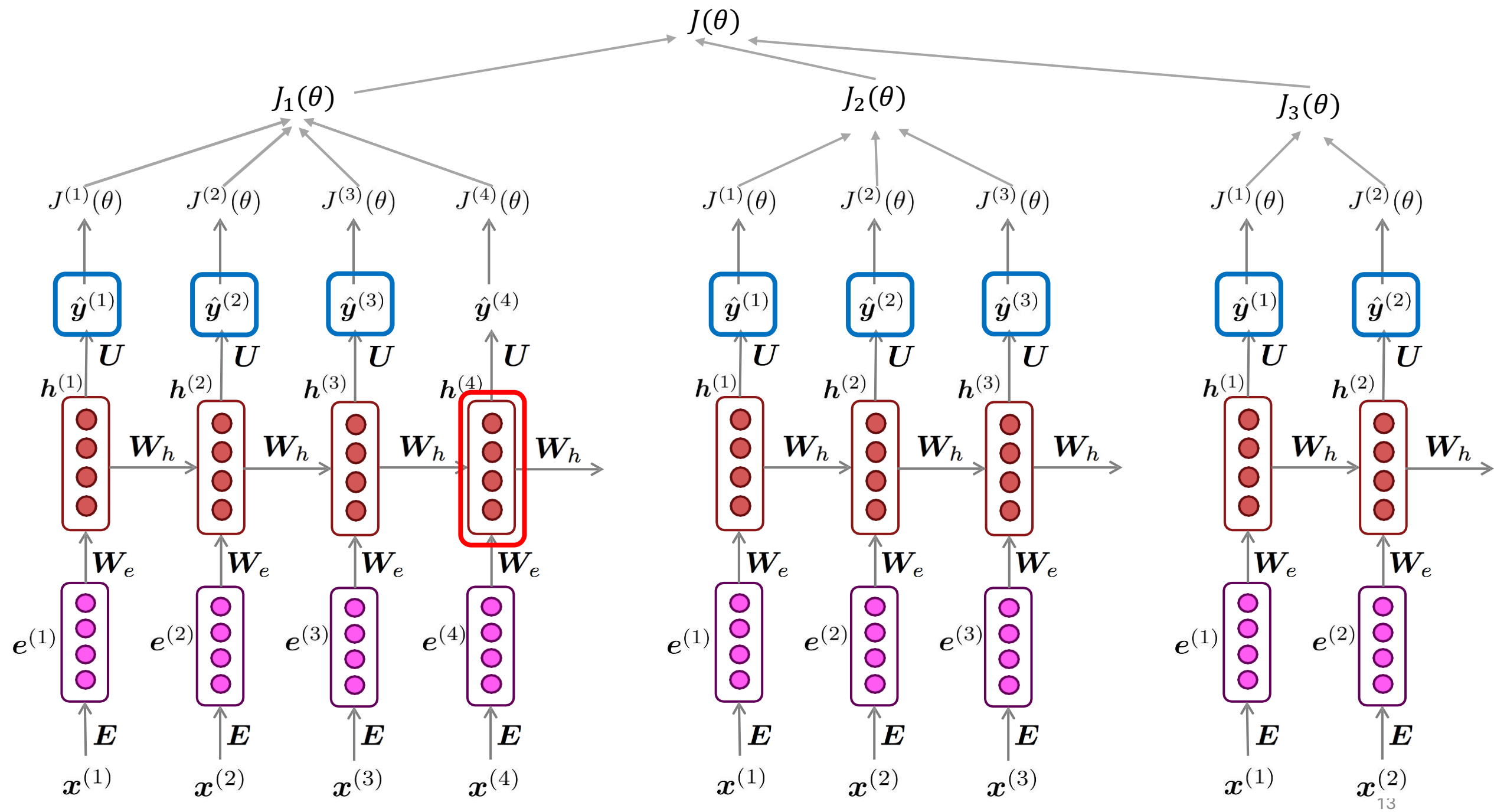


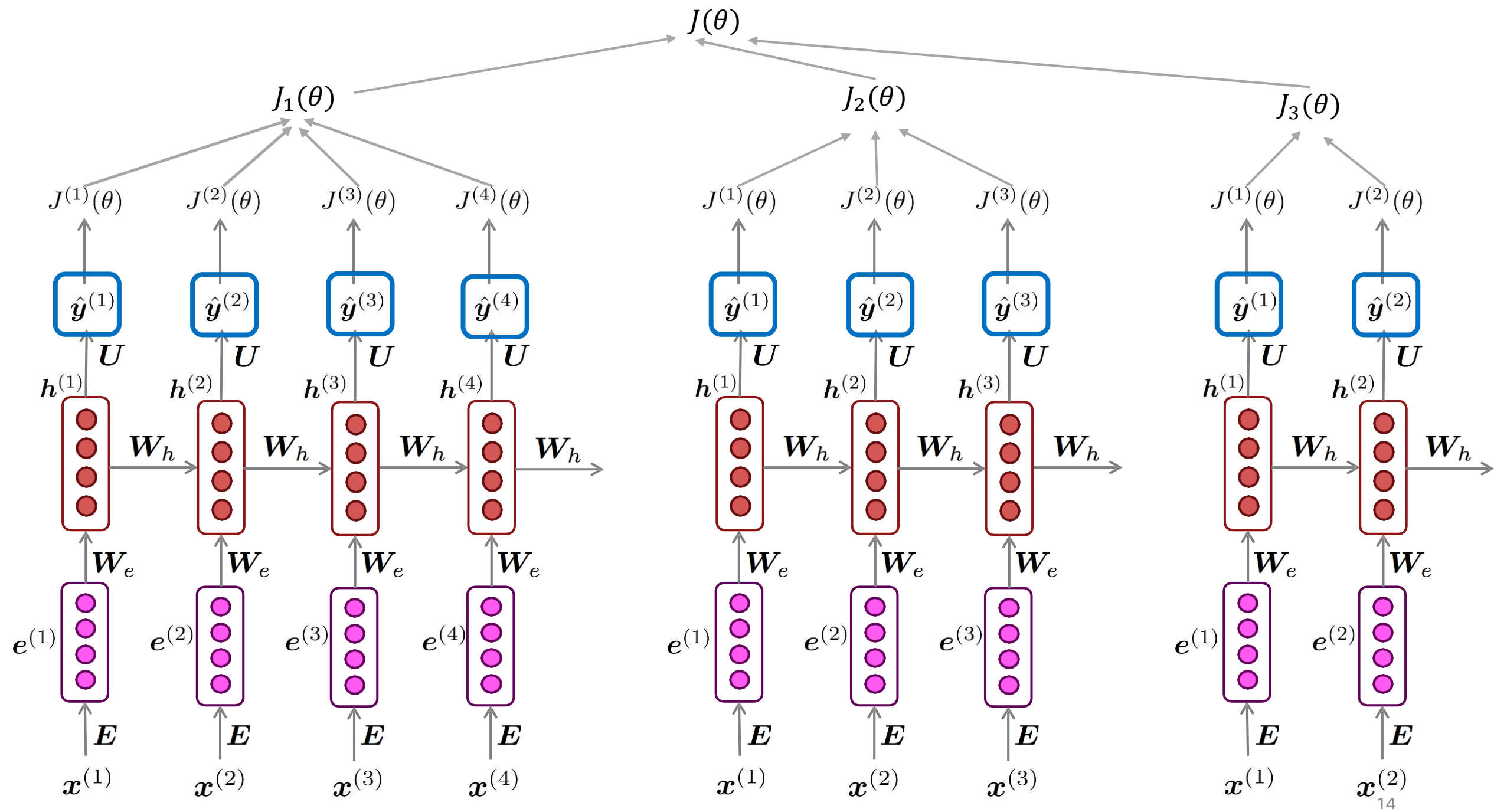


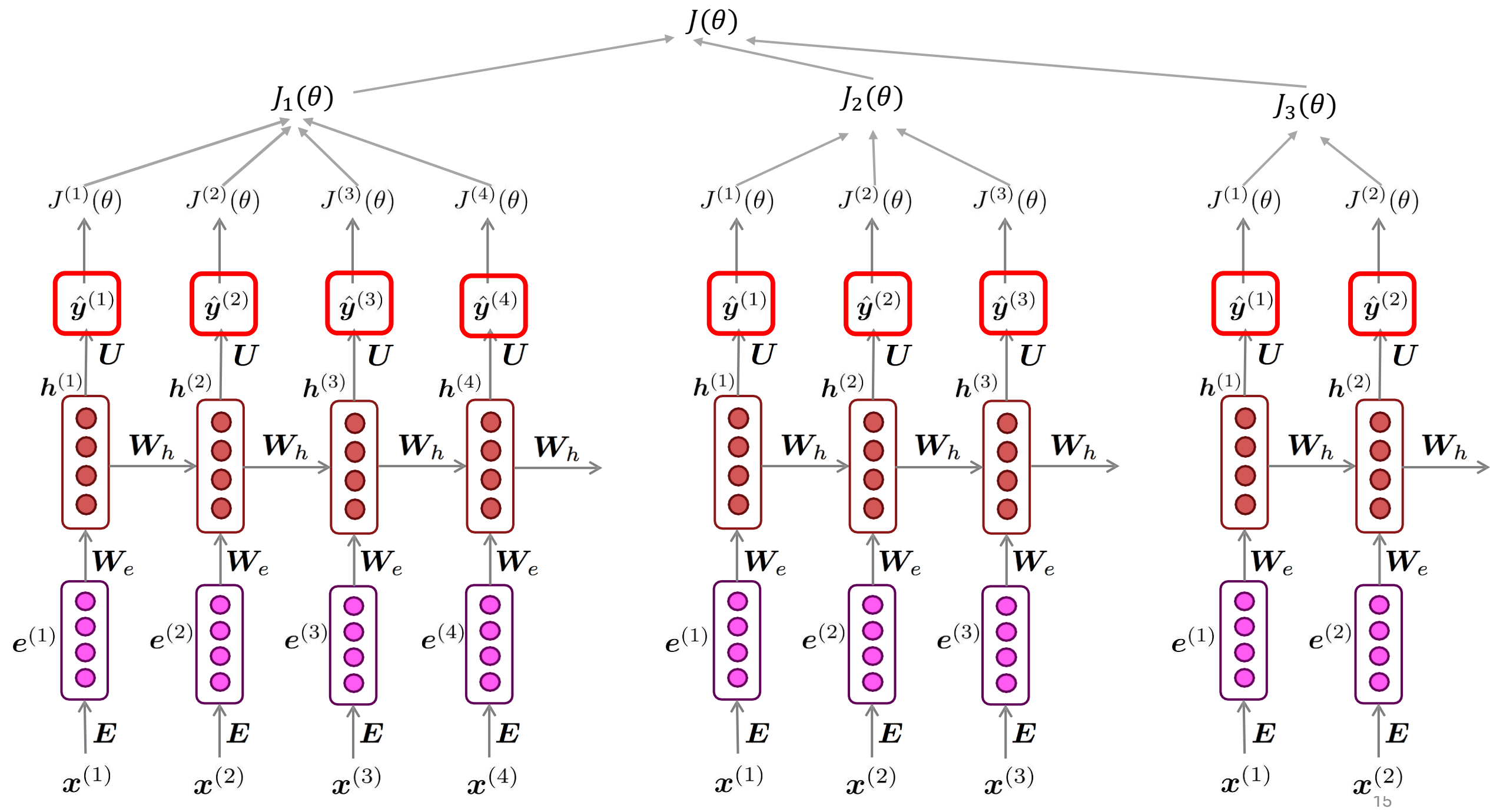


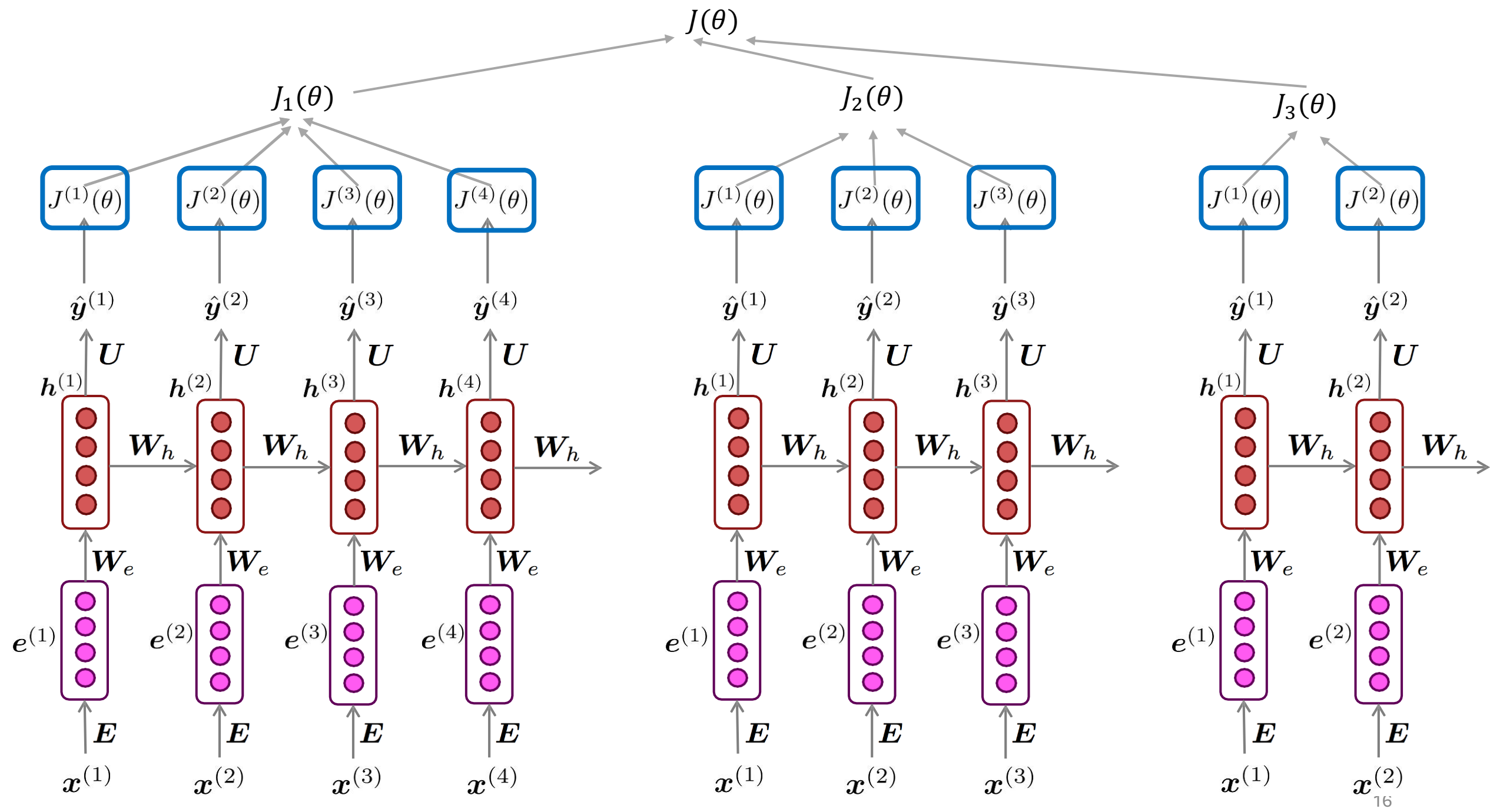


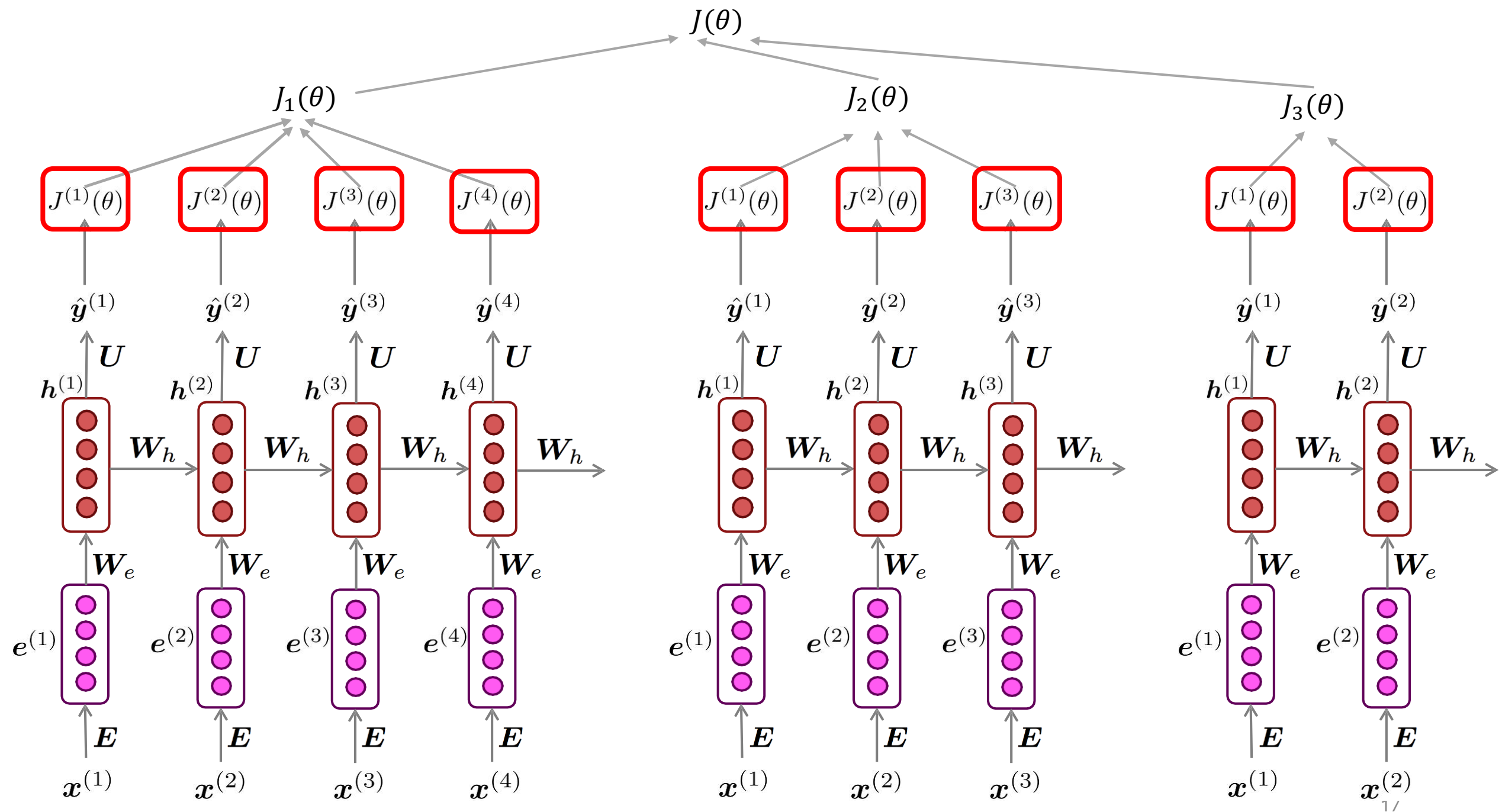


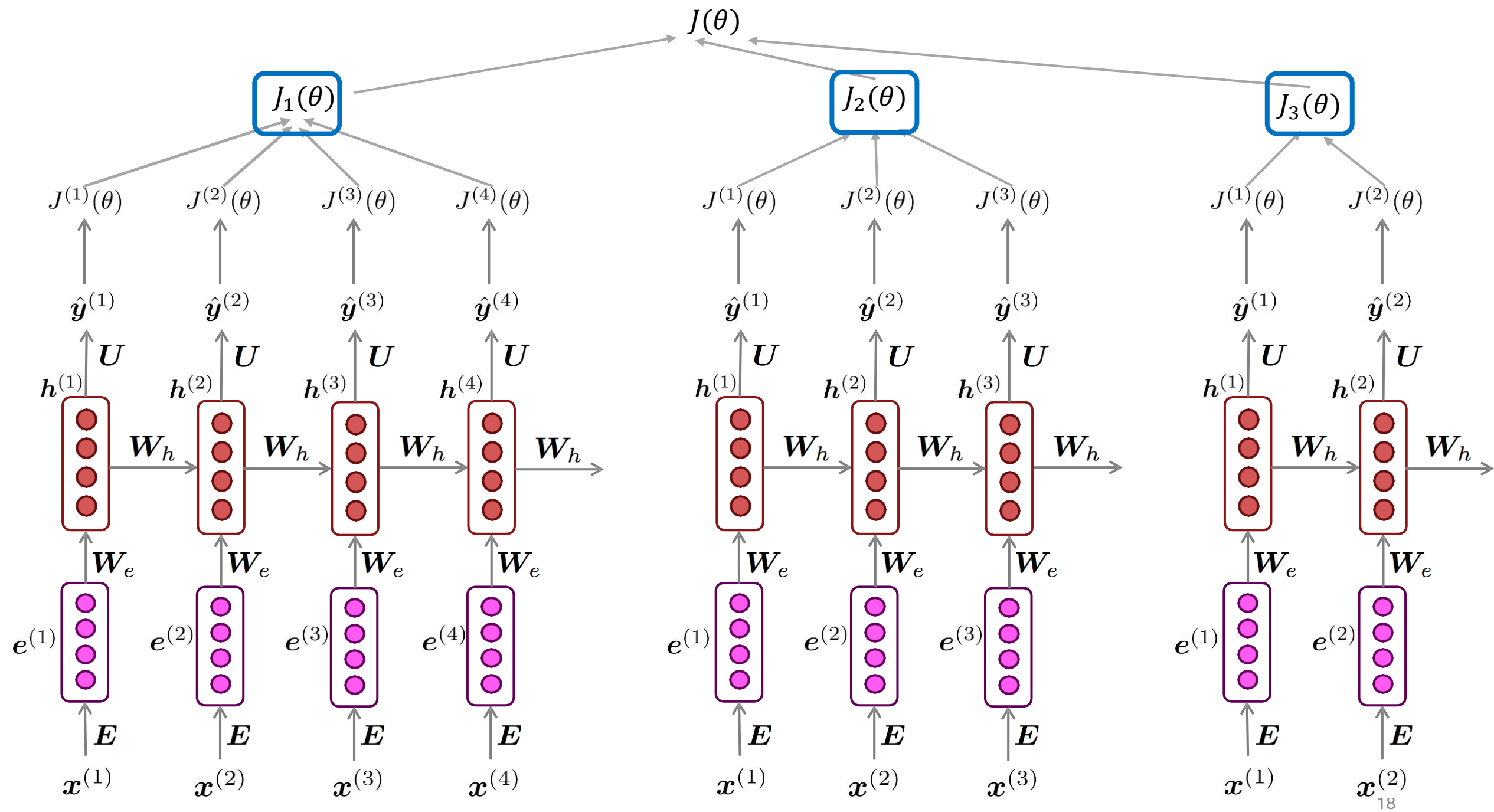


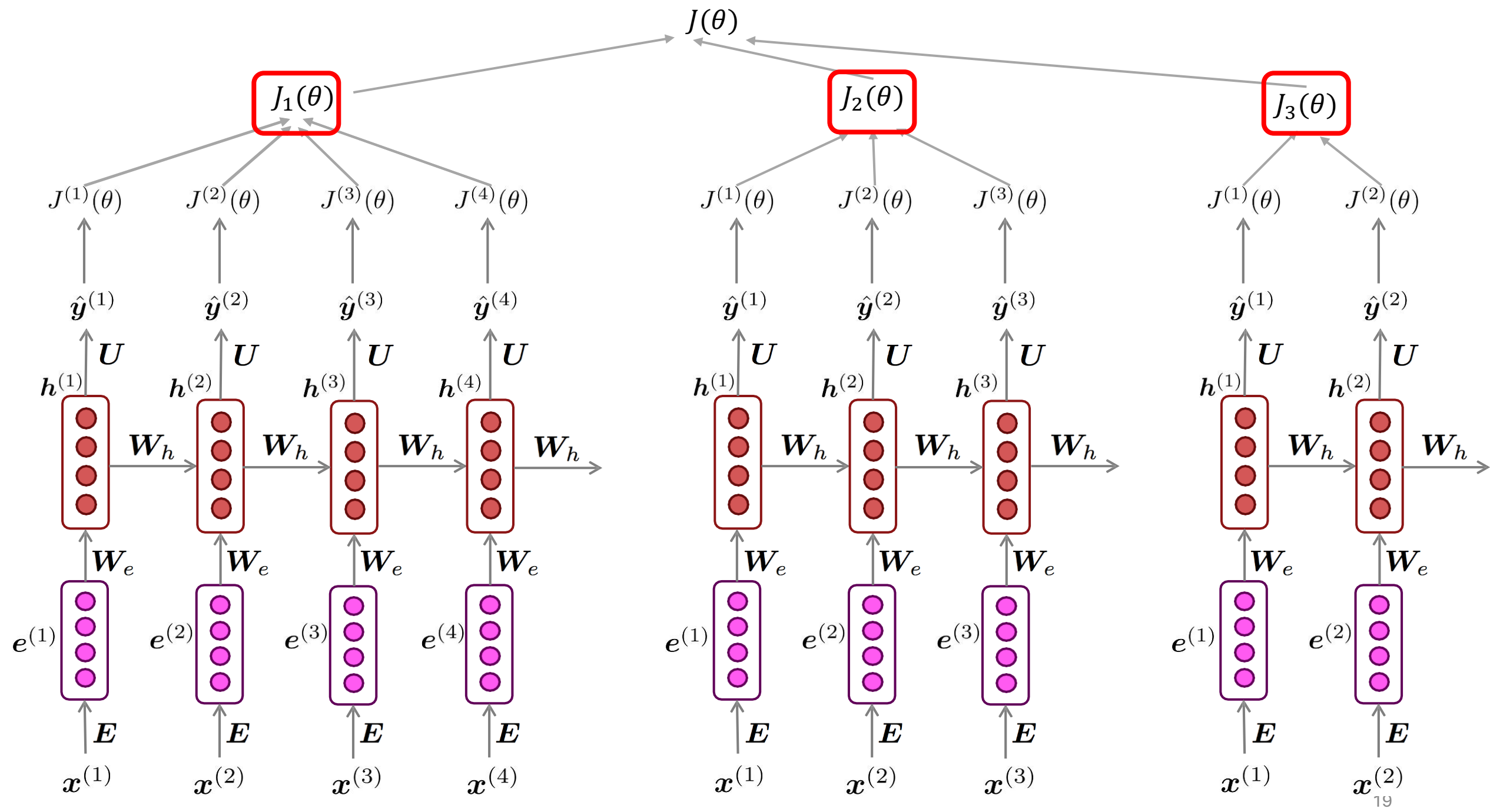


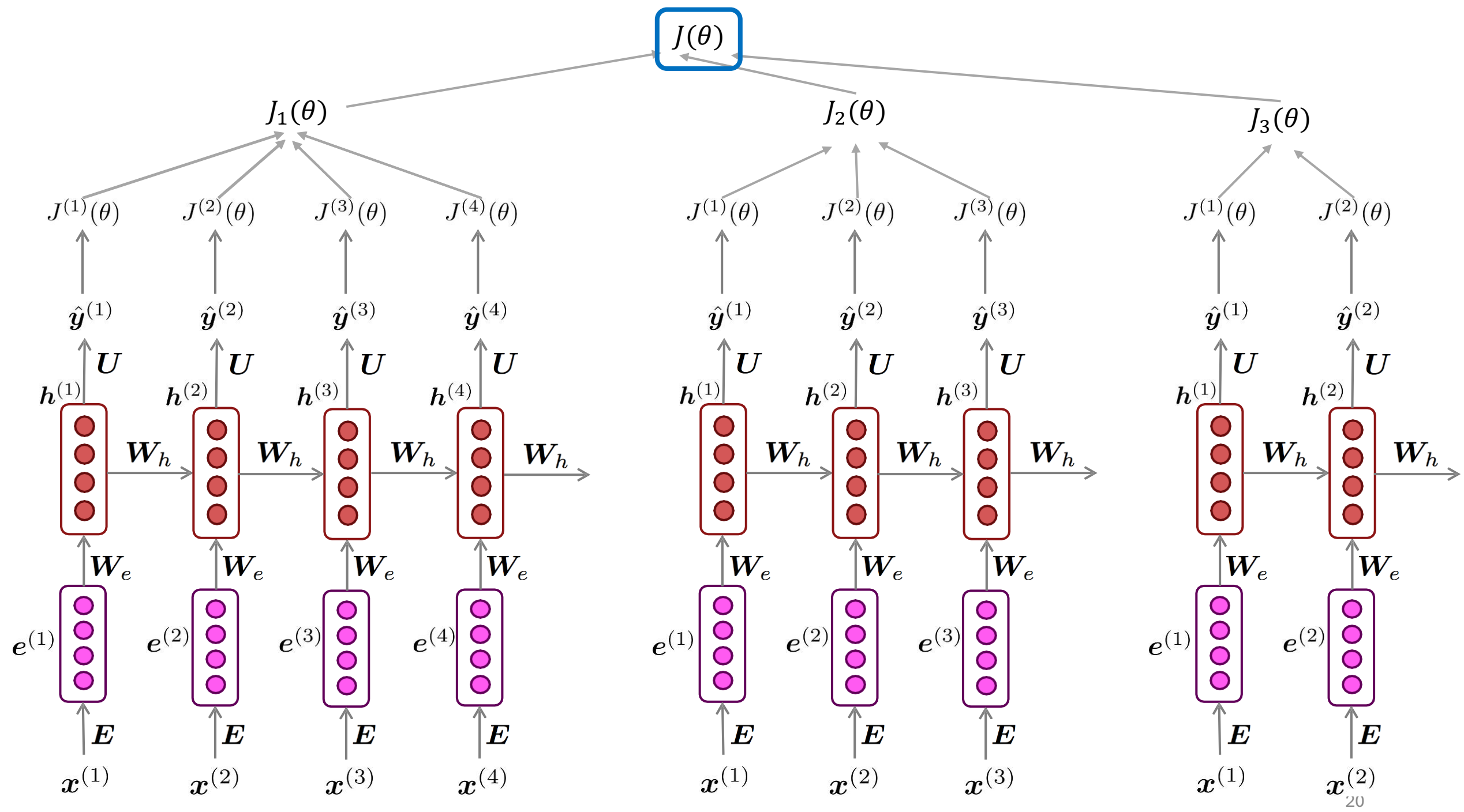


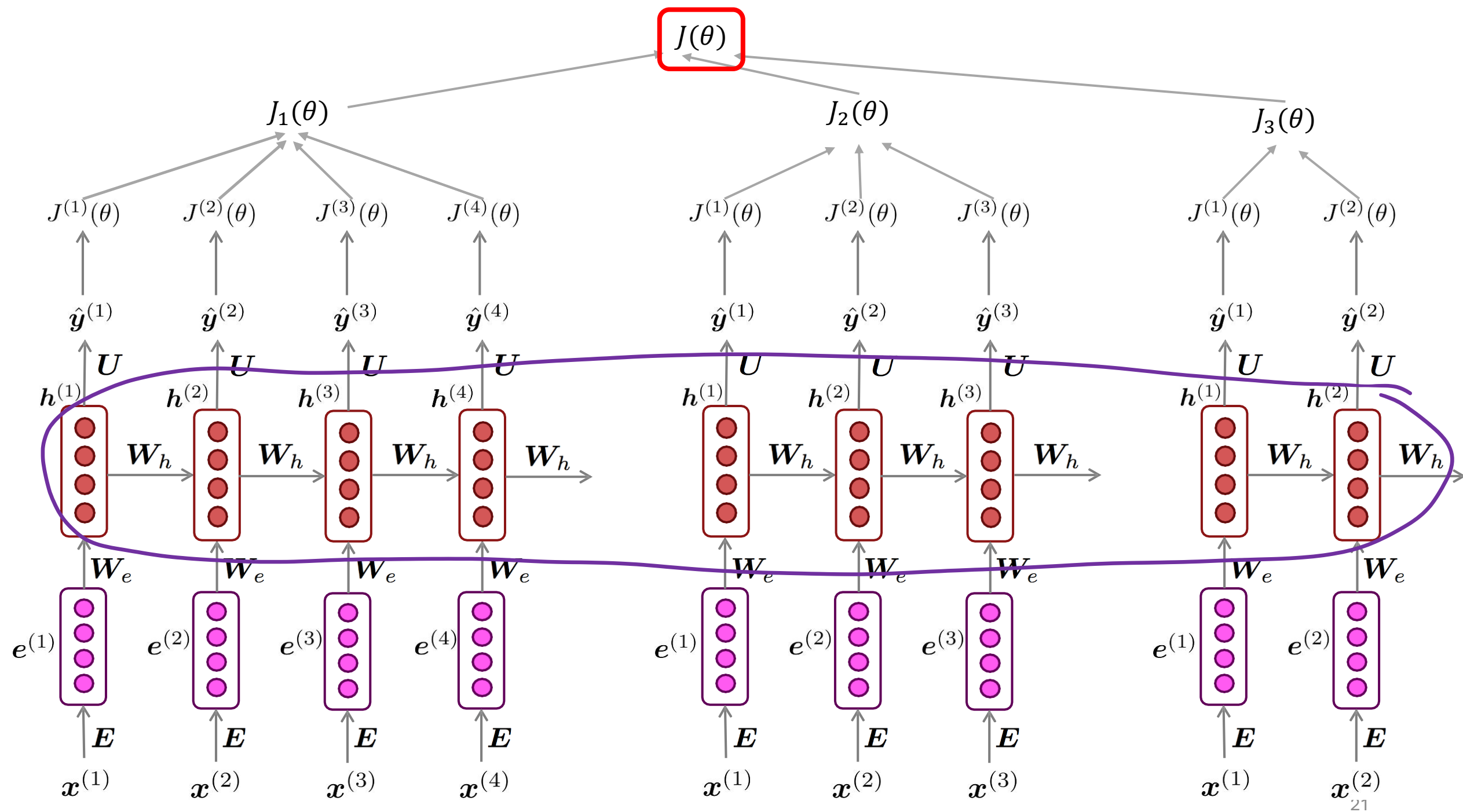












Neural Language Models

- Where are We?
 - Sparsity is mitigated by the use of dense vector representations
 - Prefix length is no longer constant due to the use RNNs/LSTMs
- Challenges Remaining
 - Hard to learn long-range dependencies: Vanishing Gradients
 - Forward and backward passes are $O(\text{sequence length})$
 - Hard to parallelize
- GOAL: An Architecture for Neural LM that is easier to parallelize
 - And, can handle long-range dependencies better.

Transformer

Attention

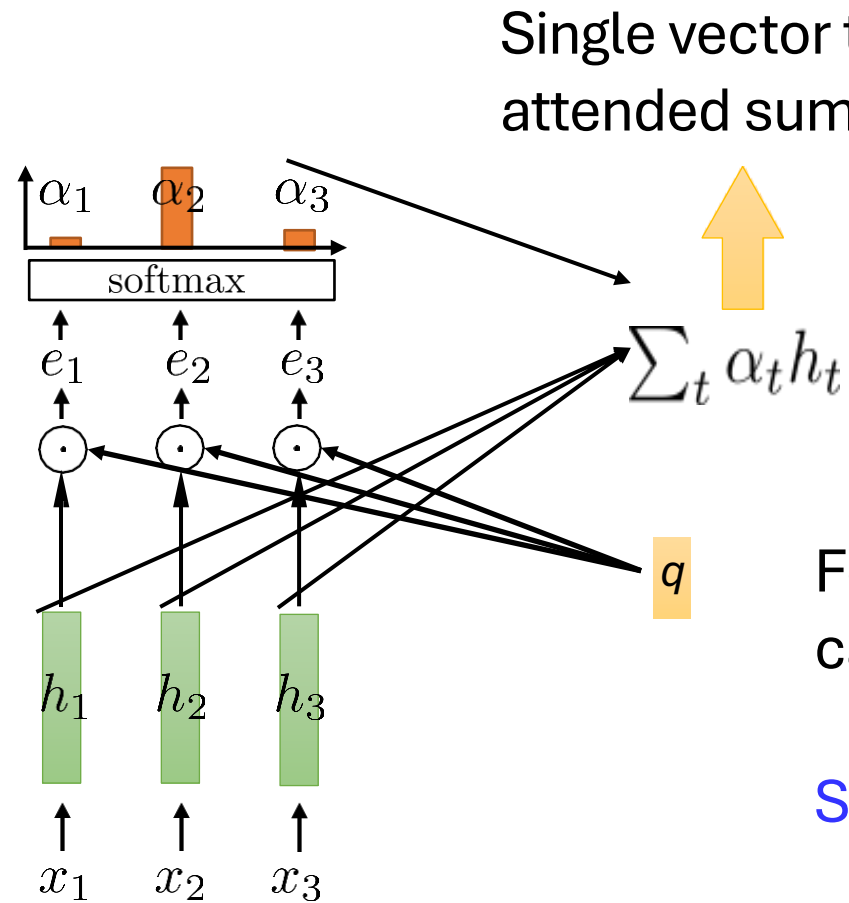
Attention

- How to encode a good bag representation?
- Approach 1: Sum 
 - As number of objects increases, addition will lose information
- Approach 2: Weighted Sum (Attention Module)
 - Focus on some objects at a time
 - But, how do we decide weights (importance of each object)?

$$\sum_i \alpha_i \mathbf{L}_i$$
$$\sum_i \alpha_i = 1$$

- query (q): decides importance of each object 
- attention weights (α_i): normalized importance of each object 
- unnormalized attention weights (e_i): intermediate step to compute α_i 
- attended summary: weighted avg of objects with α weights 

Attention



duong
Attn.

$$e_i = \frac{q^T h_i}{\sqrt{d}}$$

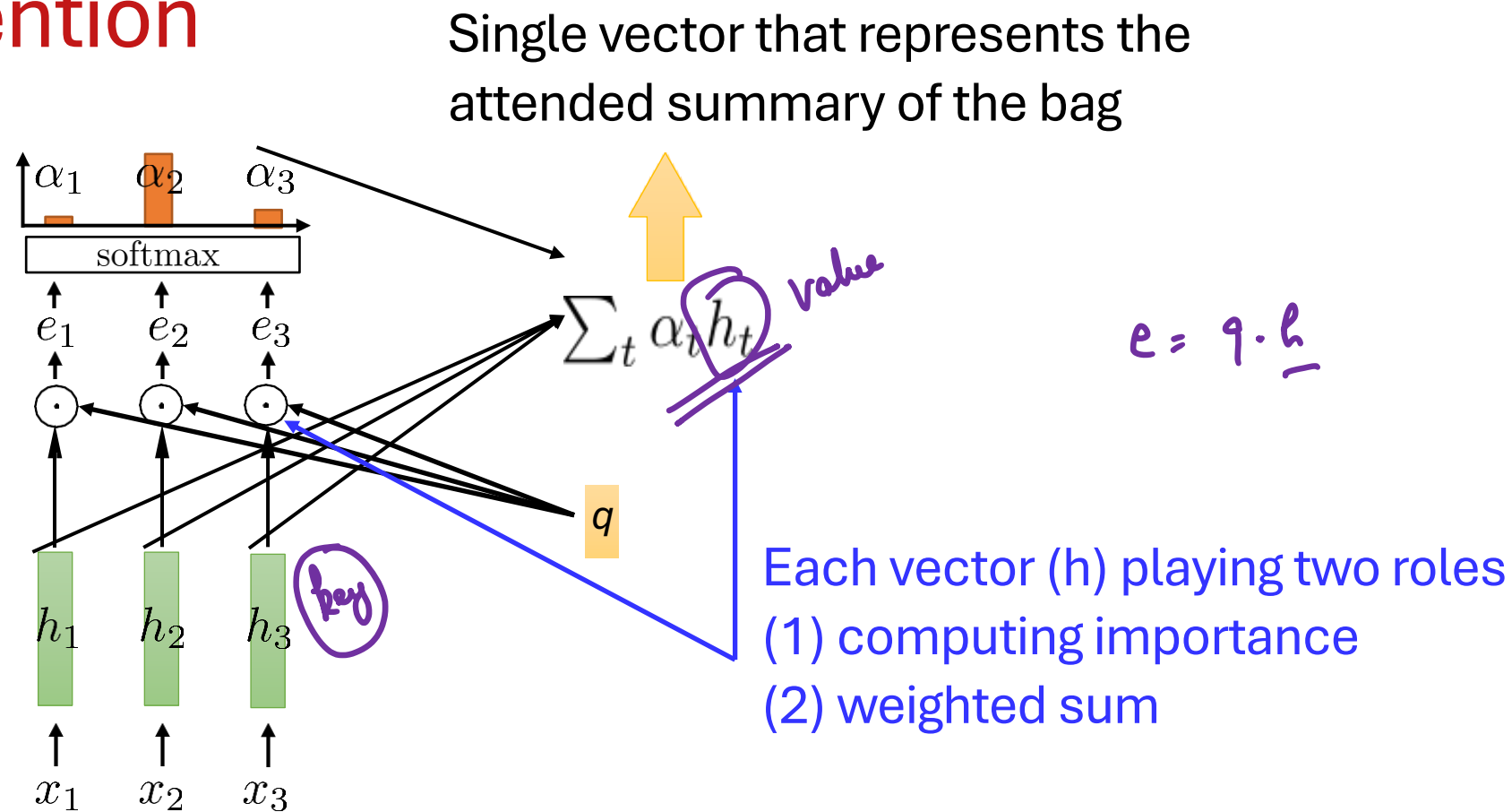
For large values of d , dot product can grow large in magnitude.

Scaled dot product attention:

$$e_i = \frac{q^T h_i}{\sqrt{d}}$$

Where does q come from?

Key-Value Attention



Analogy: DB Lookup

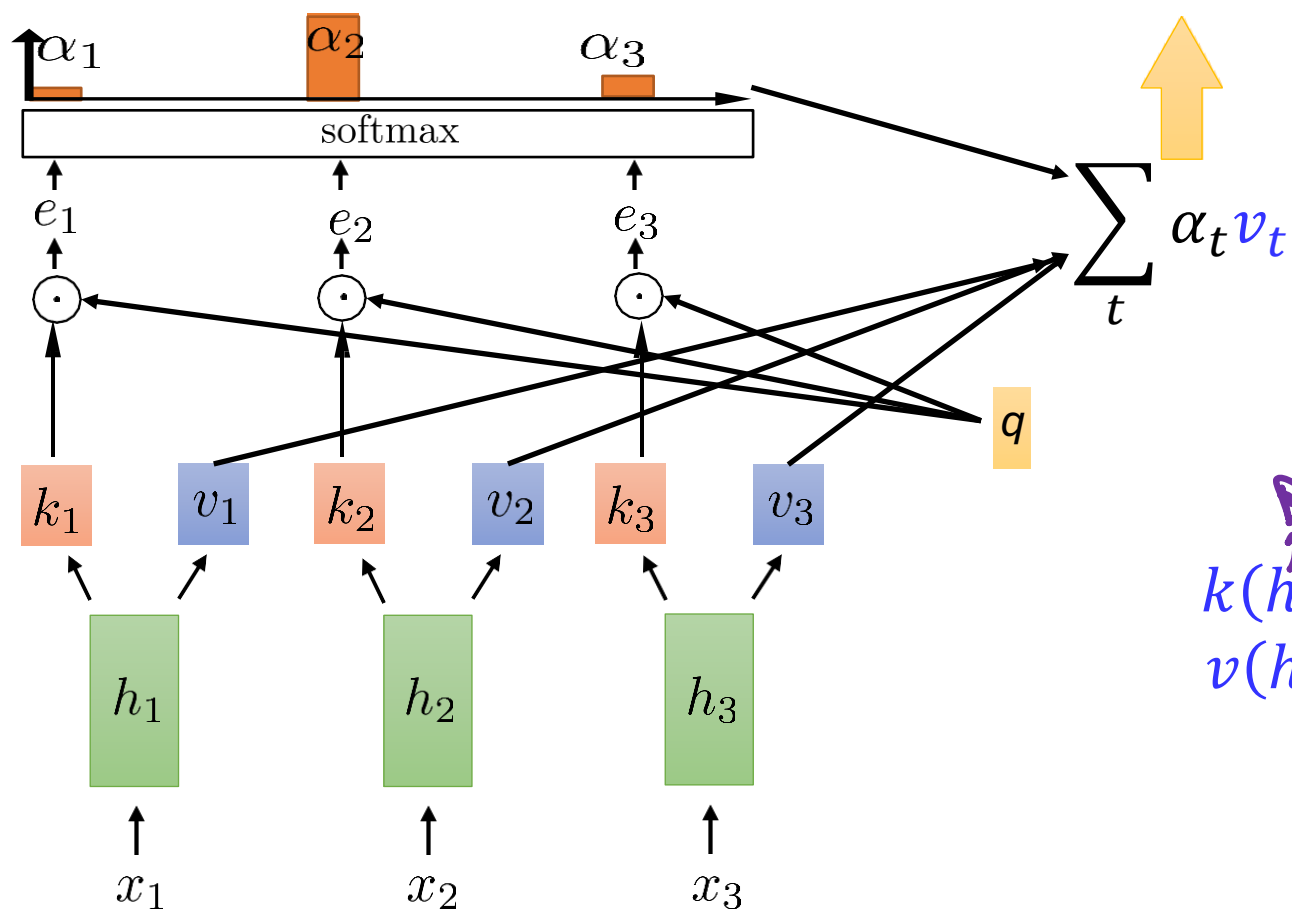
- In a DB
 - We lookup a record using its key
 - We output its value
- In Attention (so far)
 - We assign importance (soft lookup) using h
 - We output the information in vector, also using h

- Solution: Split h into two: key and value

"The PM of India is Modi" h
q. Who is PM of India

Key-Value Attention

Single vector that represents the attended summary of the bag



$$k(h_t) = W_k h_t$$
$$v(h_t) = W_v h_t$$

How do we find q ?

- Solution: Self Attention

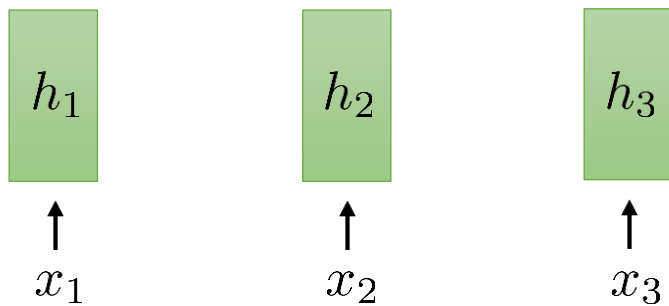
- Each word is also a query!

- Intuition:

- “The cat did not cross the road as it was too tired”
- “The cat did not cross the road as it was too wide”

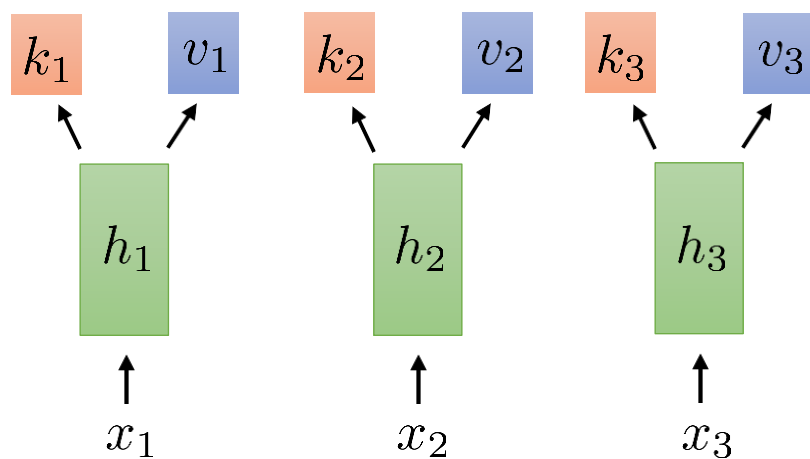
- To understand “it” in Sentence #1, importance of “cat” should be high.
- To understand “it” in Sentence #1, importance of “road” should be high.

Key-Value Self-Attention



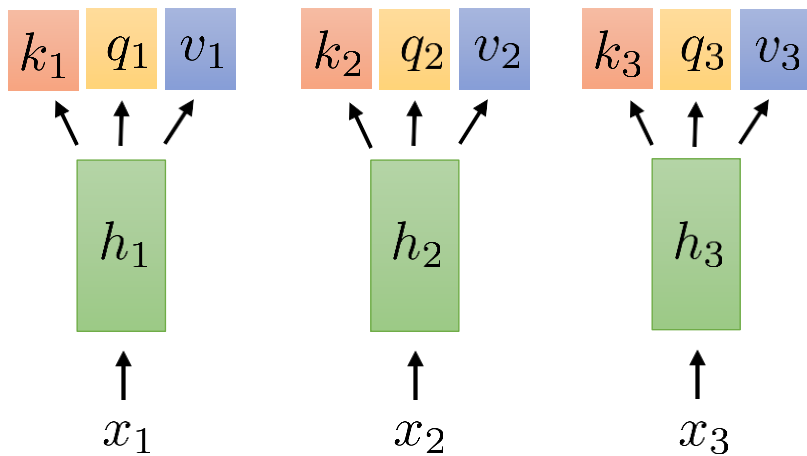
Key-Value Self-Attention

$v_t = v(h_t)$ before just had $v(h_t) = h_t$, now e.g. $v(h_t) = W_v h_t$
 $k_t = k(h_t)$ (just like before) e.g., $k_t = W_k h_t$



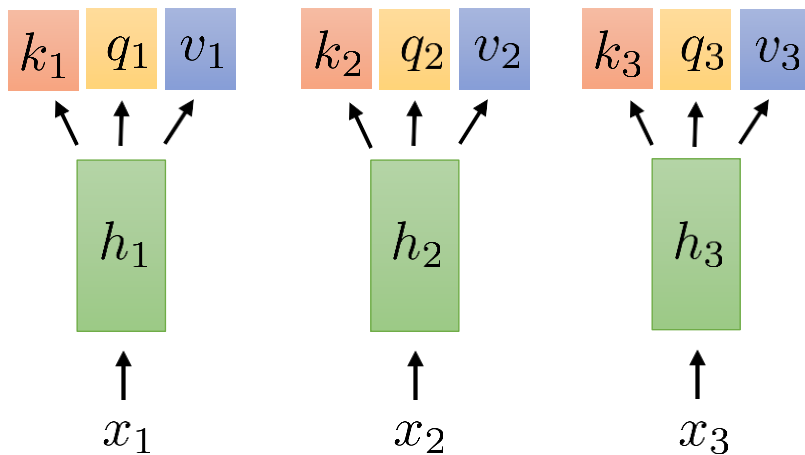
Key-Value Self-Attention

$v_t = v(h_t)$ before just had $v(h_t) = h_t$, now e.g. $v(h_t) = W_v h_t$
 $k_t = k(h_t)$ (just like before) e.g., $k_t = W_k h_t$

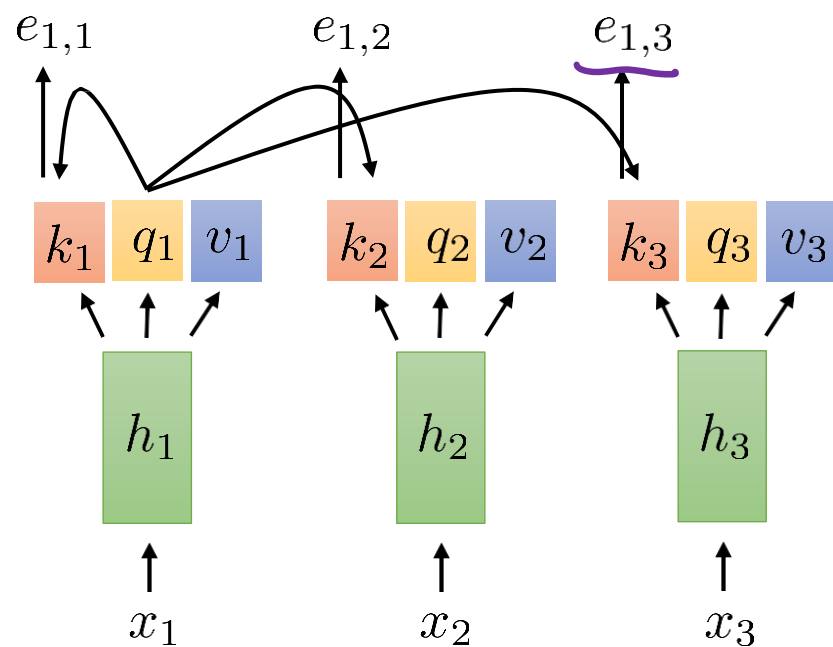


Key-Value Self-Attention

$v_t = v(h_t)$ before just had $v(h_t) = h_t$, now e.g. $v(h_t) = W_v h_t$
 $k_t = k(h_t)$ (just like before) e.g., $k_t = W_k h_t$
 $q_t = q(h_t)$ e.g., $q_t = \underline{W_q h_t}$

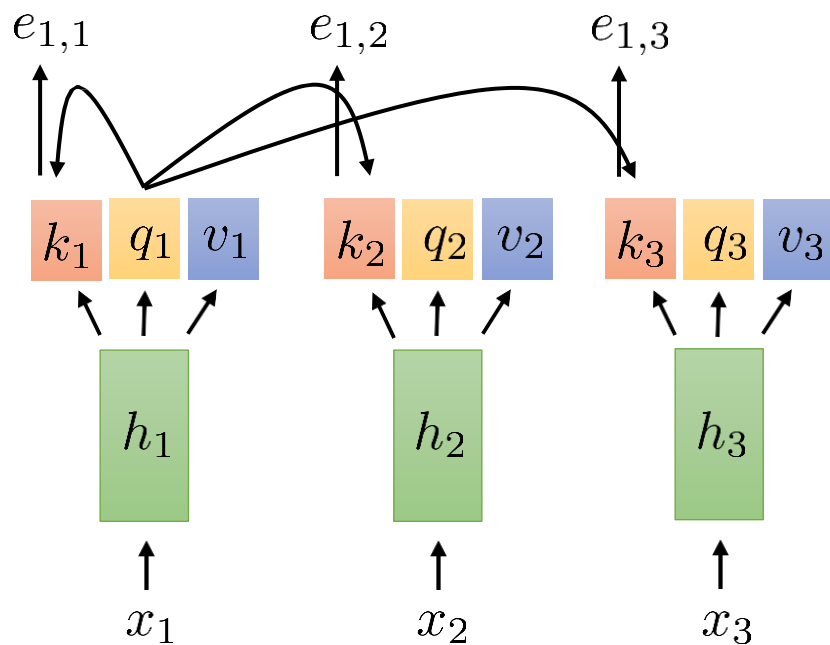


Key-Value Self-Attention



$v_t = v(h_t)$ before just had $v(h_t) = h_t$, now e.g. $v(h_t) = W_v h_t$
 $k_t = k(h_t)$ (just like before) e.g., $k_t = W_k h_t$
 $q_t = q(h_t)$ e.g., $q_t = W_q h_t$

Key-Value Self-Attention



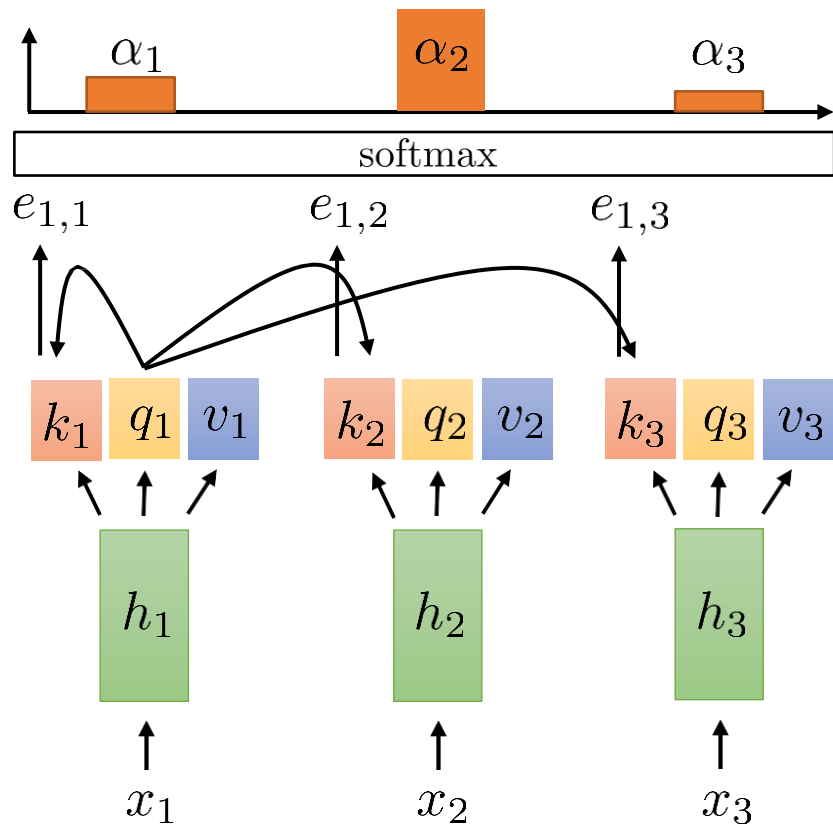
$$e_{l,t} = \frac{q_l \cdot k_t}{\sqrt{d}}$$

$v_t = v(h_t)$ before just had $v(h_t) = h_t$, now e.g. $v(h_t) = W_v h_t$

$k_t = k(h_t)$ (just like before) e.g., $k_t = W_k h_t$

$q_t = q(h_t)$ e.g., $q_t = W_q h_t$

Key-Value Self-Attention



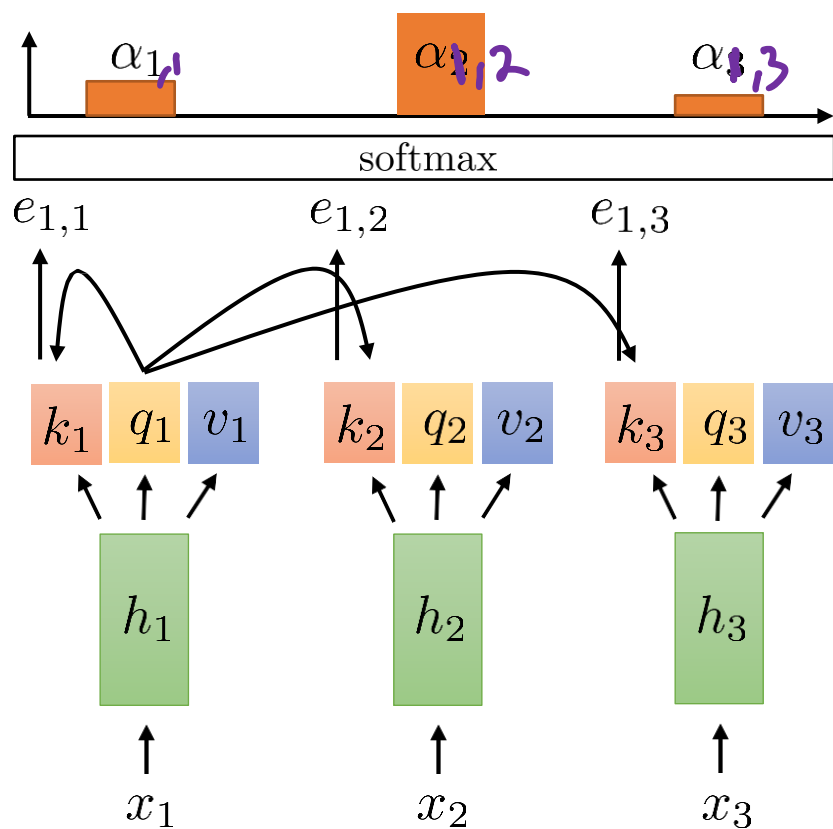
$$e_{l,t} = q_l \cdot k_t$$

$v_t = v(h_t)$ before just had $v(h_t) = h_t$, now e.g. $v(h_t) = W_v h_t$

$k_t = k(h_t)$ (just like before) e.g., $k_t = W_k h_t$

$q_t = q(h_t)$ e.g., $q_t = W_q h_t$

Key-Value Self-Attention



$$\alpha_{l,t} = \exp(e_{l,t}) / \sum_{t'} \exp(e_{l,t'})$$

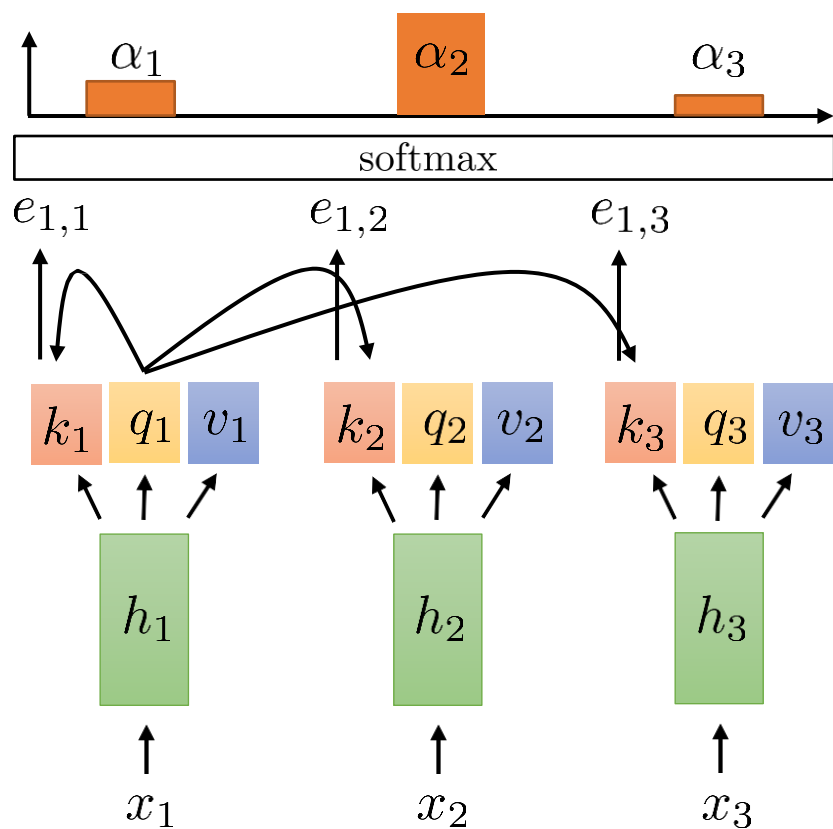
$$e_{l,t} = q_l \cdot k_t$$

$v_t = v(h_t)$ before just had $v(h_t) = h_t$, now e.g. $v(h_t) = W_v h_t$

$k_t = k(h_t)$ (just like before) e.g., $k_t = W_k h_t$

$q_t = q(h_t)$ e.g., $q_t = W_q h_t$

Key-Value Self-Attention



$$\alpha_{l,t} = \exp(e_{l,t}) / \sum_{t'} \exp(e_{l,t'})$$

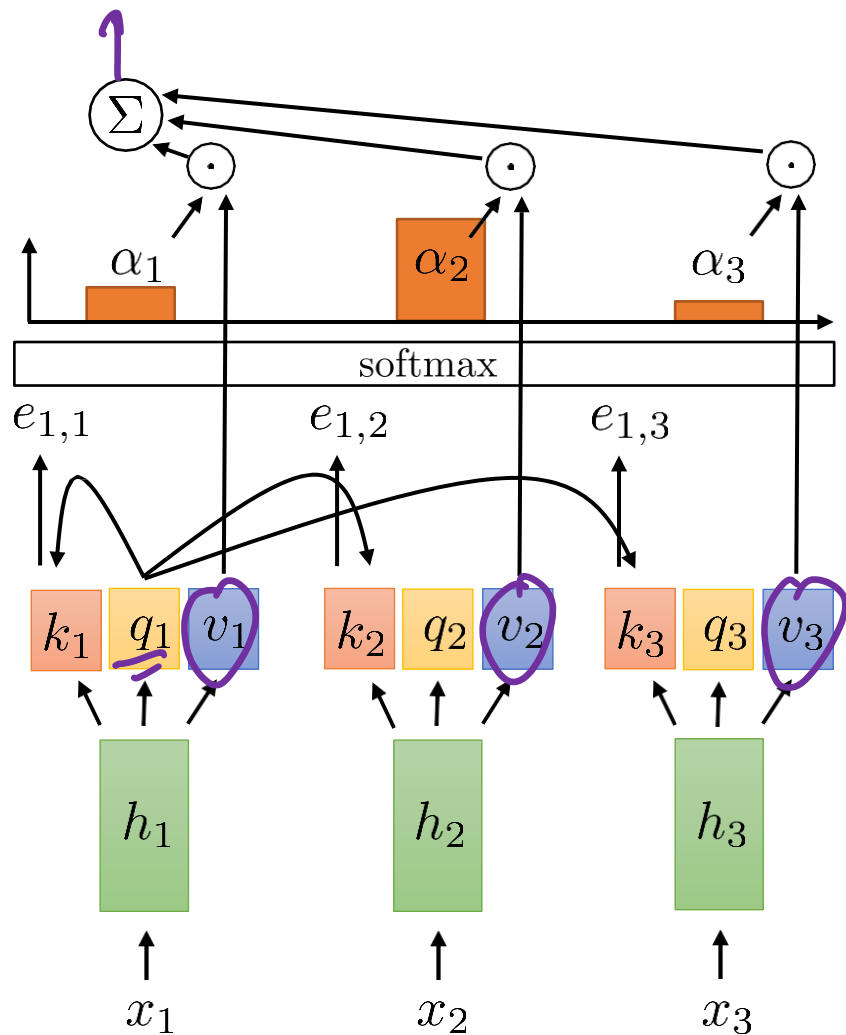
$$e_{l,t} = q_l \cdot k_t$$

$v_t = v(h_t)$ before just had $v(h_t) = h_t$, now e.g. $v(h_t) = W_v h_t$

$k_t = k(h_t)$ (just like before) e.g., $k_t = W_k h_t$

$q_t = q(h_t)$ e.g., $q_t = W_q h_t$

Key-Value Self-Attention



$$\alpha_{l,t} = \exp(e_{l,t}) / \sum_{t'} \exp(e_{l,t'})$$

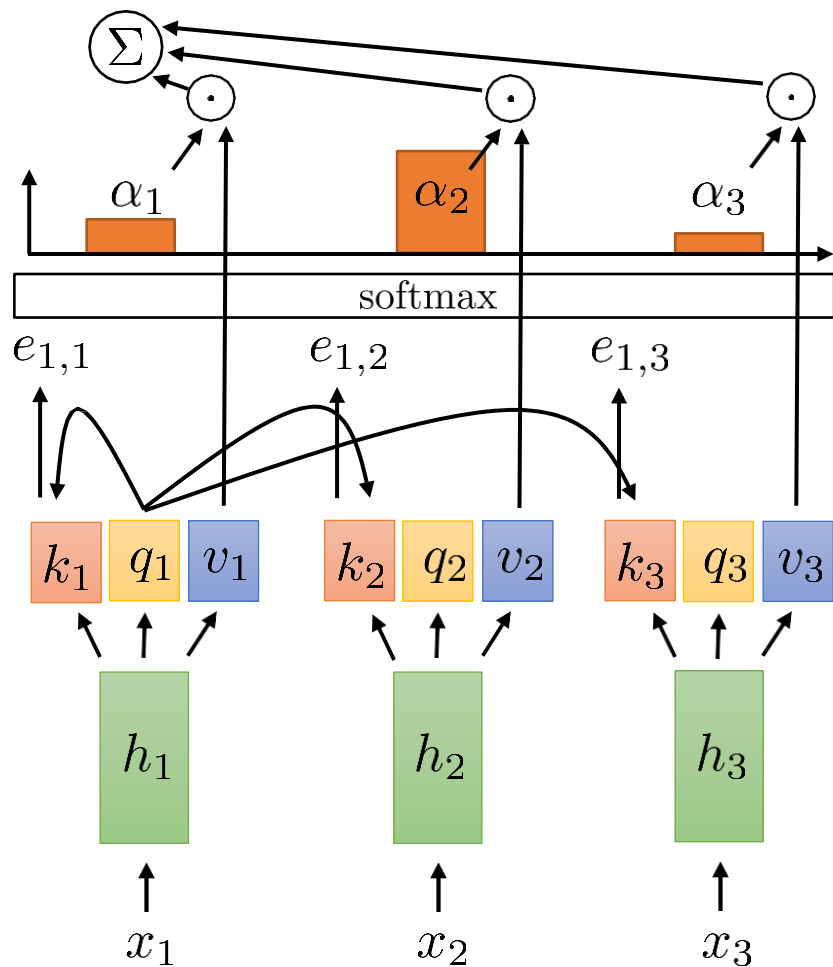
$$e_{l,t} = q_l \cdot k_t$$

$v_t = v(h_t)$ before just had $v(h_t) = h_t$, now e.g. $v(h_t) = W_v h_t$

$k_t = k(h_t)$ (just like before) e.g., $k_t = W_k h_t$

$q_t = q(h_t)$ e.g., $q_t = W_q h_t$

Key-Value Self-Attention



$$a_l = \sum_t \alpha_{l,t} v_t$$

$$\alpha_{l,t} = \exp(e_{l,t}) / \sum_{t'} \exp(e_{l,t'})$$

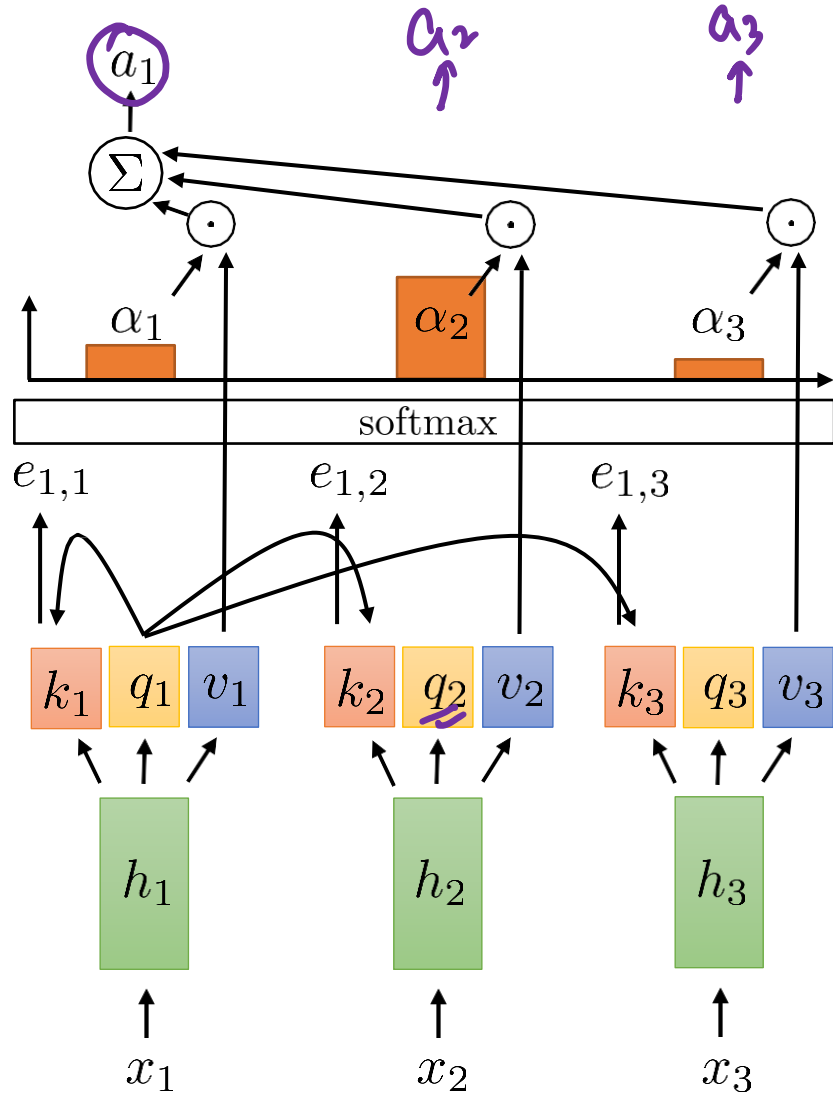
$$e_{l,t} = q_l \cdot k_t$$

$v_t = v(h_t)$ before just had $v(h_t) = h_t$, now e.g. $v(h_t) = W_v h_t$

$k_t = k(h_t)$ (just like before) e.g., $k_t = W_k h_t$

$q_t = q(h_t)$ e.g., $q_t = W_q h_t$

Key-Value Self-Attention



$$a_l = \sum_t \alpha_{l,t} v_t$$

$$\alpha_{l,t} = \exp(e_{l,t}) / \sum_{t'} \exp(e_{l,t'})$$

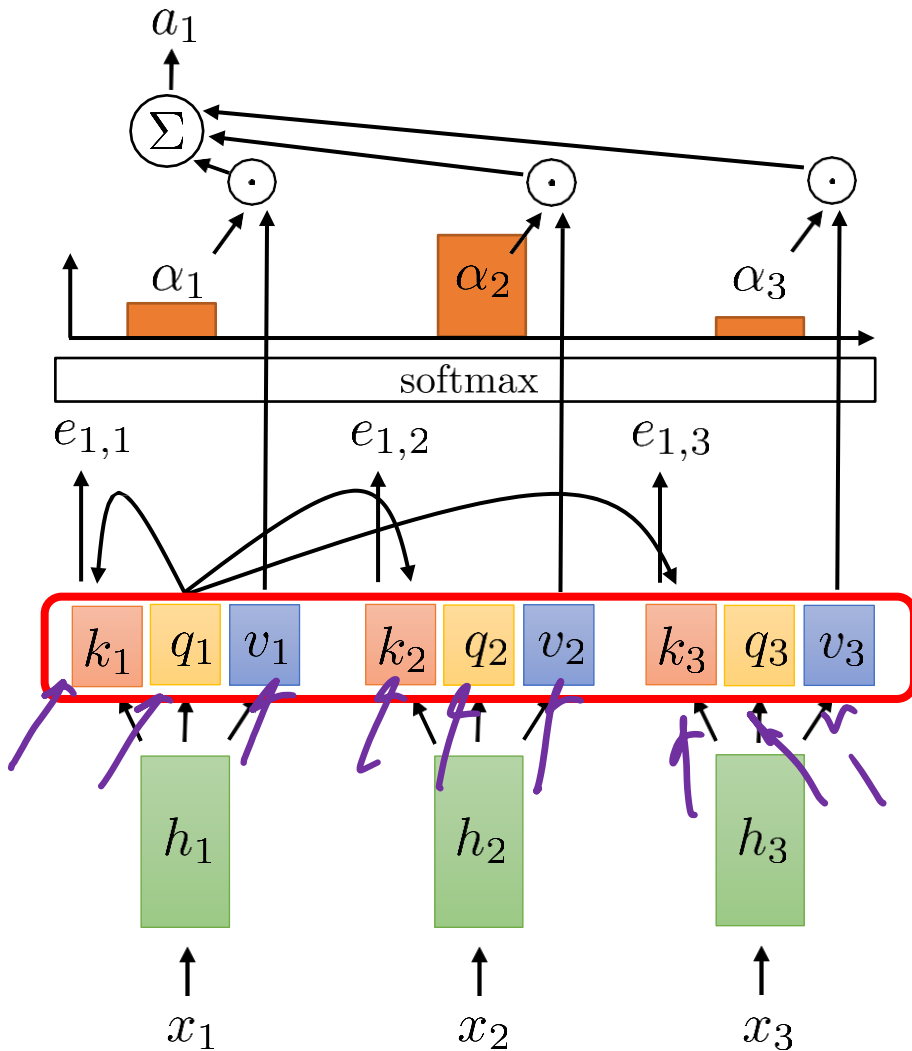
$$e_{l,t} = q_l \cdot k_t$$

$v_t = v(h_t)$ before just had $v(h_t) = h_t$, now e.g. $v(h_t) = W_v h_t$

$k_t = k(h_t)$ (just like before) e.g., $k_t = W_k h_t$

$q_t = q(h_t)$ e.g., $q_t = W_q h_t$

Key-Value Self-Attention



$$a_l = \sum_t \alpha_{l,t} v_t$$

$$\alpha_{l,t} = \exp(e_{l,t}) / \sum_{t'} \exp(e_{l,t'})$$

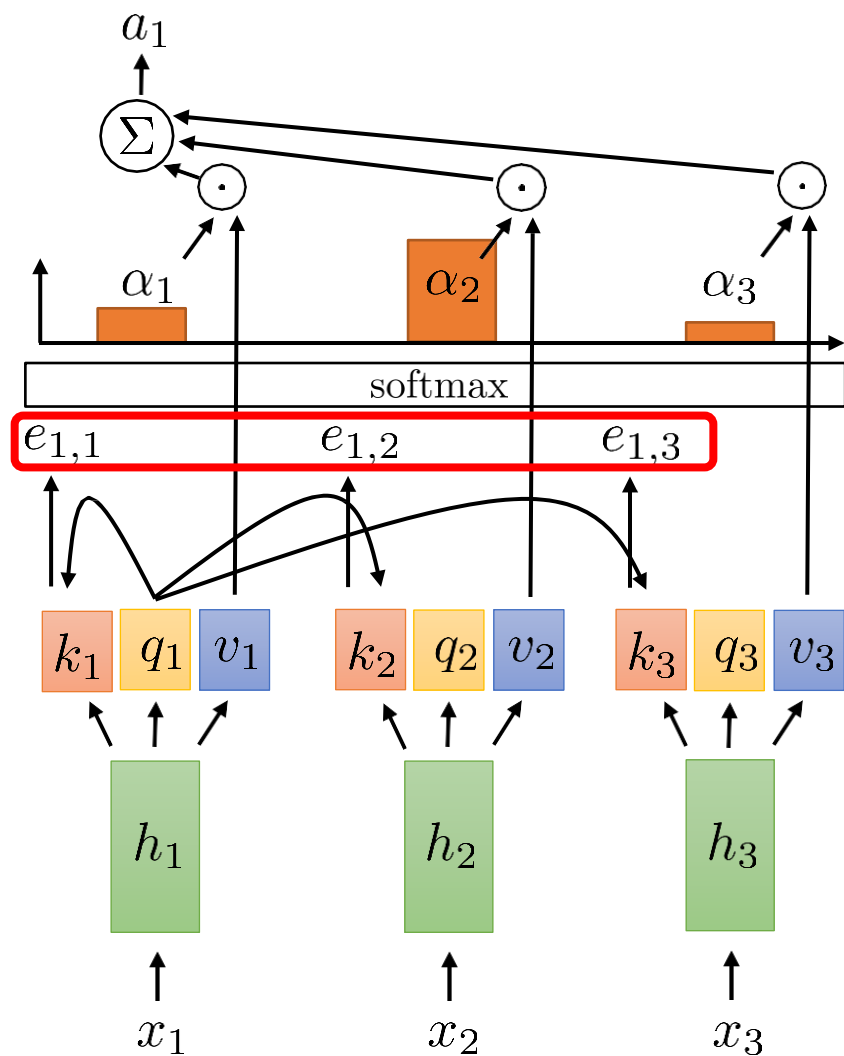
$$e_{l,t} = q_l \cdot k_t$$

$v_t = v(h_t)$ before just had $v(h_t) = h_t$, now e.g. $v(h_t) = W_v h_t$

$k_t = k(h_t)$ (just like before) e.g., $k_t = W_k h_t$

$q_t = q(h_t)$ e.g., $q_t = W_q h_t$

Key-Value Self-Attention



$$a_l = \sum_t \alpha_{l,t} v_t$$

$$\alpha_{l,t} = \exp(e_{l,t}) / \sum_{t'} \exp(e_{l,t'})$$

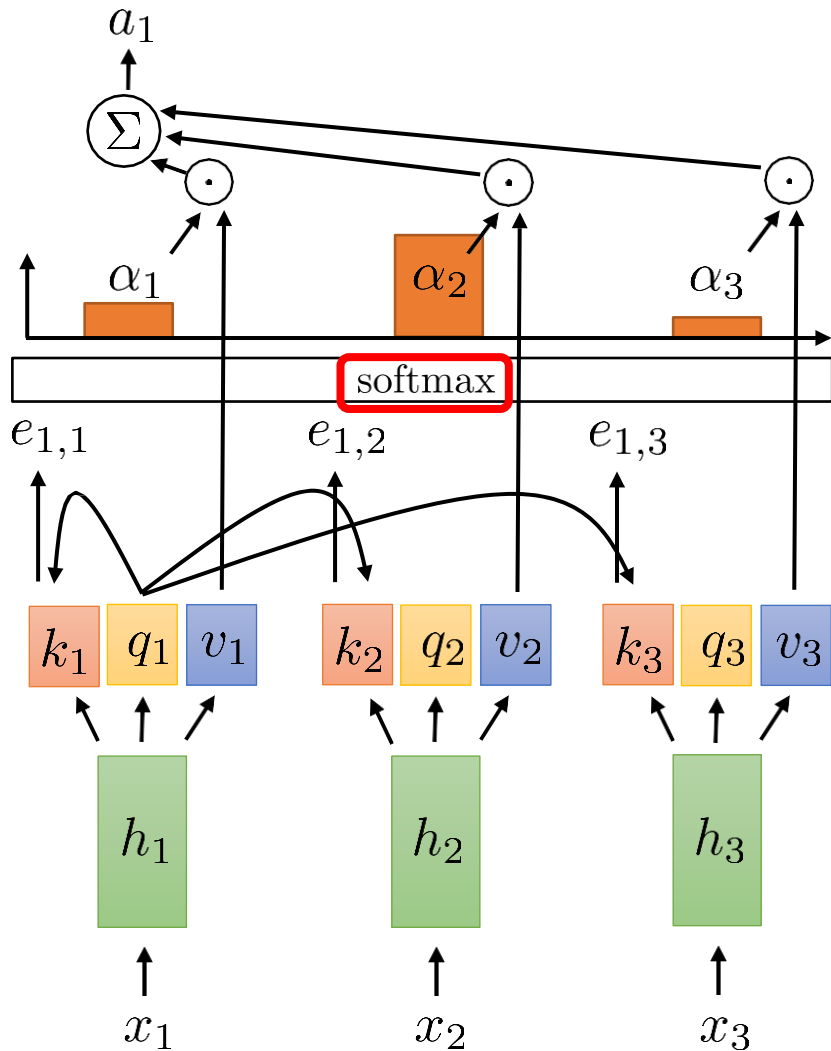
$$e_{l,t} = q_l \cdot k_t$$

$v_t = v(h_t)$ before just had $v(h_t) = h_t$, now e.g. $v(h_t) = W_v h_t$

$k_t = k(h_t)$ (just like before) e.g., $k_t = W_k h_t$

$q_t = q(h_t)$ e.g., $q_t = W_q h_t$

Key-Value Self-Attention



$$a_l = \sum_t \alpha_{l,t} v_t$$

$$\alpha_{l,t} = \exp(e_{l,t}) / \sum_{t'} \exp(e_{l,t'})$$

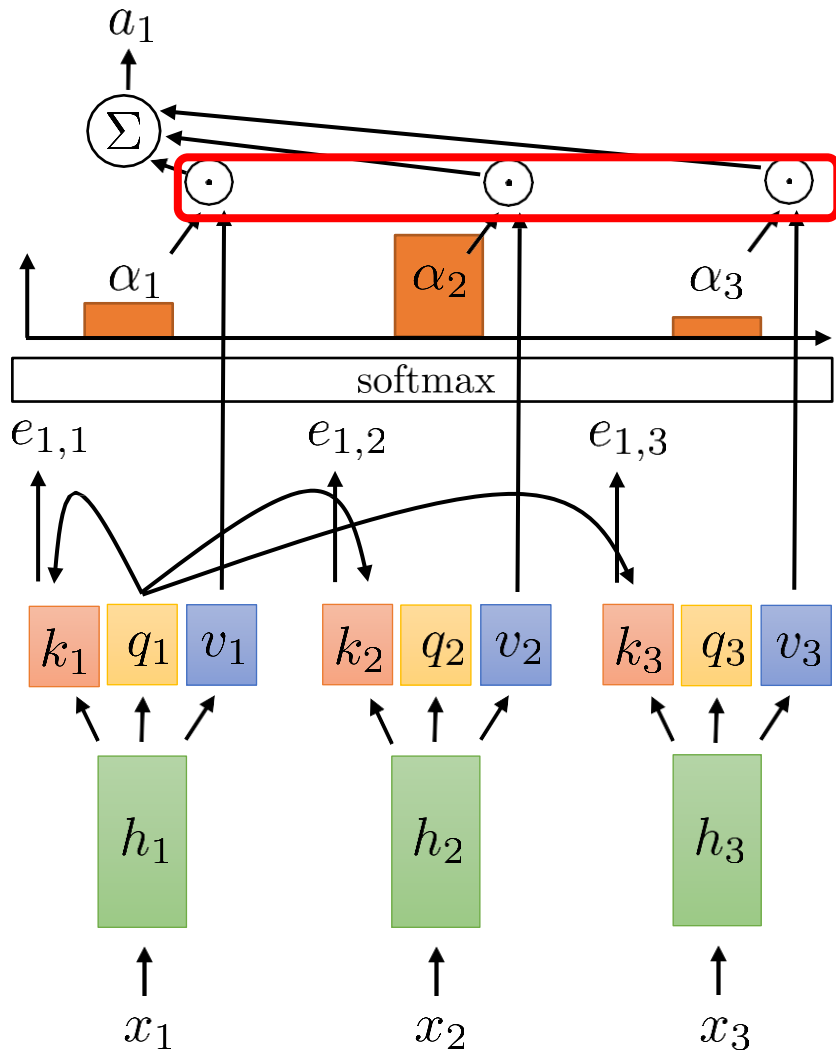
$$e_{l,t} = q_l \cdot k_t$$

$v_t = v(h_t)$ before just had $v(h_t) = h_t$, now e.g. $v(h_t) = W_v h_t$

$k_t = k(h_t)$ (just like before) e.g., $k_t = W_k h_t$

$q_t = q(h_t)$ e.g., $q_t = W_q h_t$

Key-Value Self-Attention



$$a_l = \sum_t \alpha_{l,t} v_t$$

$$\alpha_{l,t} = \exp(e_{l,t}) / \sum_{t'} \exp(e_{l,t'})$$

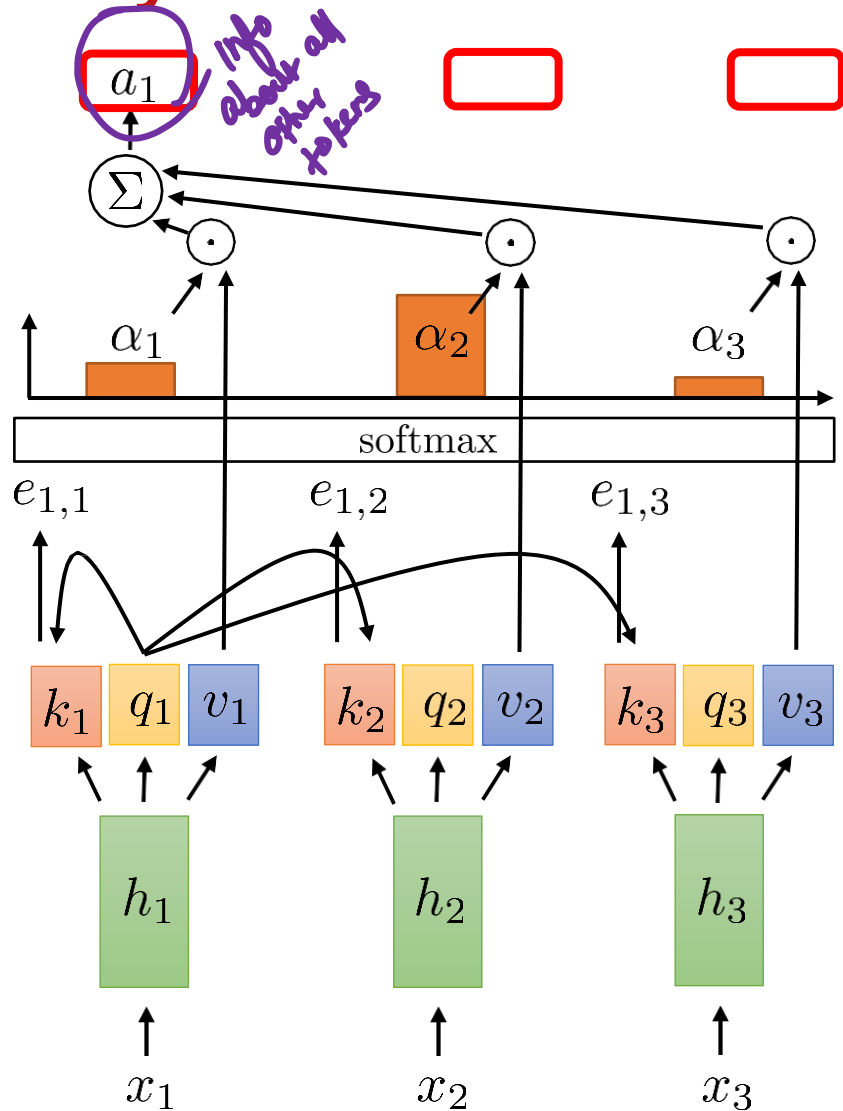
$$e_{l,t} = q_l \cdot k_t$$

$v_t = v(h_t)$ before just had $v(h_t) = h_t$, now e.g. $v(h_t) = W_v h_t$

$k_t = k(h_t)$ (just like before) e.g., $k_t = W_k h_t$

$q_t = q(h_t)$ e.g., $q_t = W_q h_t$

Key-Value Self-Attention



$$a_l = \sum_t \alpha_{l,t} v_t$$

$$\alpha_{l,t} = \exp(e_{l,t}) / \sum_{t'} \exp(e_{l,t'})$$

$$e_{l,t} = q_l \cdot k_t$$

$v_t = v(h_t)$ before just had $v(h_t) = h_t$, now e.g. $v(h_t) = W_v h_t$

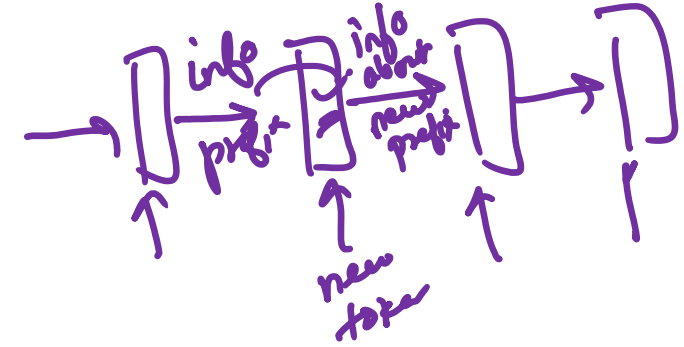
$k_t = k(h_t)$ (just like before) e.g., $k_t = W_k h_t$

$q_t = q(h_t)$ e.g., $q_t = W_q h_t$

Attention → Transformers

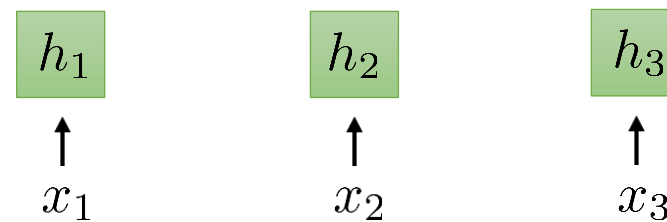
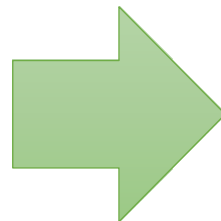
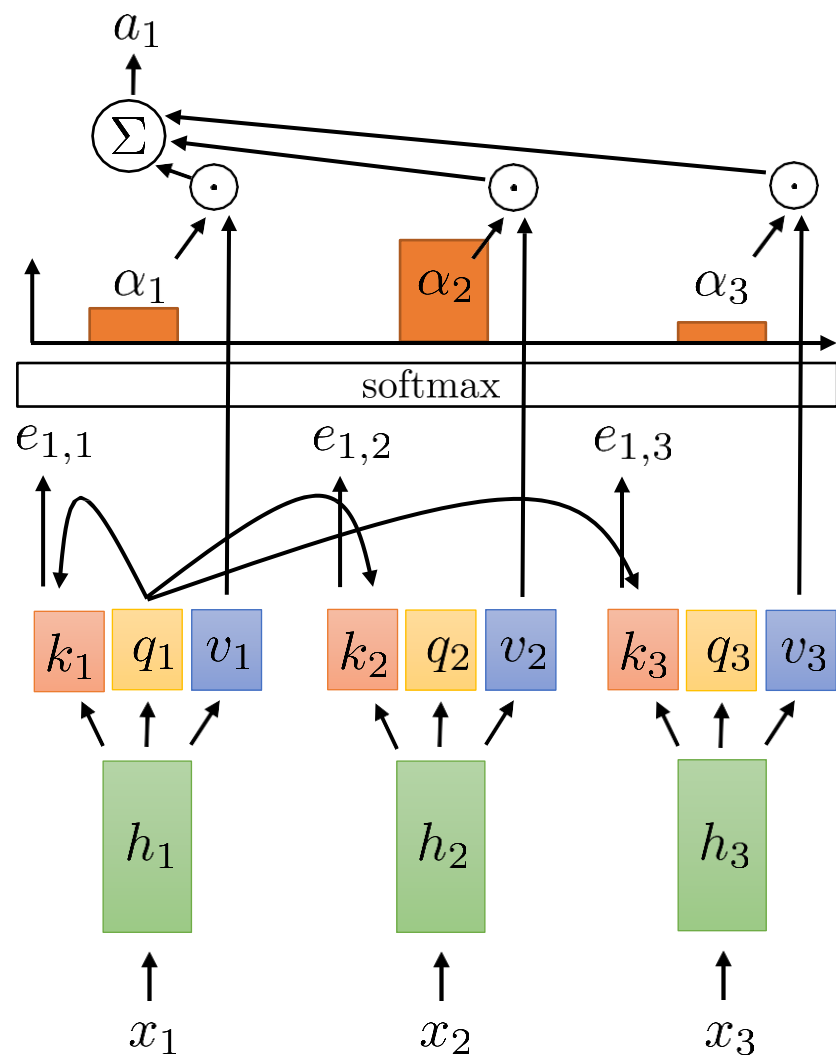


Do we need recurrence at all?

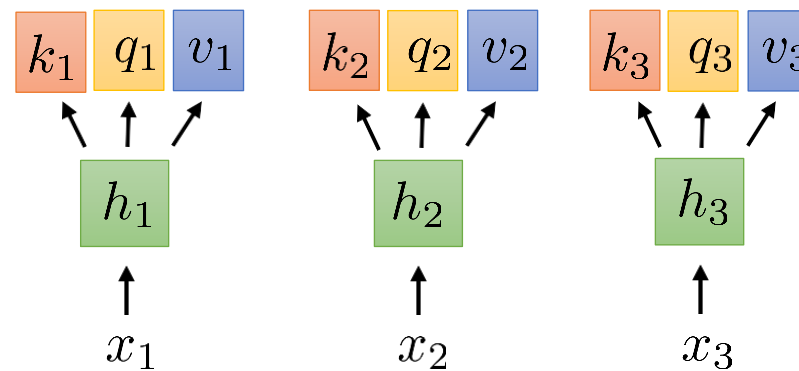
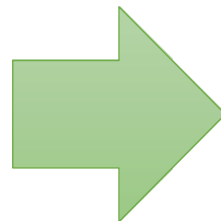
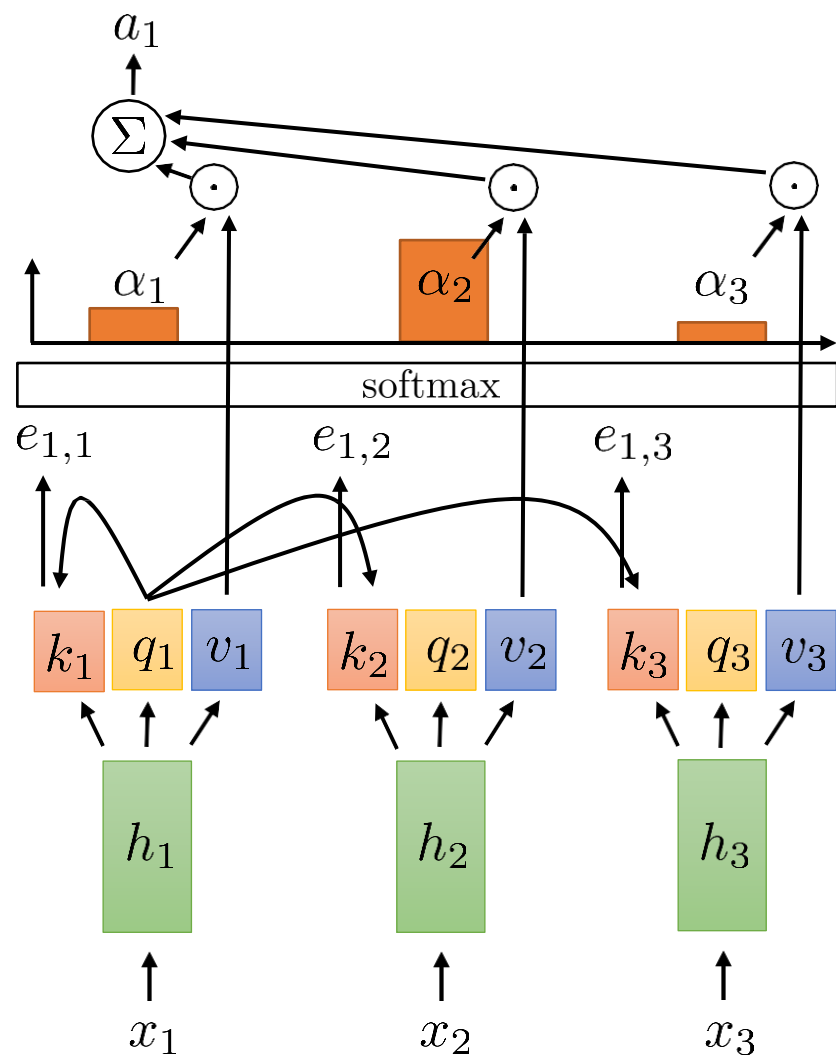


- Recurrence is powerful but
 - Issues with learnability: vanishing gradients
 - Issues with remembering long sentences
 - Issues with scalability:
 - backpropagation time high due to sequentiality in sentence length
 - Issues with scalability:
 - can't be parallelized even at test time – $O(\text{sentence length})$
- Abstractly: Attention is a way to pass information between tokens
 - That is also exactly what RNNs are used for – to pass information
 - Get rid of RNN? – maybe attention is a better way to pass information
- Attempt 1: layers of Self Attention

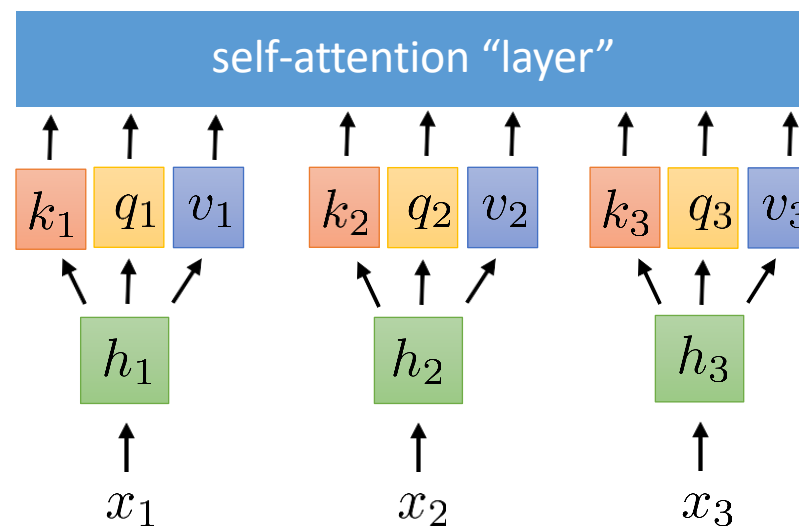
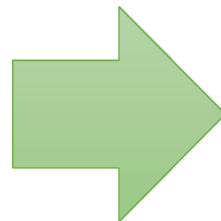
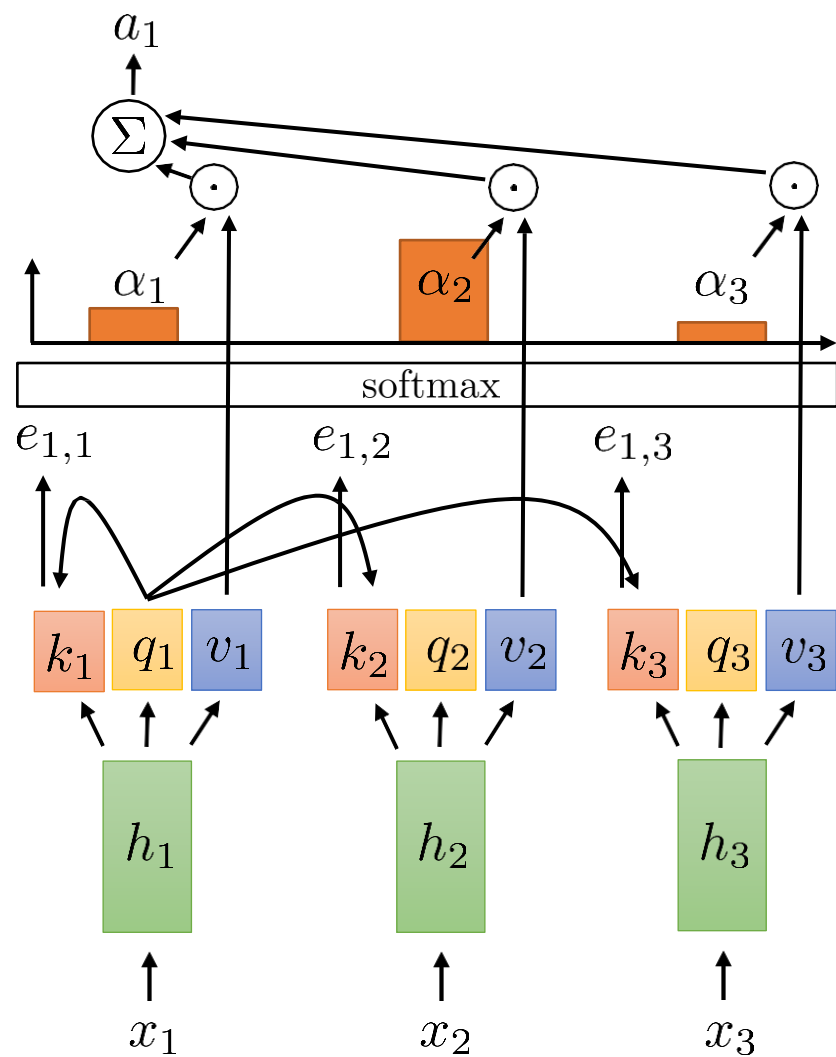
Attempt



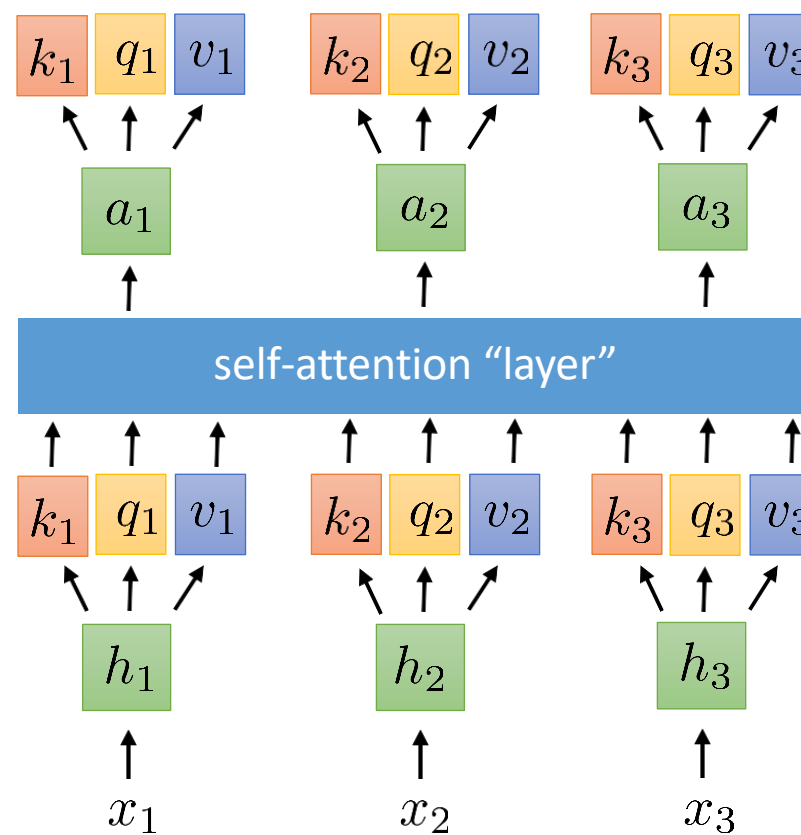
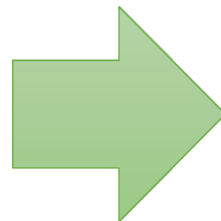
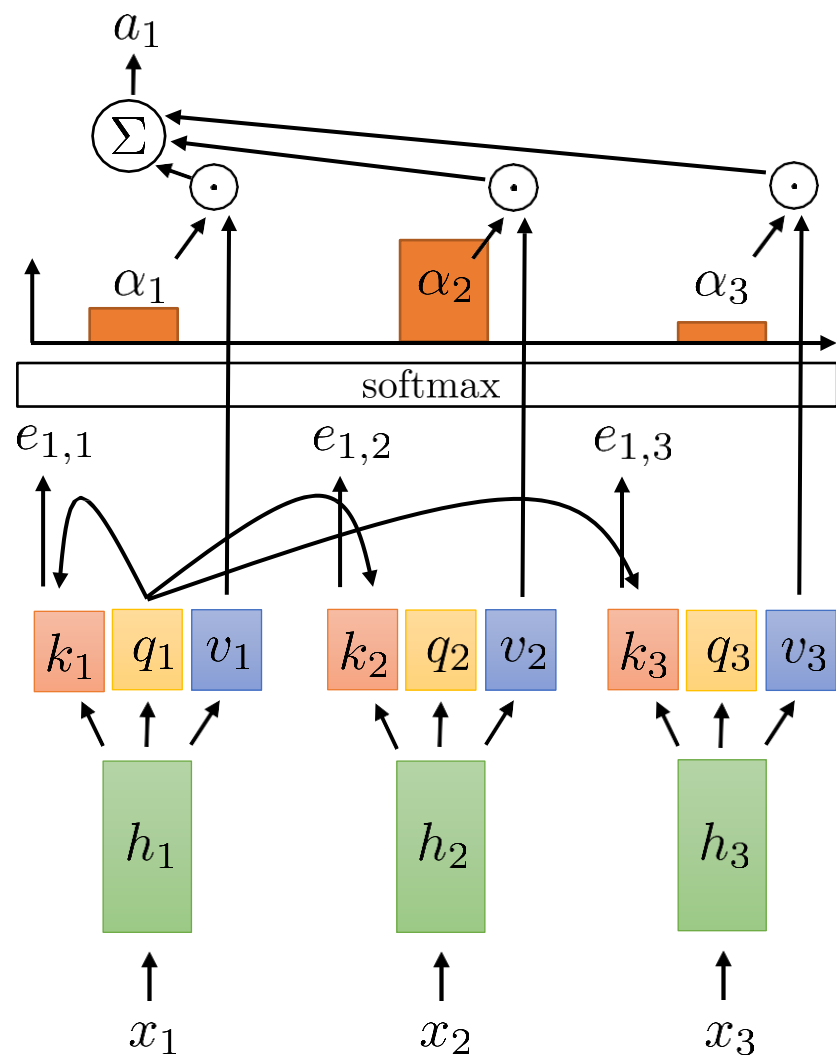
Attempt



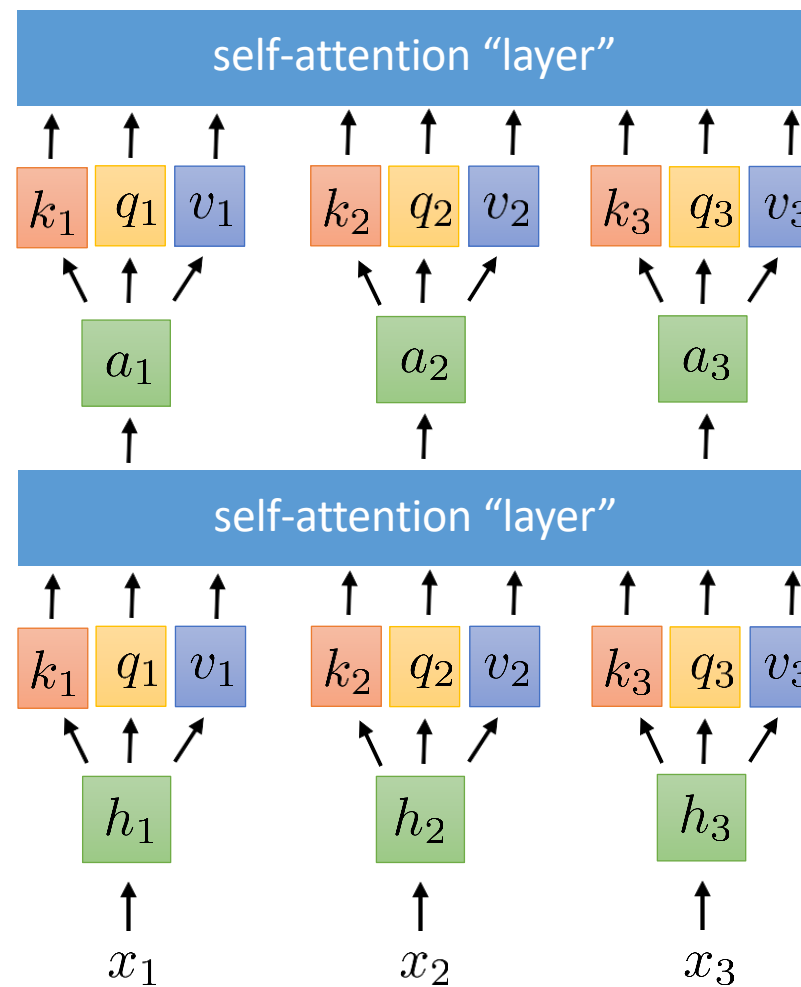
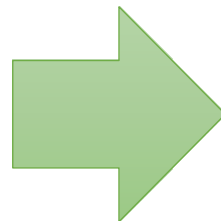
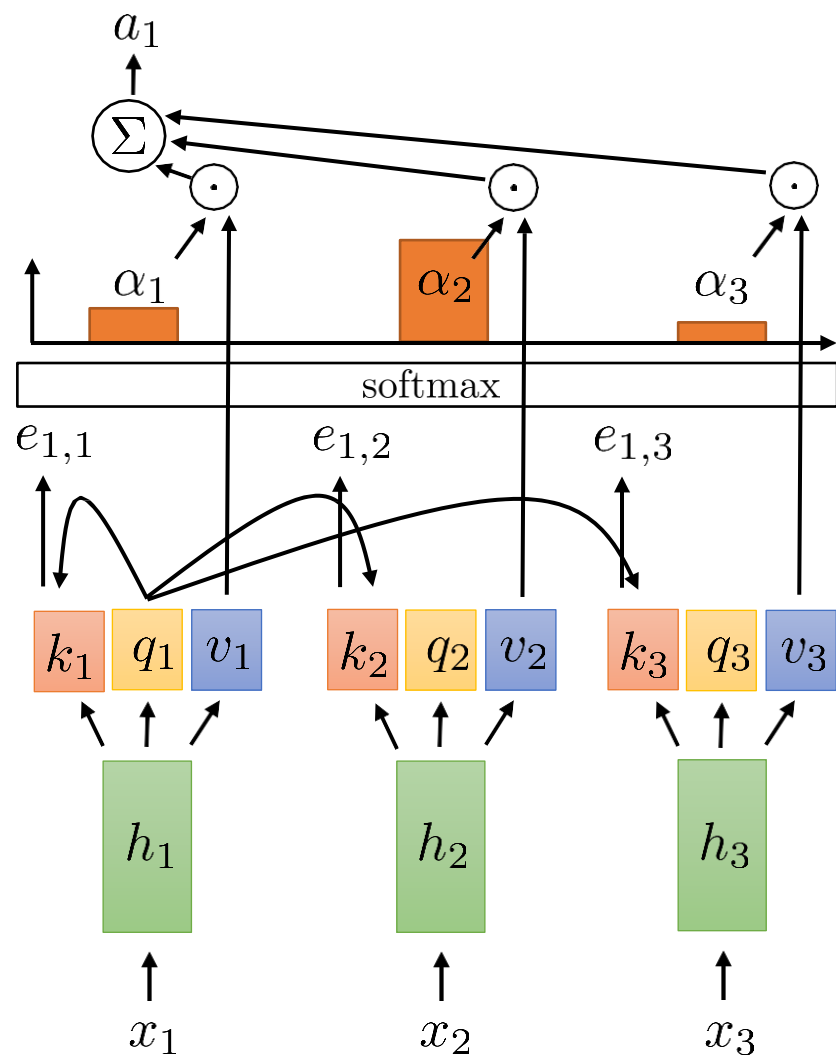
Attempt



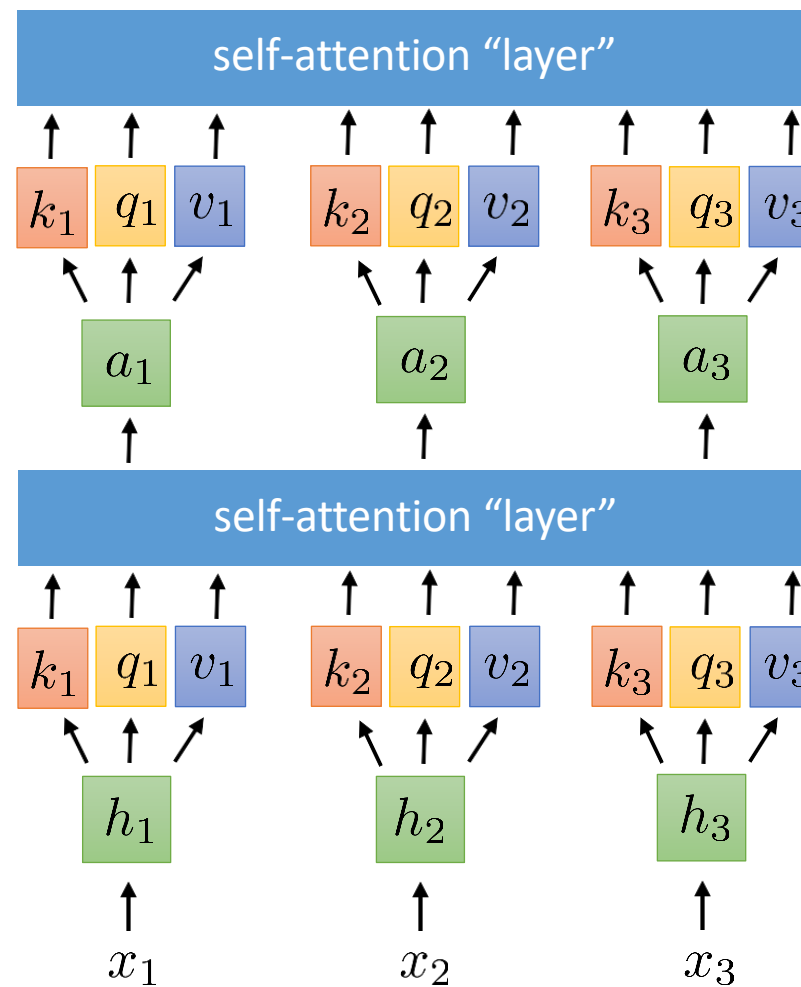
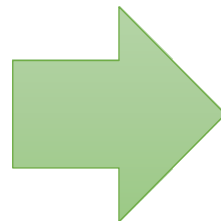
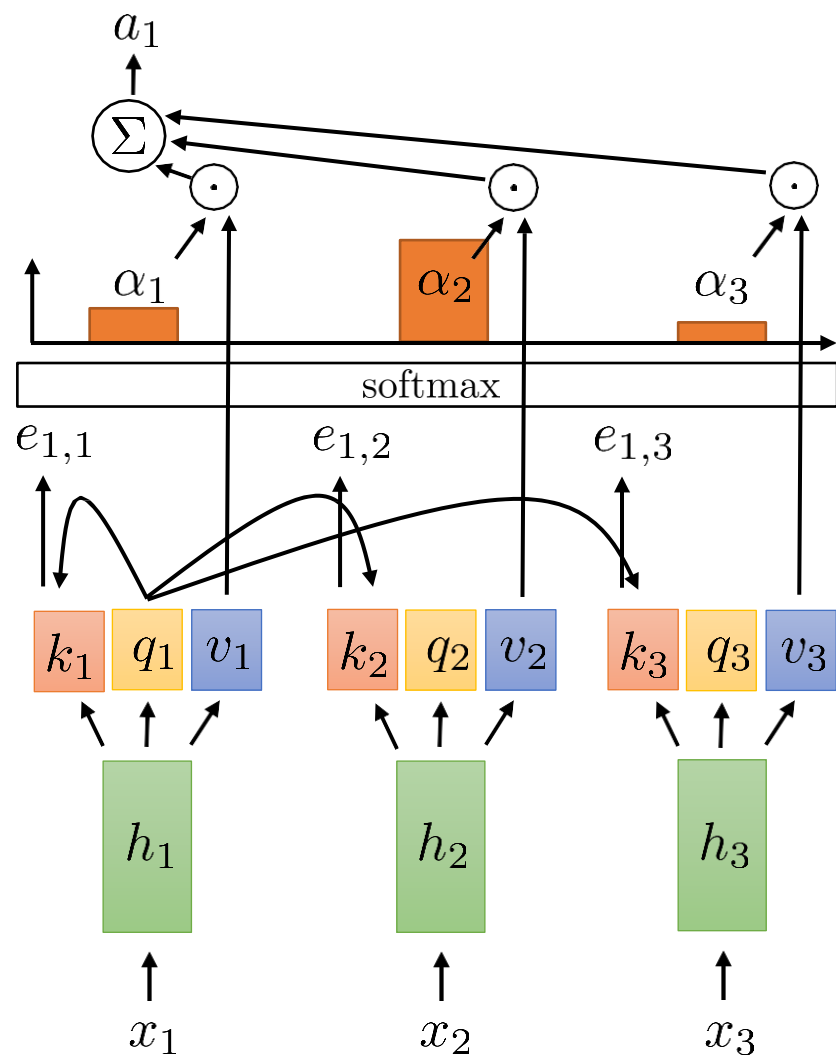
Attempt



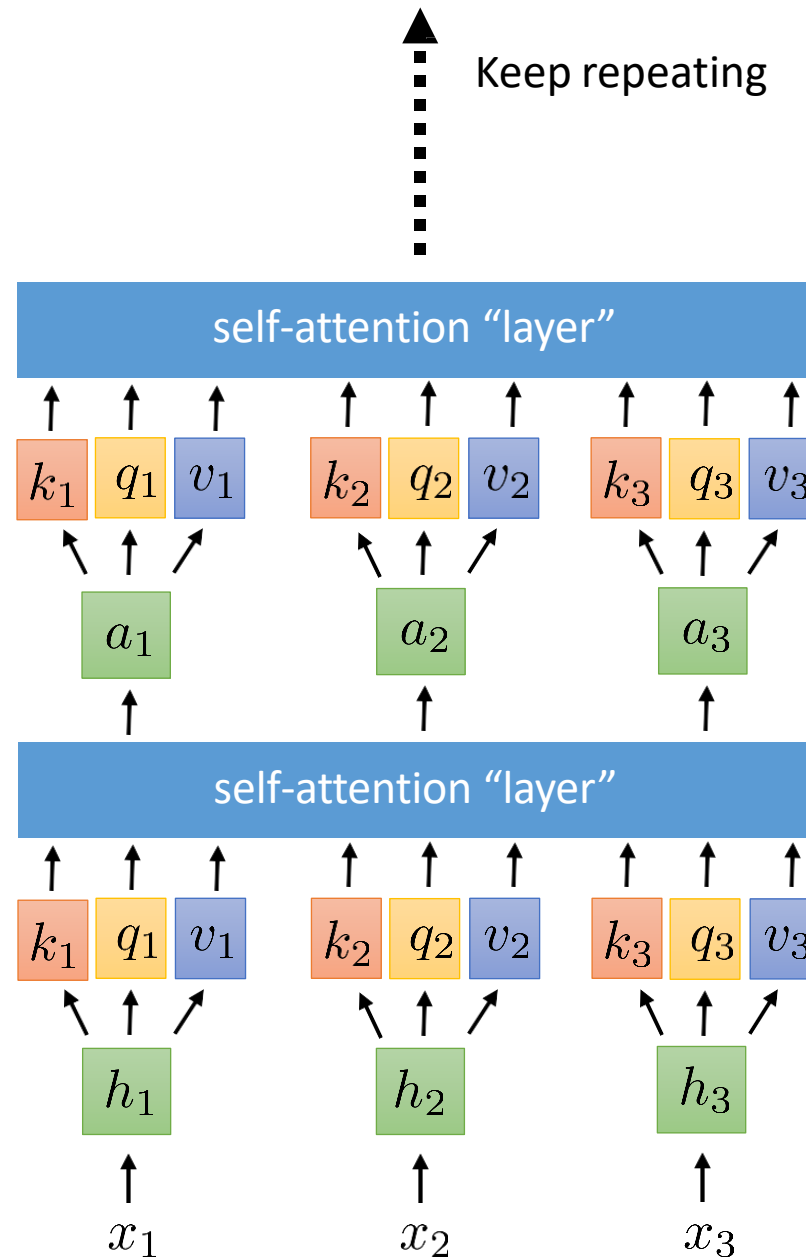
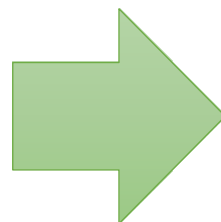
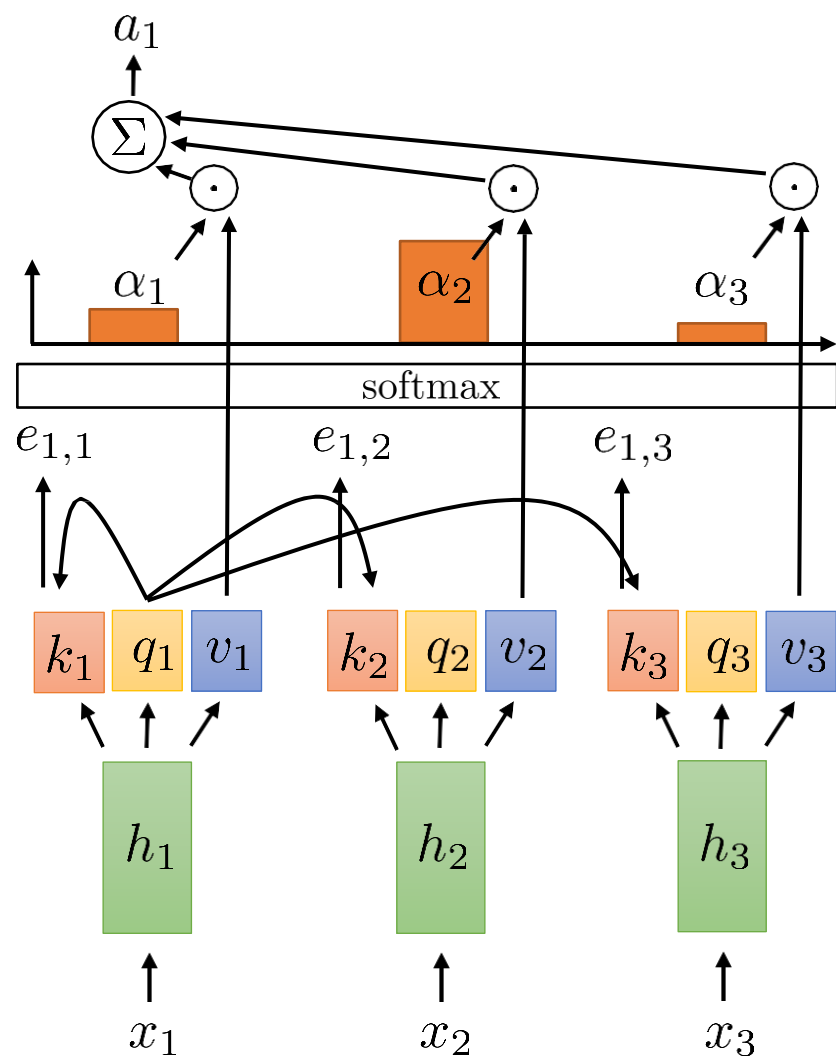
Attempt



Attempt



Attempt

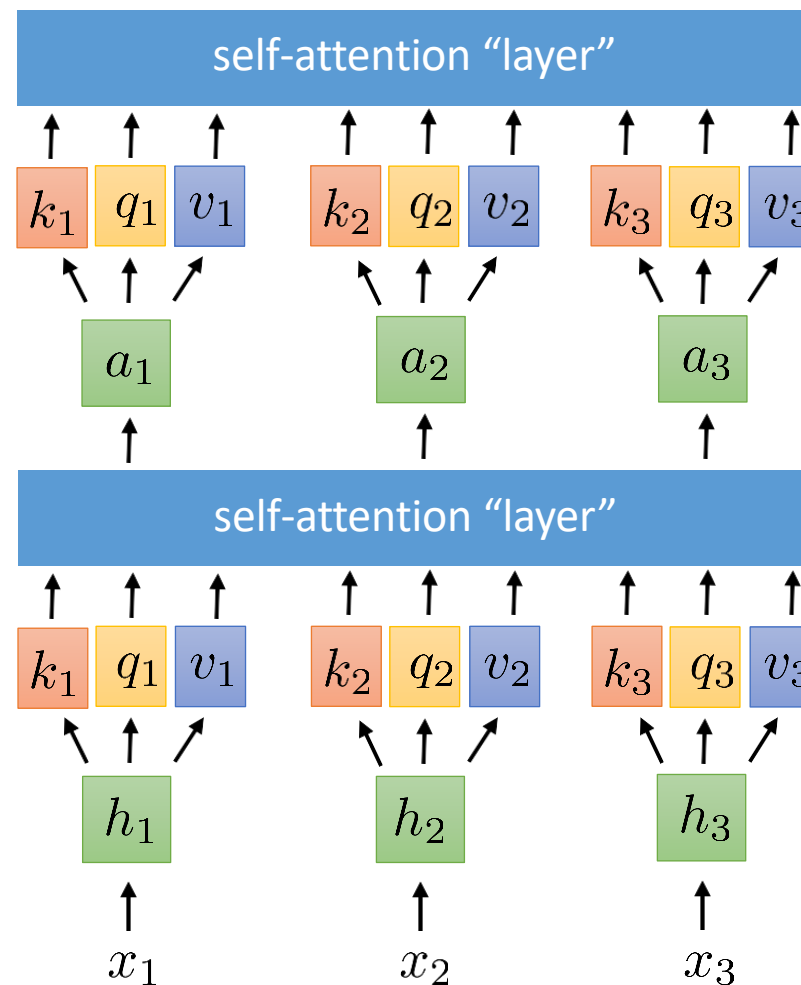
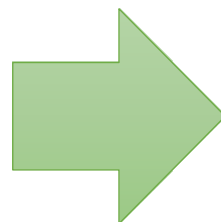
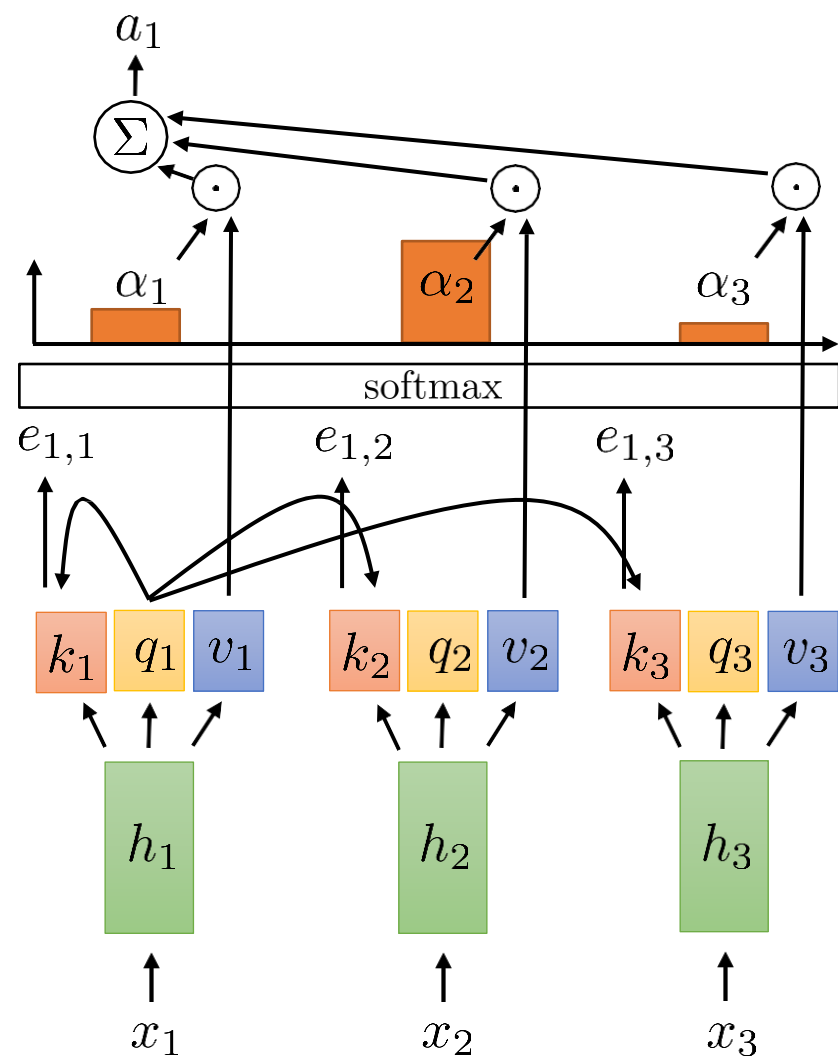


Issues with Attempt

- Complete lack of sequential information!
 - Solution: add sequential information in the input

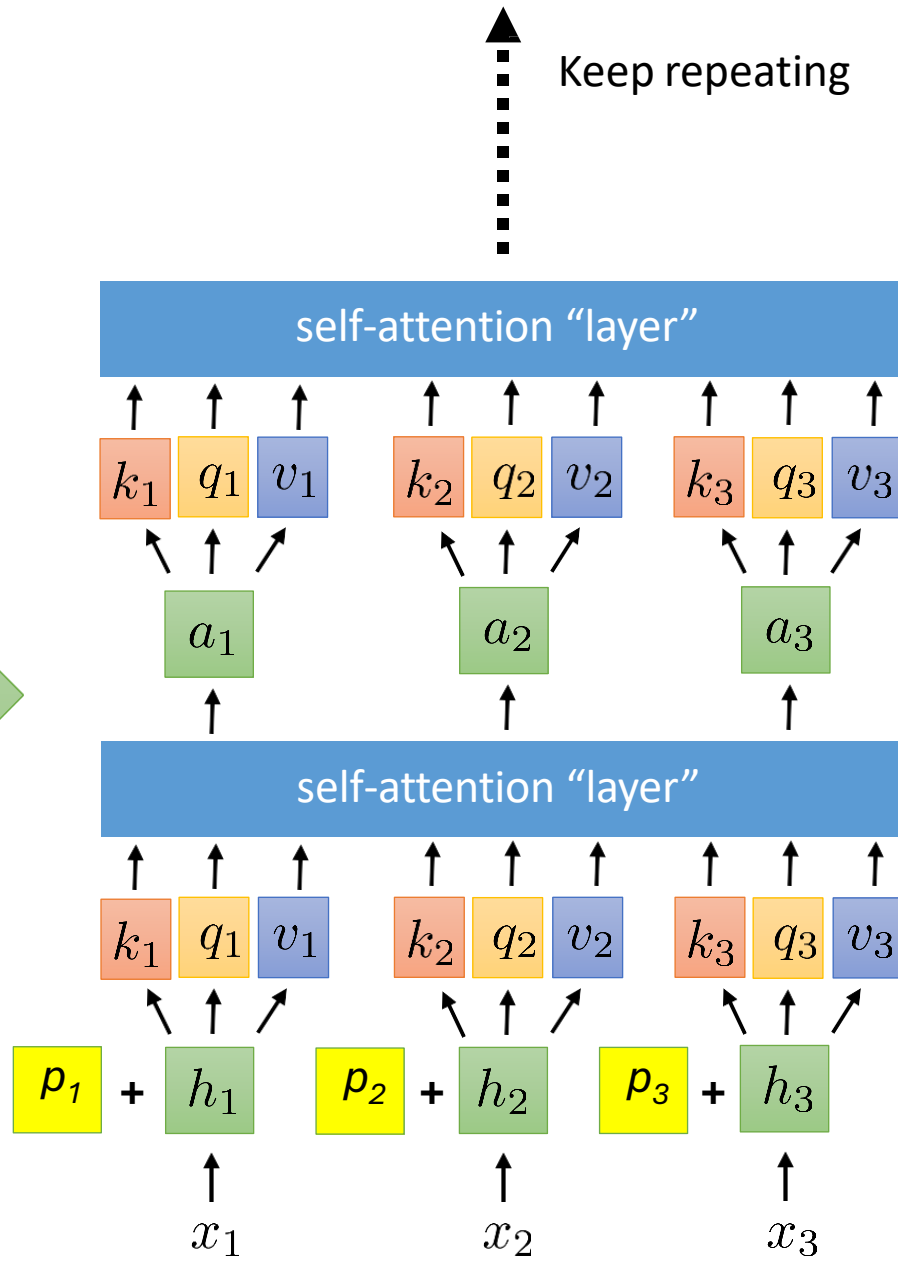
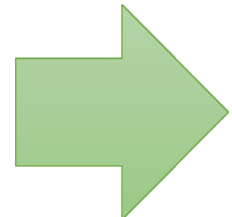
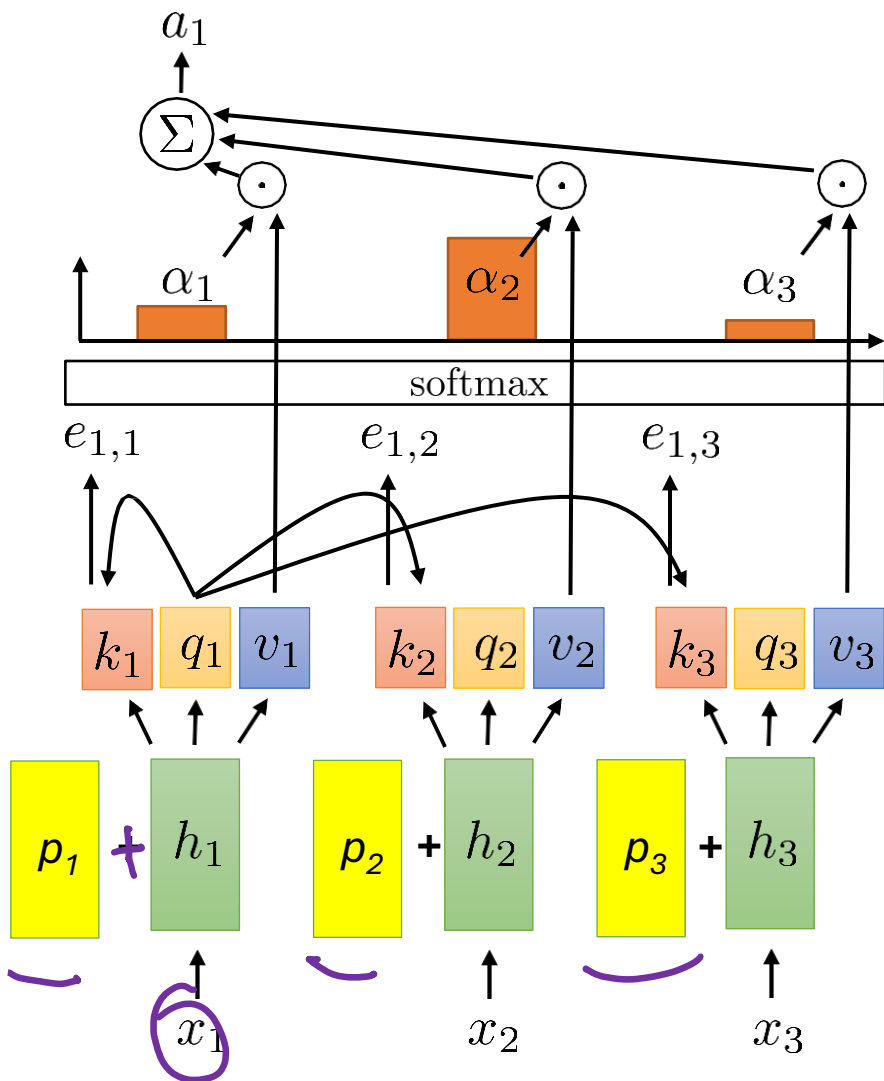


Key-Value Self-Attention



Bag
Repr.
N inputs
(seq ind.)
↓
N vector
outputs

Key-Value Self-Attention



Sinusoidal
Absolute
Relative



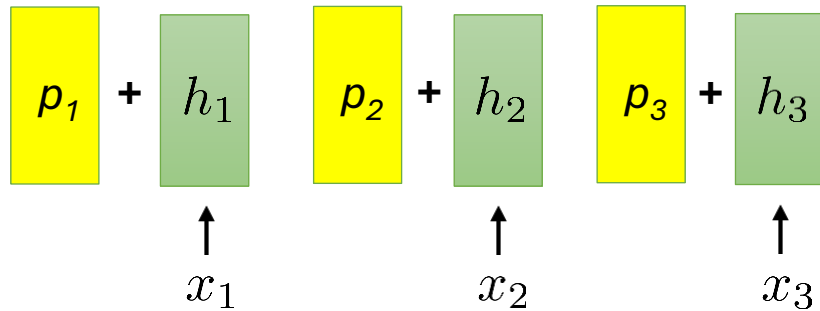
Issues with Attempt

- Complete lack of sequential information!
 - Positional Embeddings
- Different tokens are important depending upon the attribute
 - “John gave Mary a book yesterday”
 - “gave” attends to “John” //subject
 - “gave” attends to “Mary” //recipient
 - “gave” attends to “book” //theme (direct object)
 - “gave” attends to “yesterday” //temporal
- Solution: repeat computations for different attributes
 - Induce attributes automatically



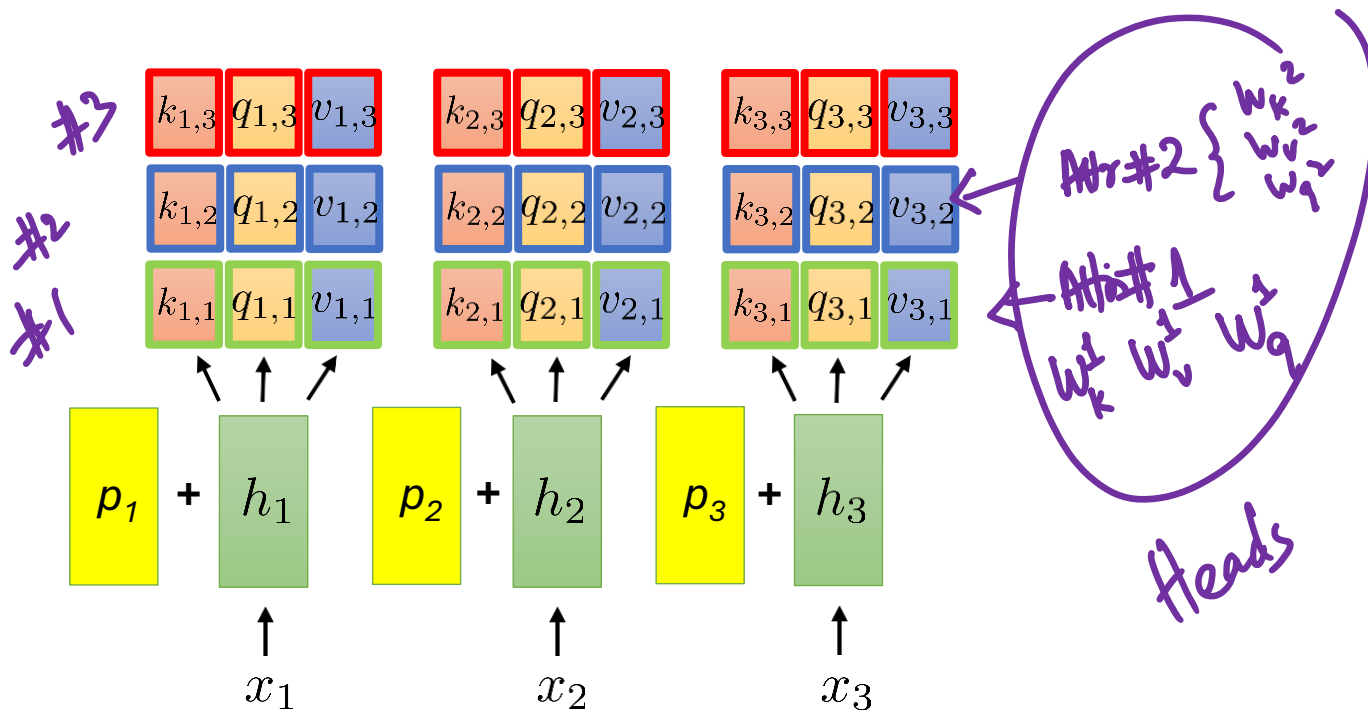
Multi-Head Attention

Solution: Use multiple keys, queries, and values for each time step



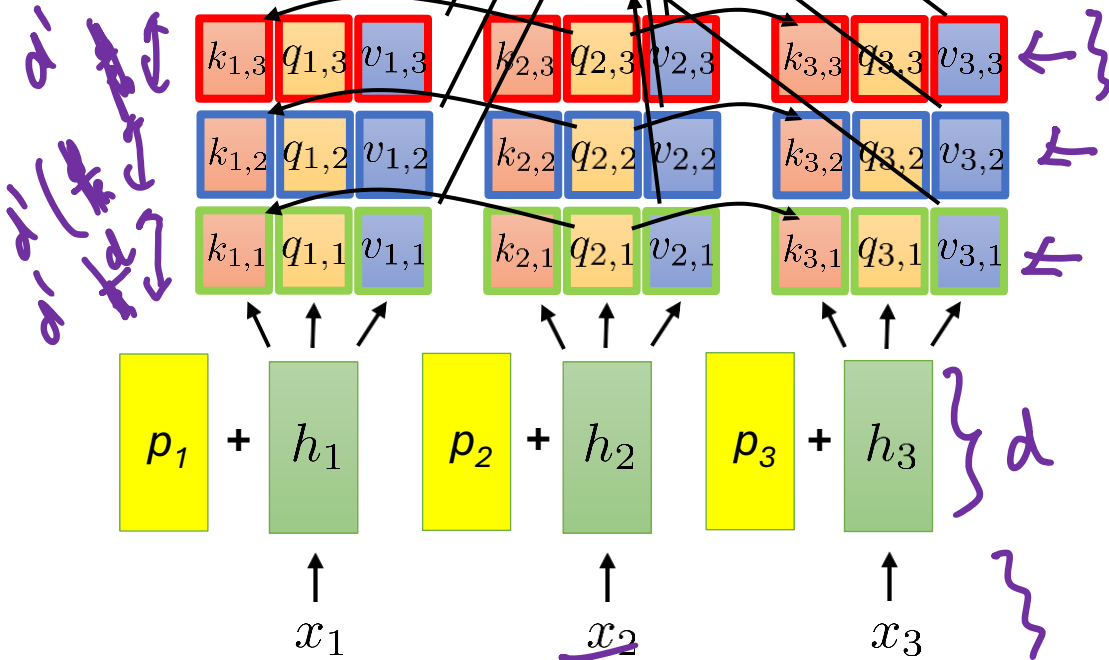
Multi-Head Attention

Solution: Use multiple keys, queries, and values for each time step



Multi-Head Attention

Solution: Use multiple keys, queries, and values for each time step



full attention vector formed by concatenation:

$$d \rightarrow a_2 = \begin{bmatrix} a_{2,1} \\ a_{2,2} \\ a_{2,3} \end{bmatrix}$$

$$\frac{k \times d'}{h} = d$$

compute weights **independently** for each head

$$e_{l,t,i} = q_{l,i} \cdot k_{l,i}$$

$$\alpha_{l,t,i} = \exp(e_{l,t,i}) / \sum_{t'} \exp(e_{l,t',i})$$

$$a_{l,i} = \sum_t \alpha_{l,t,i} v_{t,i}$$

$$W_k^{(h)} W_v^{(h)} W_q^{(h)}$$

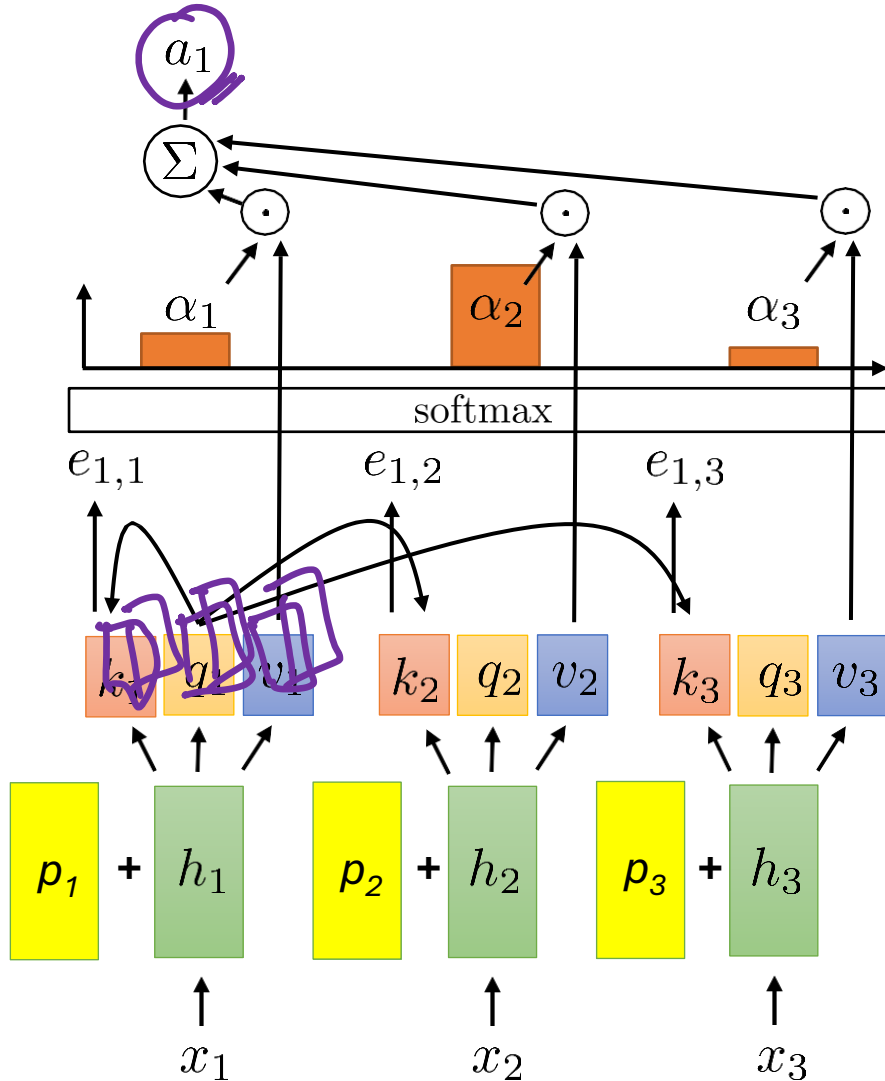


Issues with Attempt

- Complete lack of sequential information!
 - Positional Embeddings
- Different tokens are important depending upon the attribute
 - Multi-Head Attention
- So far, each successive layer is linear
 - Introduce non-linearities



Self-Attention Is “Linear”



$$k_t = W_k h_t \quad q_t = W_q h_t \quad v_t = W_v h_t$$

$$\alpha_{l,t} = \exp(e_{l,t}) / \sum_{t'} \exp(e_{l,t'})$$

$$e_{l,t} = q_l \cdot k_t$$

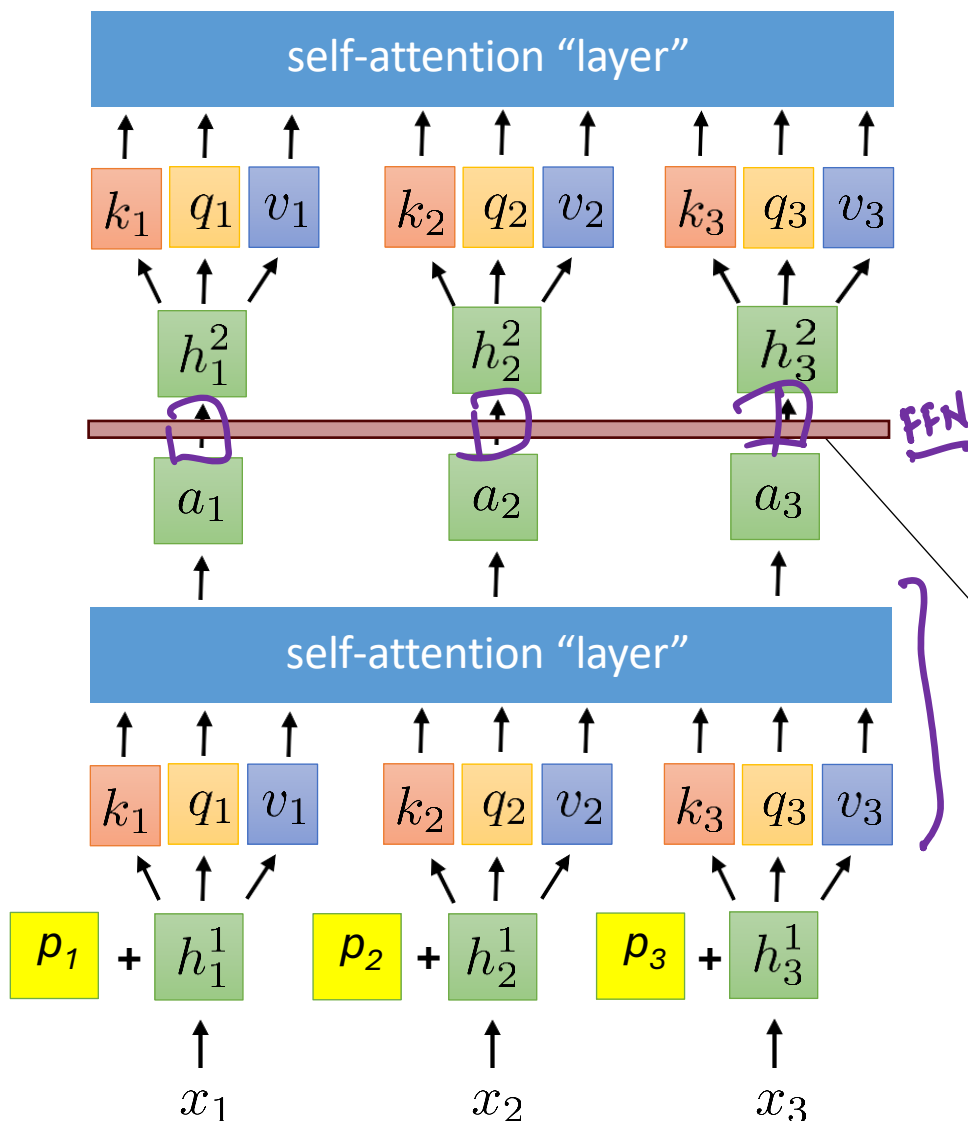
$$a_l = \sum_t \alpha_{l,t} v_t = \sum_t \alpha_{l,t} W_v h_t = W_v \sum_t \alpha_{l,t} h_t$$

linear combination (pointing to the summation $\sum_t \alpha_{l,t} h_t$)
 linear transformation (pointing to W_v)
 non-linear weights (pointing to $\alpha_{l,t}$)

Problem: Every self-attention layer is a linear transformation of the previous layer with non-linear weights.

Position-wise Feed-Forward Networks

*Parameters are layer/dep
but not pos.-dependent*



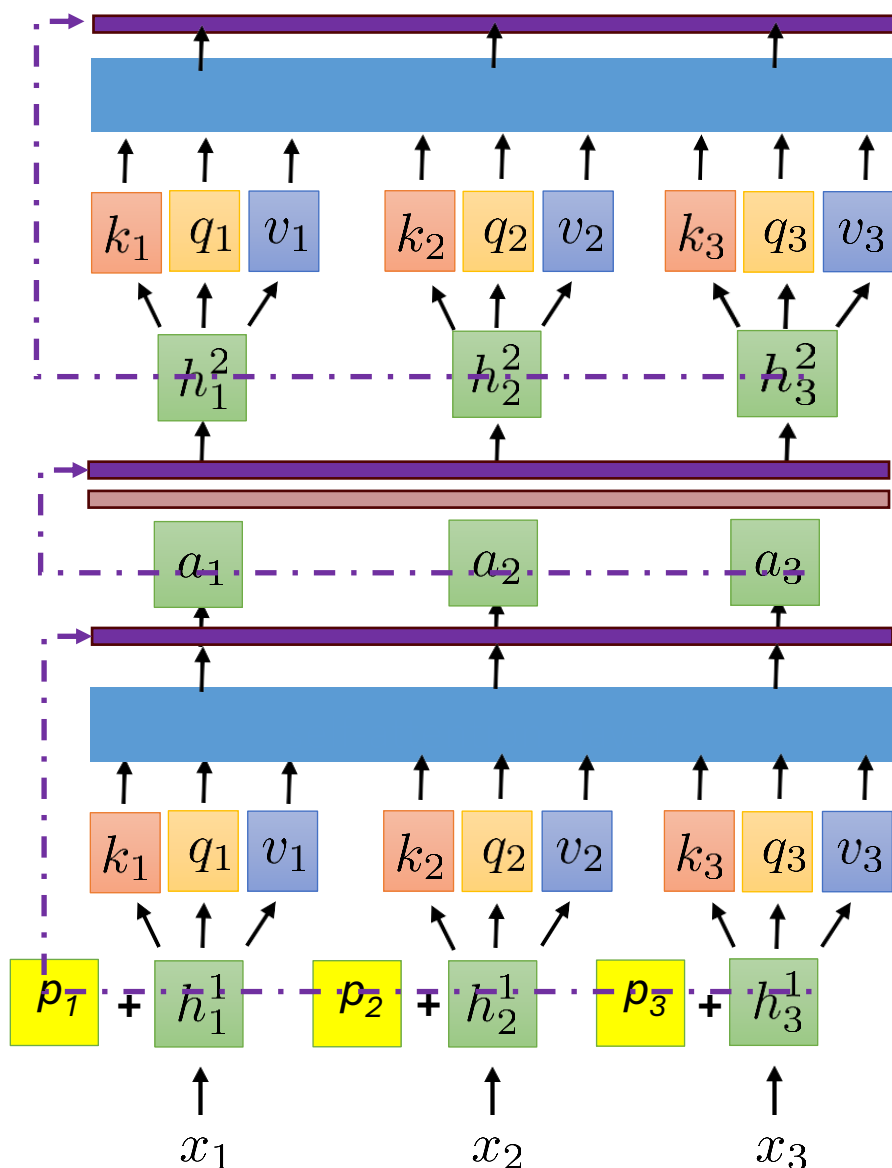
- **Solution** : Make the model more expressive is by alternating use of self-attention and non-linearity.
 - Non-linearity is incorporated by means of a feed-forward network which consists of two linear transformations with a ReLU activation in between.
- $$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$
- The same non-linearity is utilized across various positions but they differ from layer to layer.

Issues with Attempt

- Complete lack of sequential information!
 - Positional Embeddings
- Different tokens are important depending upon the attribute
 - Multi-Head Attention
- So far, each successive layer is linear
 - Feedforward Network in every layer
- Training Stability & Vanishing Gradients
 - Normalize ; Residual Connections



Layer Norm & Residual Connections



different *dimensions* of a

$$\mu = \frac{1}{d} \sum_{i=1}^d a_i$$

1-dim

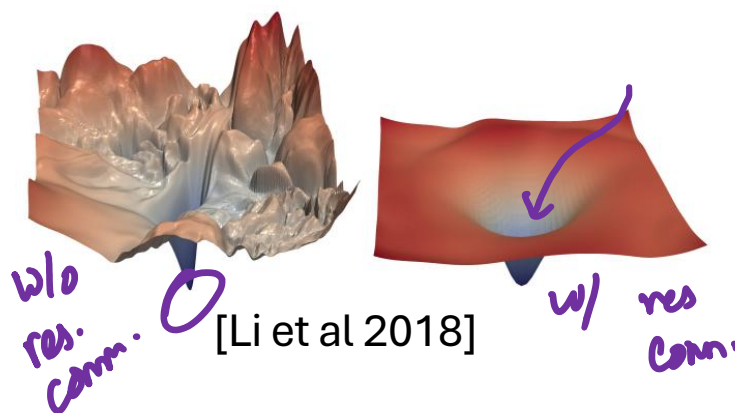
$$\sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (a_i - \mu)^2}$$

$$\bar{a} = \frac{a - \mu}{\sigma} \gamma + \beta$$

Layer Norm. convert each vector to controlled mean and variance

gain

bias



Issues with Attempt

- Complete lack of sequential information!

- Positional Embeddings

$$P(\vec{w}) = P(w_1) P(\underline{w_2} | w_1) P(\underline{w_3} | w_{1:2}) P(\underline{w_4} | w_{1:3}) \dots$$

- Different tokens are important depending upon the attribute

- Multi-Head Attention

- So far, each successive layer is linear

- Feedforward Network in every layer

- Training Stability & Vanishing Gradients

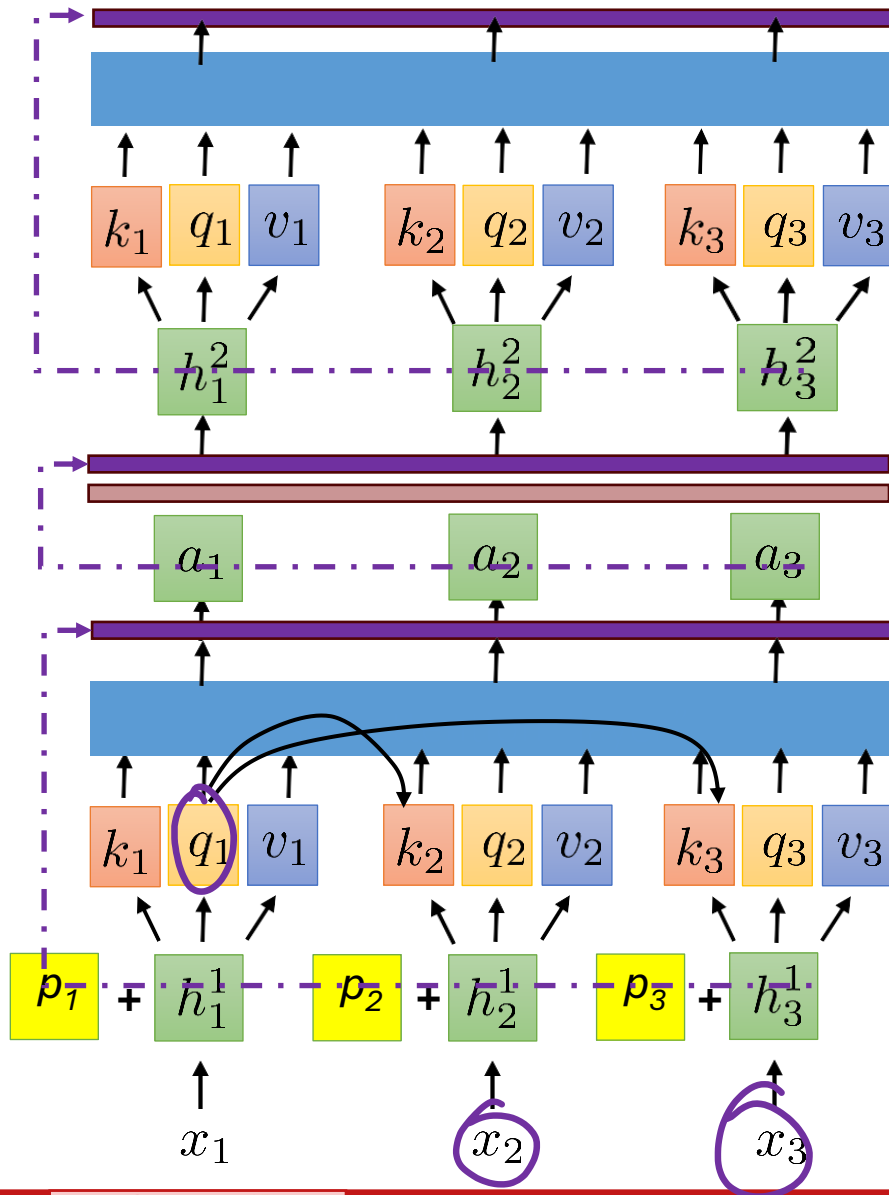
- Normalize ; Residual Connections

- Model can see the future!

- Mask attention to future tokens



Self-attention can see the future!



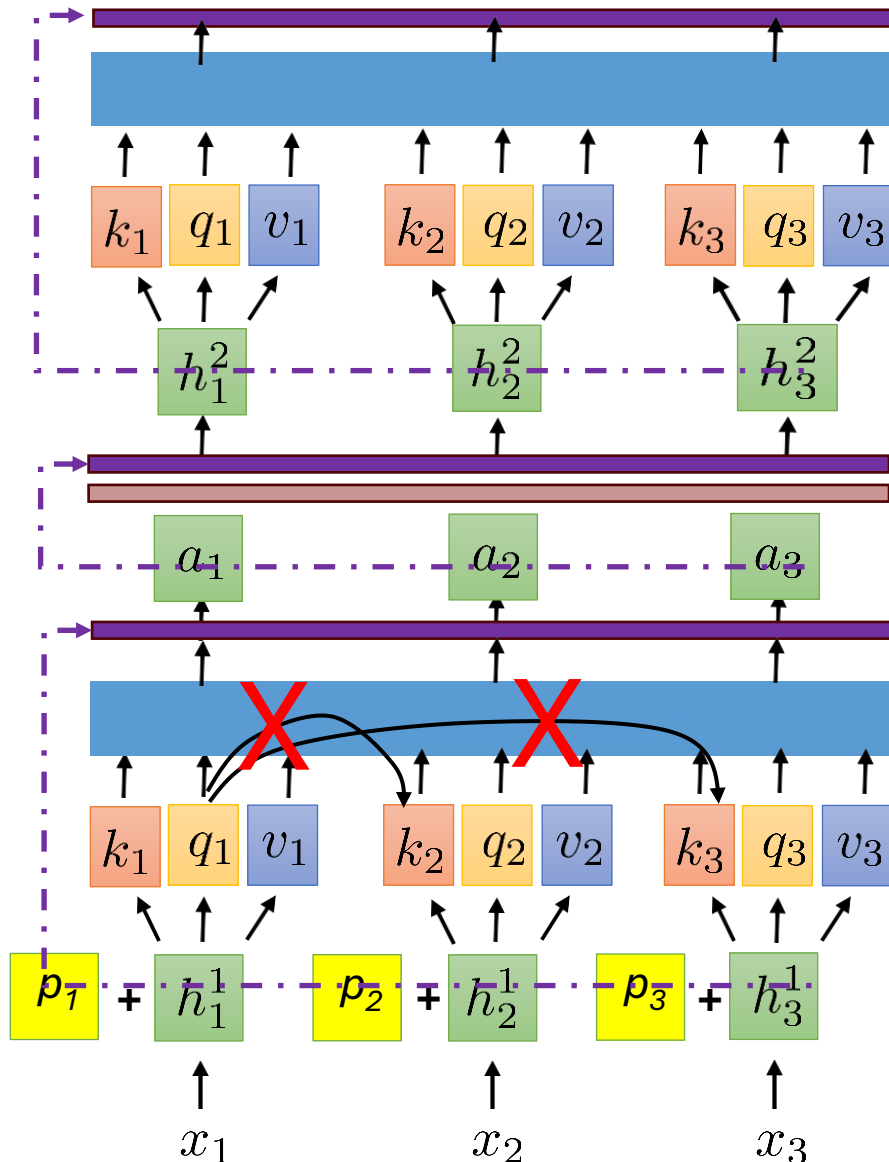
A **crude** self-attention “language model”:

In practice, there would be several alternating self-attention layers and position-wise feedforward networks

Big problem: self-attention at step 1 can look at the value at steps 2 & 3, which is based on the **inputs** at steps 2 & 3

At test time (when decoding), the **inputs** at steps 2 & 3 will be based on the output at step 1...
...which requires knowing the **input** at steps 2 & 3

Masked Attention



At test time (when decoding), the **inputs** at steps 2 & 3 will be based on the output at step 1...

...which requires knowing the **input** at steps 2 & 3

Must allow self-attention into the **past**...

...but not into the **future**

Easy solution:

~~$$e_{l,t} = q_l \cdot k_t$$~~

$$e_{l,t} = \begin{cases} q_l \cdot k_t & \text{if } l \geq t \\ -\infty & \text{otherwise} \end{cases}$$

in practice:

just replace $\exp(e_{l,t})$ with 0 if $l < t$

inside the softmax

Making

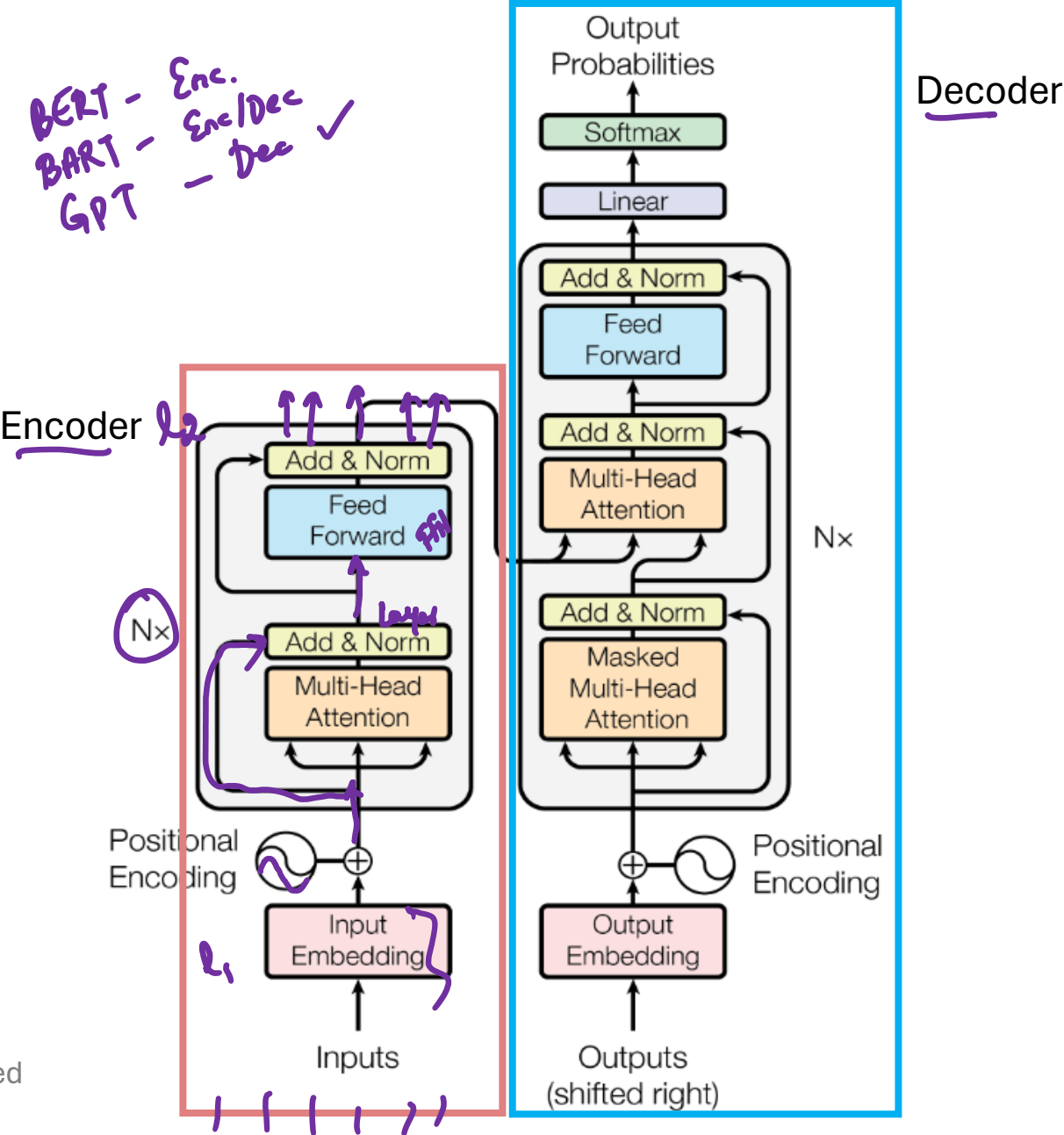
Attempt → Transformer

- Complete lack of sequential information!
 - Positional Embeddings
- Different tokens are important depending upon the attribute
 - Multi-Head Attention
- So far, each successive layer is linear
 - Feedforward Network in every layer
- Training Stability & Vanishing Gradients
 - Normalize ; Residual Connections
- Model can see the future!
 - Masked Attention



Transformer Architecture

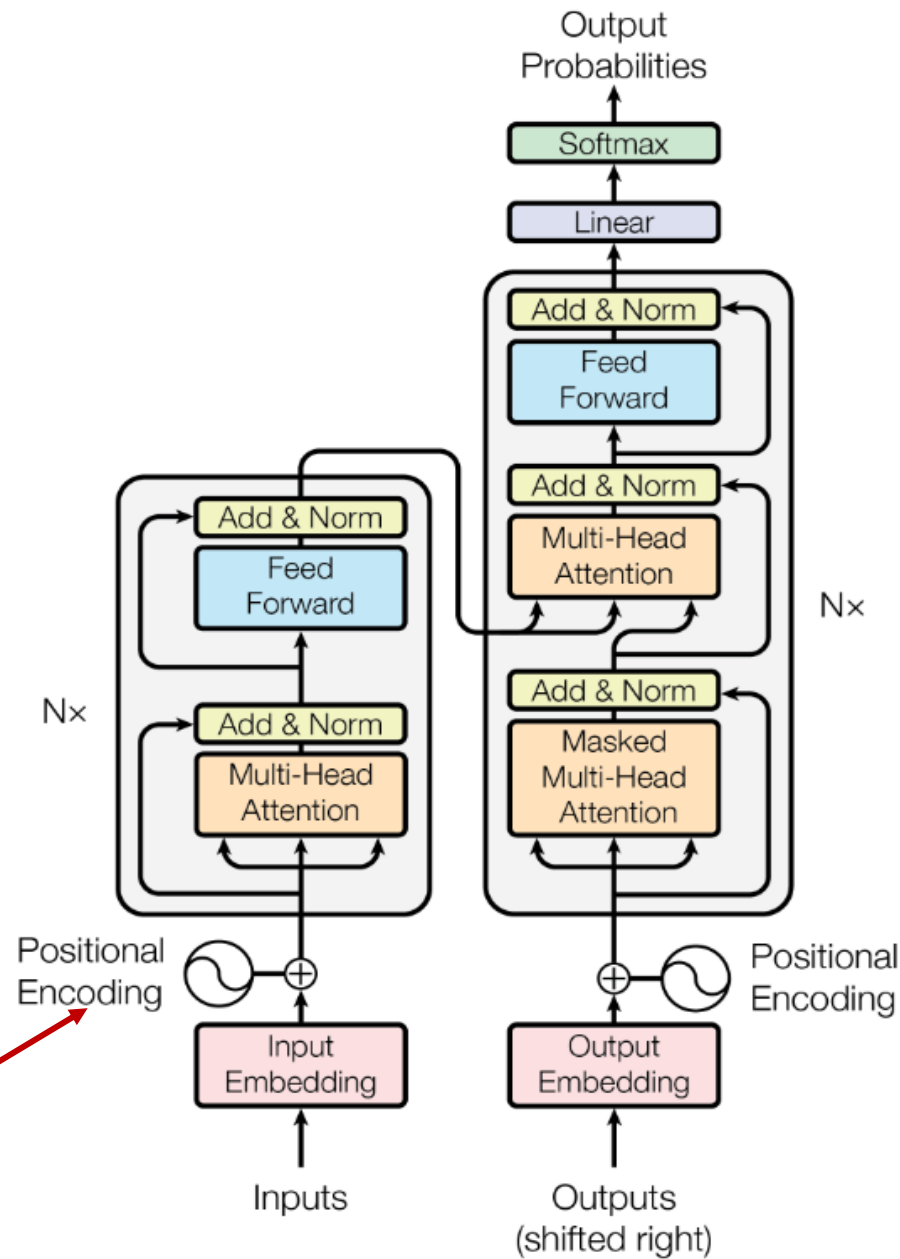
BERT - Enc.
BART - Enc/Dec
GPT - Dec ✓



Source of Image : Attention is all you need
(Vaswani et al., 2017)



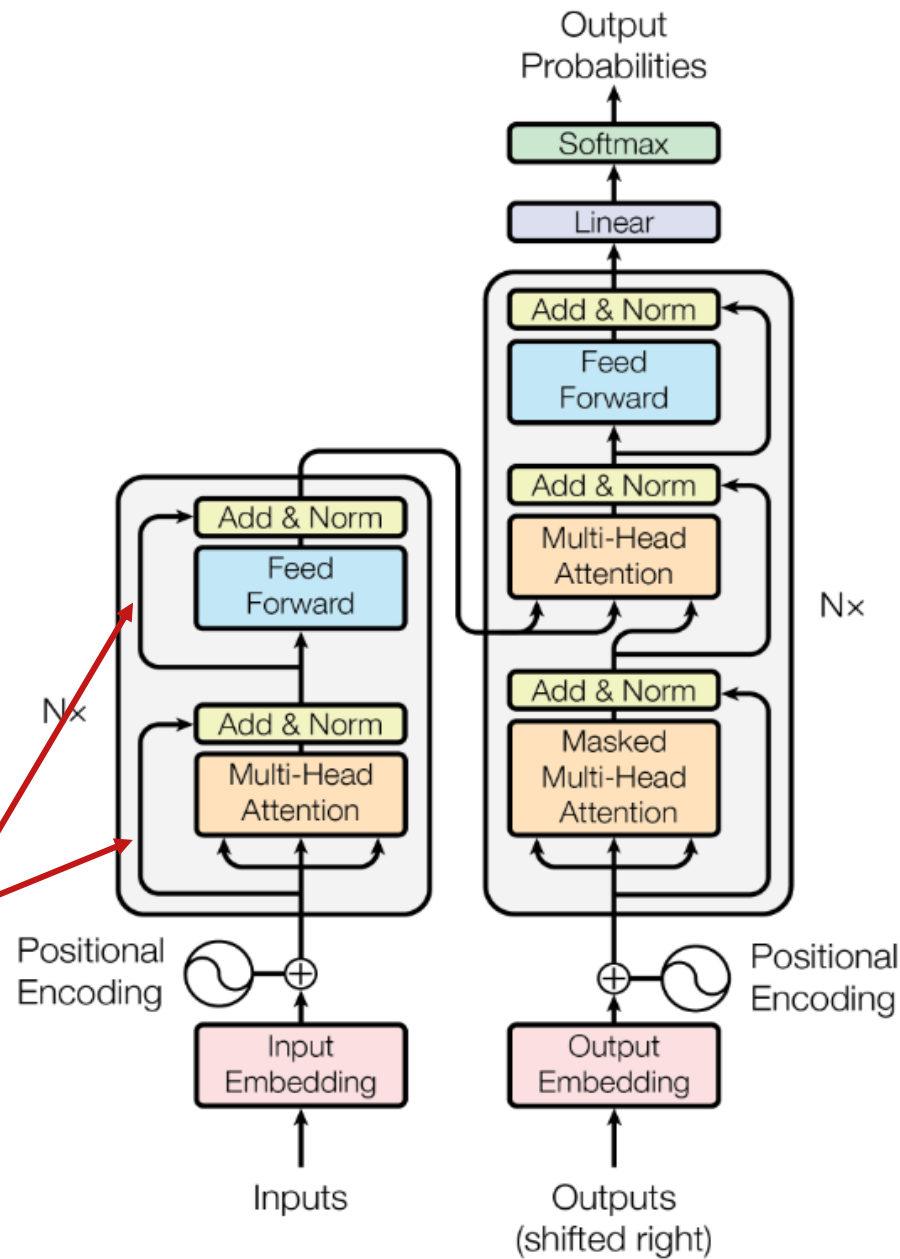
Transformer Architecture



Position embeddings are *added* to each word embedding. Otherwise, since we have no recurrence, our model is unaware of the position of a word in the sequence!

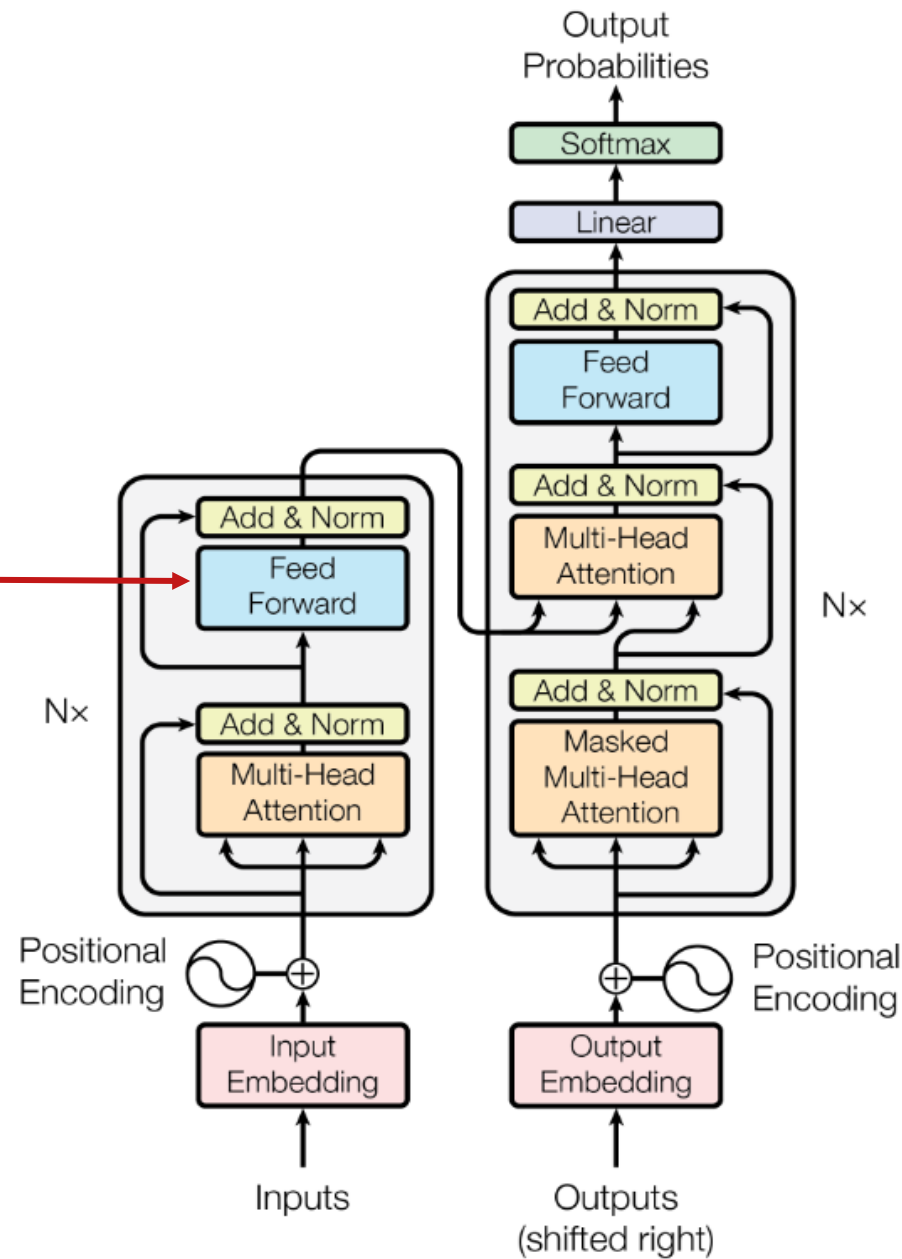
Transformer Architecture

Residual connections, which mean that we add the input to a particular block to its output, help improve gradient flow



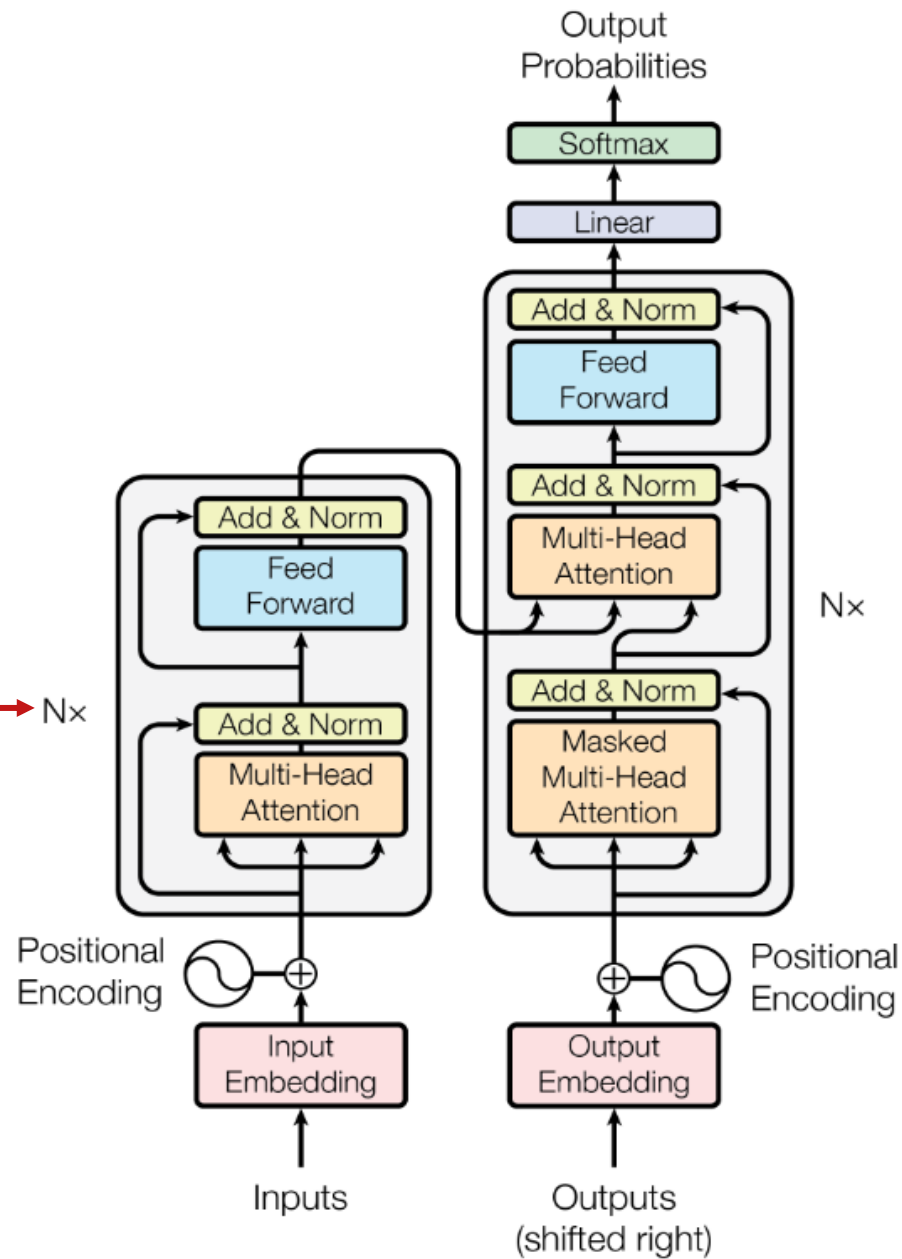
Transformer Architecture

A **feed-forward layer** on top of the attention- weighted averaged value vectors allows us to add more parameters / nonlinearity

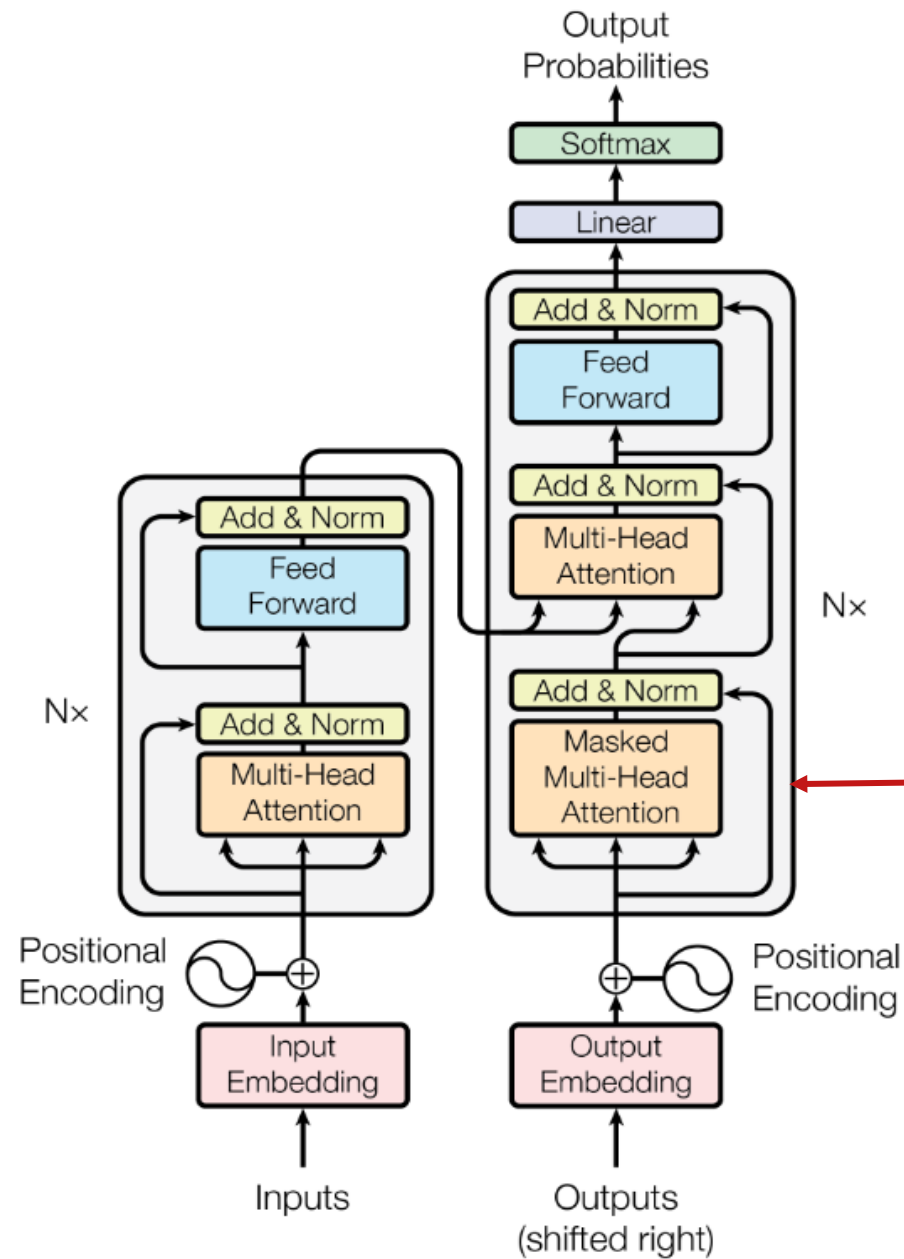


Transformer Architecture

We stack as many of these **Transformer blocks** on top of each other as we can (bigger models are generally better given enough data!)

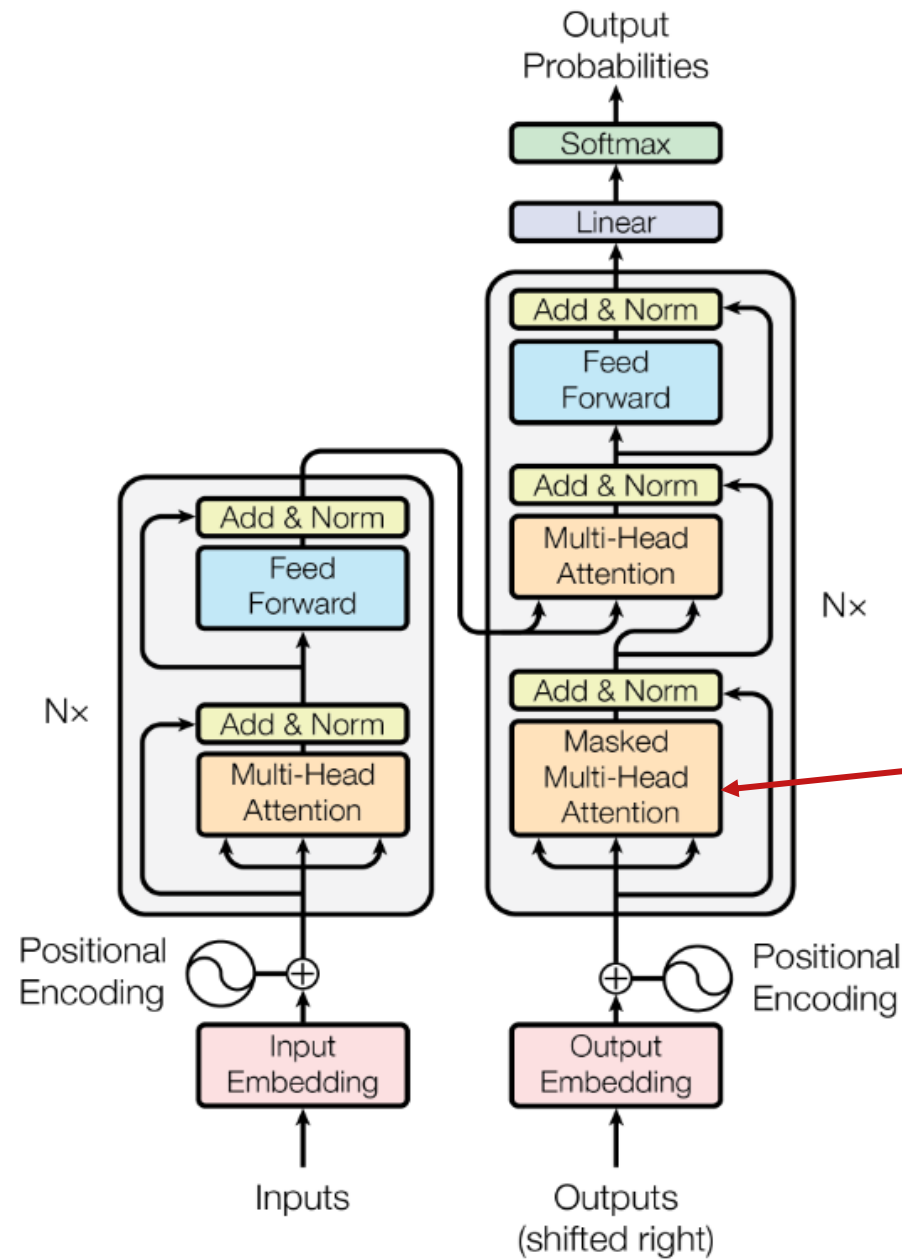


Transformer Architecture



Moving onto the decoder, which takes in English sequences that have been shifted to the right (e.g., *<START> schools opened their*)

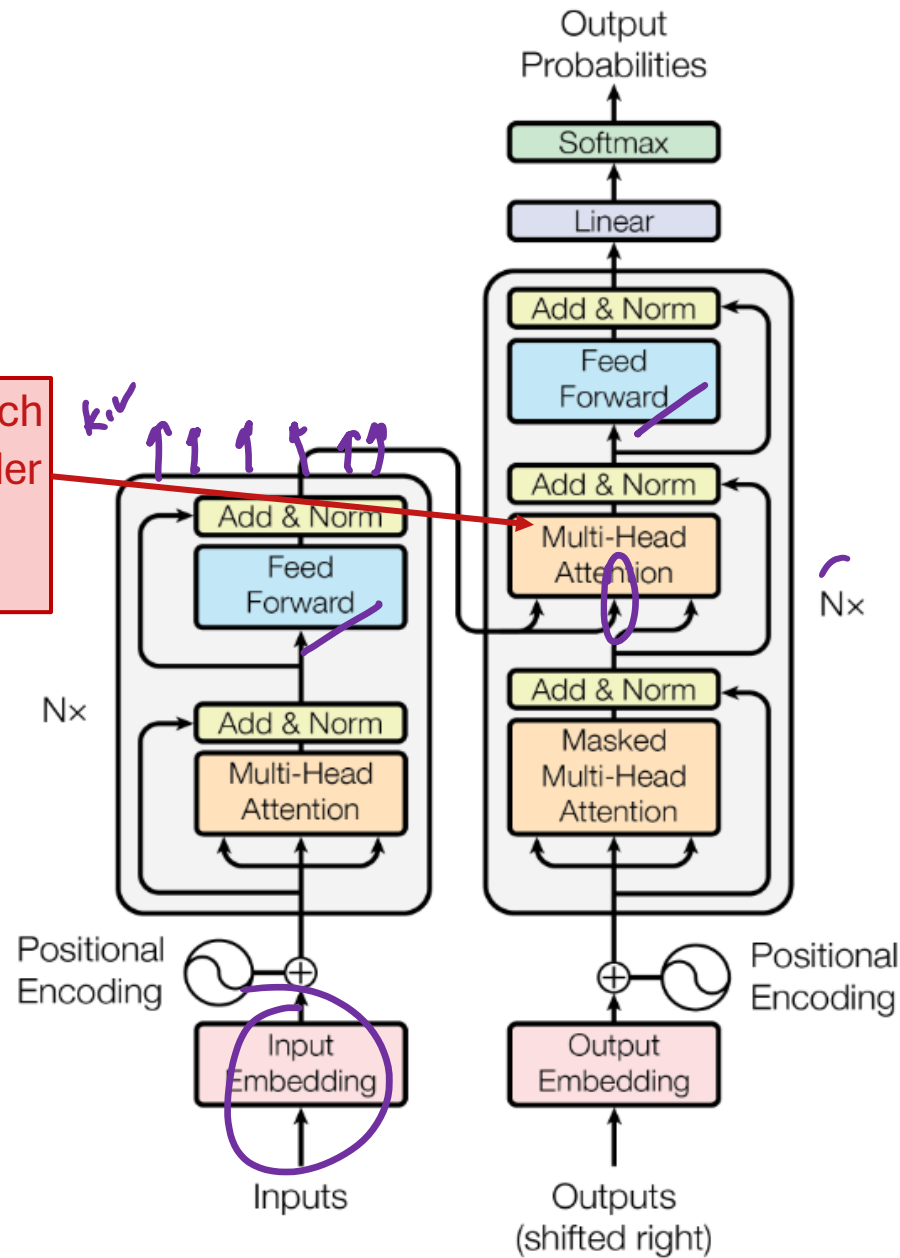
Transformer Architecture



We first have an instance of **masked self attention**. Since the decoder is responsible for predicting the English words, we need to apply masking as we saw before.

Transformer Architecture

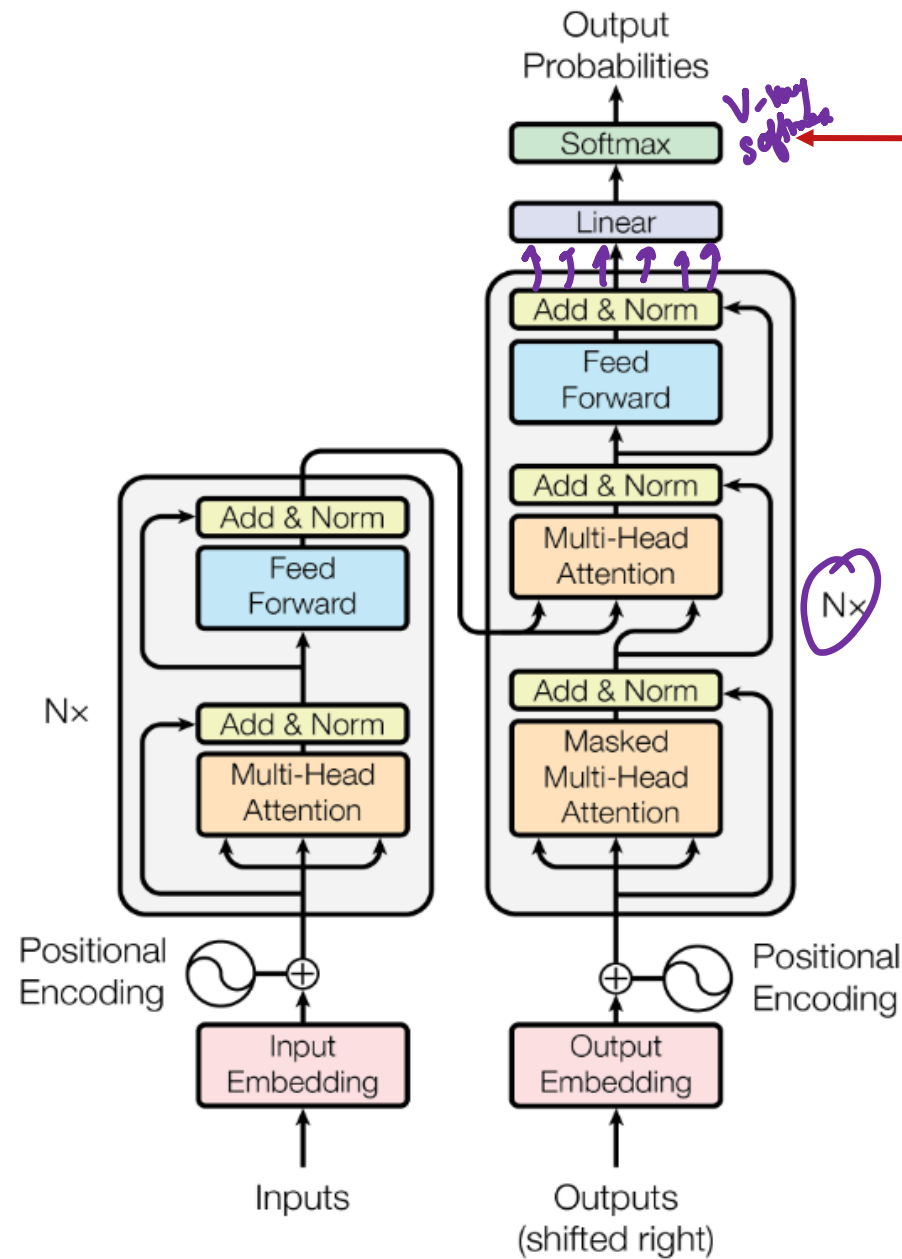
Now, we have **cross attention**, which connects the decoder to the encoder by enabling it to attend over the encoder's final hidden states.



Source of Image : Attention is all you need
(Vaswani et al., 2017)

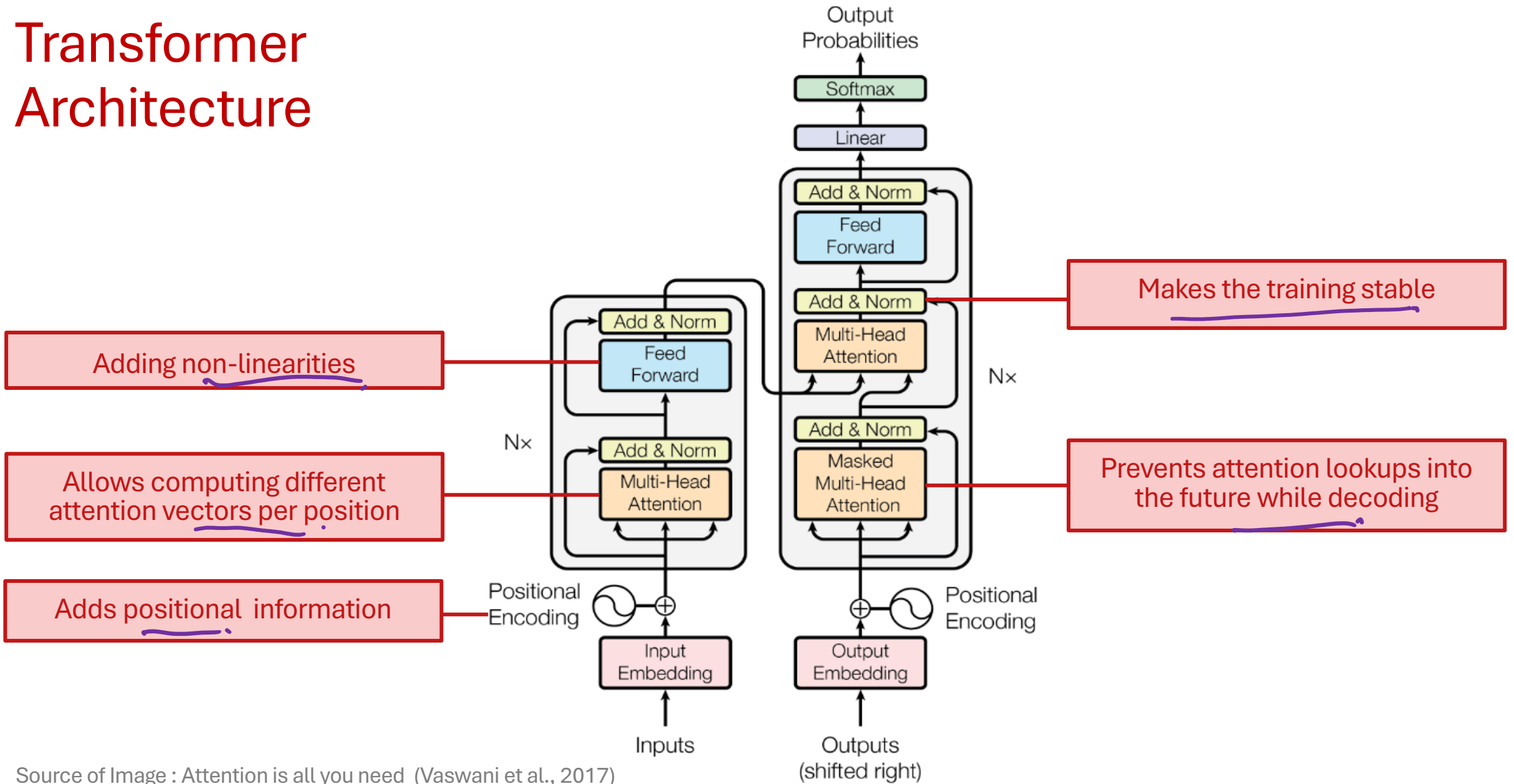


Transformer Architecture



After stacking a bunch of these decoder blocks, we finally have our familiar **softmax** layer to predict the next English word.

Transformer Architecture



Source of Image : Attention is all you need (Vaswani et al., 2017)



TITLE	CITED BY	YEAR
Attention is all you need A Vaswani, N Shazeer, N Parmar, J Uszkoreit, L Jones, AN Gomez, ... Advances in neural information processing systems 30	270698 ~ 350k	2017

(2017)

Prio
(1951)

